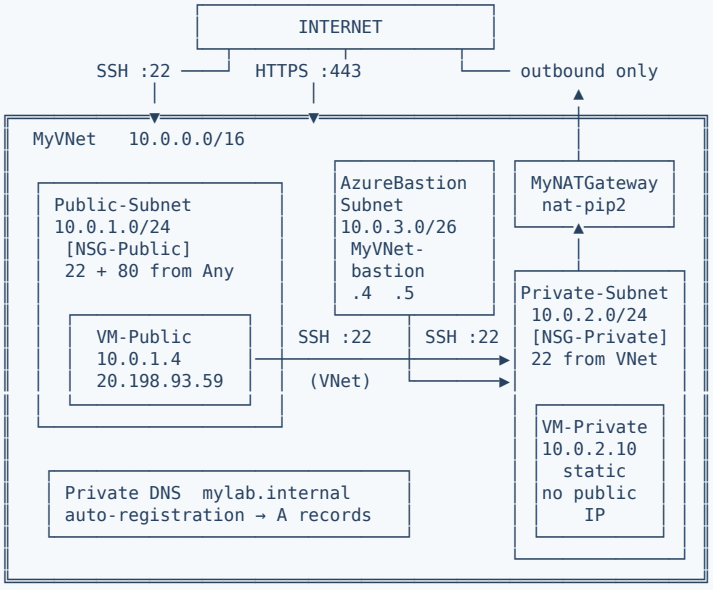
HANDS-ON LAB GUIDE

Azure Virtual Network, NSGs, NAT, Bastion & DNS

Building and proving a public/private subnet architecture



54 steps | 6 verification tests | Central India

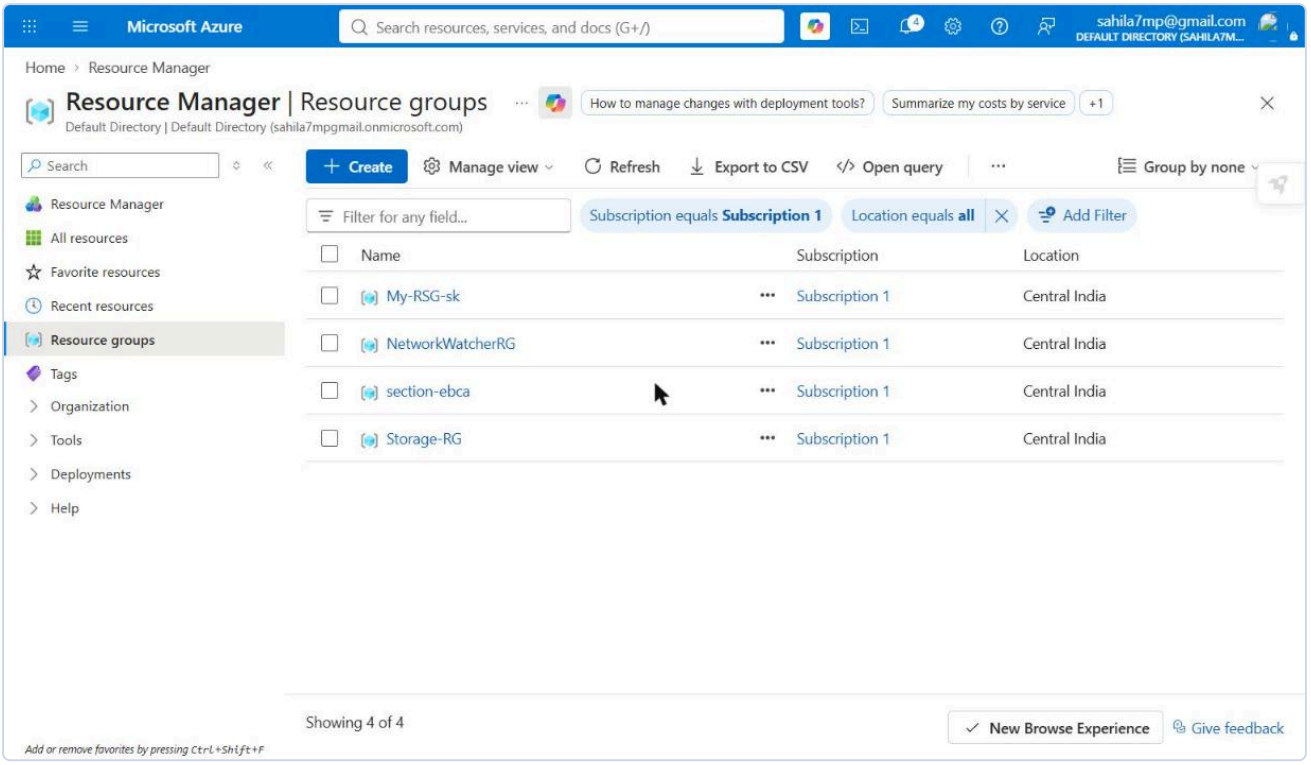
Resource group **My-RSG-sk** | October 2026

Part 1 — Foundation: Resource Group & Virtual Network

1 Create the Resource Group

WHAT WAS DONE

In the Azure Portal, opened **Resource groups** and clicked **+ Create**. Named the group `My-RSG-sk` and placed it in the **Central India** region. Every resource built in this lab — the VNet, both NSGs, both VMs and the NAT Gateway — lives inside this one group.



WHAT THE RESULT SHOWS

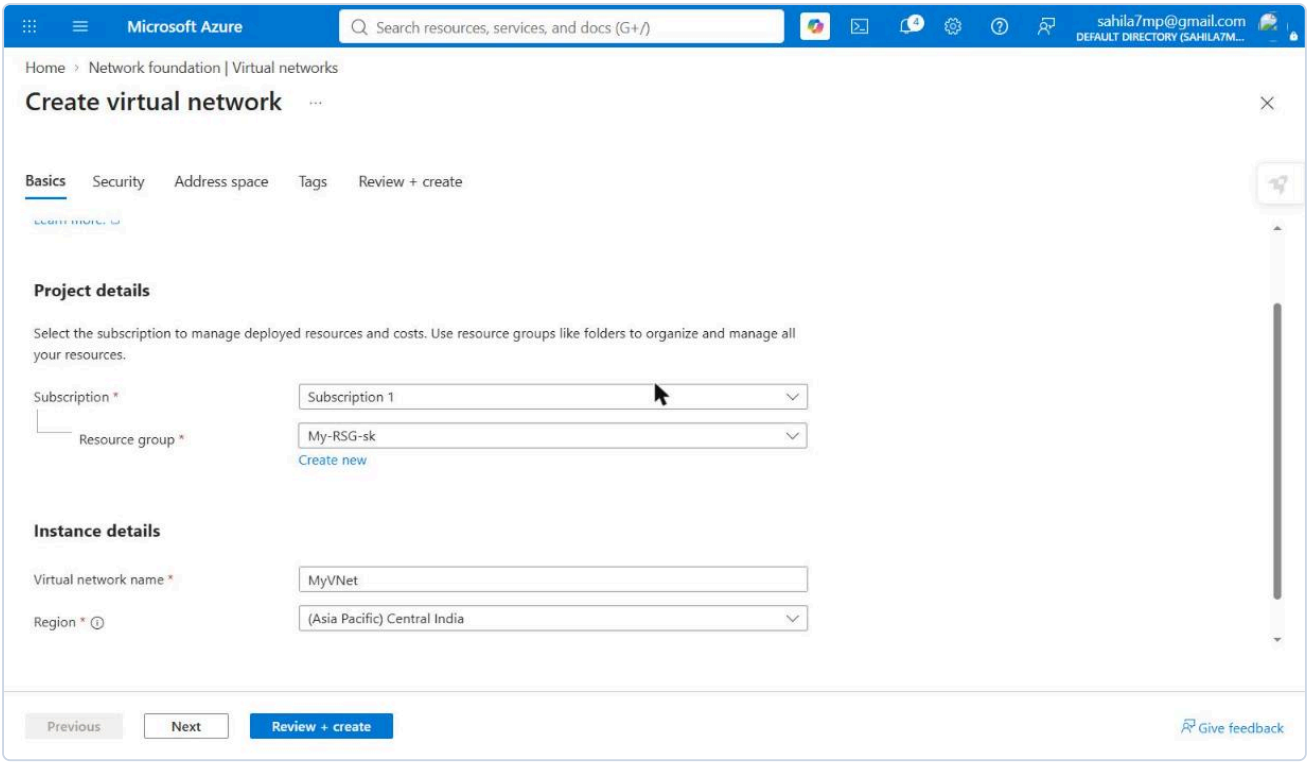
The resource group list now shows `My-RSG-sk` alongside the pre-existing groups. Showing 4 of 4 confirms the create succeeded.

> **KEY CONCEPT**
> A Resource Group is a logical container, not a network boundary. It groups resources for billing, access control (RBAC) and lifecycle — deleting the group deletes everything inside it. It does *not* restrict traffic between resources.

2 Start the Virtual Network and name it

WHAT WAS DONE

Opened **Virtual networks → + Create**. Selected subscription and the `My-RSG-sk` resource group, then corrected the auto-filled name `My-vnet` to **MyVNet**. Region set to Central India — the VNet must sit in the same region as the VMs that will use it.



WHAT THE RESULT SHOWS

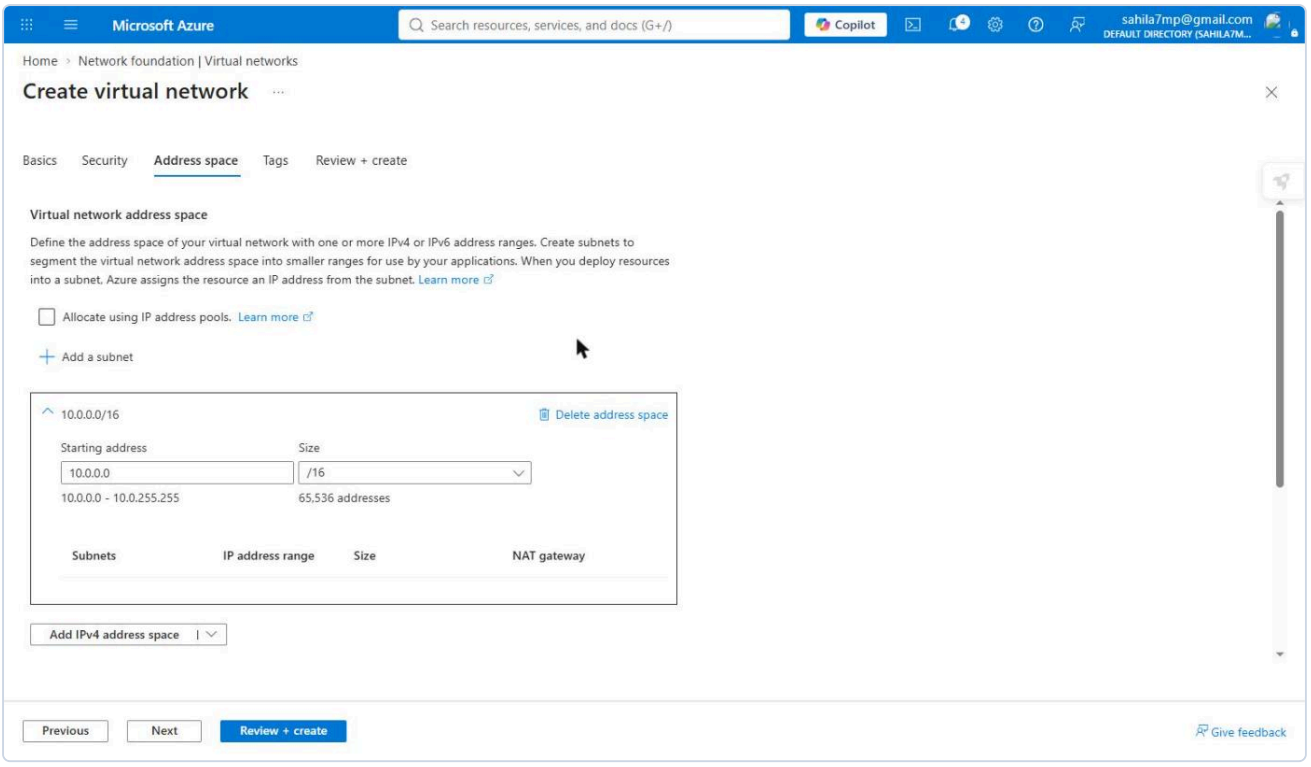
The Basics tab shows Virtual network name = `MyVNet`, Resource group = `My-RSG-sk`, Region = Central India. The form is ready to move to the Address space tab.

> **KEY CONCEPT**
>
> A VNet is region-bound. Resources in a different region cannot join it directly — they would need VNet peering. Naming matters: the VNet name is referenced later by VMs, NSGs and the NAT Gateway.

3 Define the VNet address space — 10.0.0.0/16

WHAT WAS DONE

On the **Address space** tab, set the starting address to `10.0.0.0` with size `/16`.



WHAT THE RESULT SHOWS

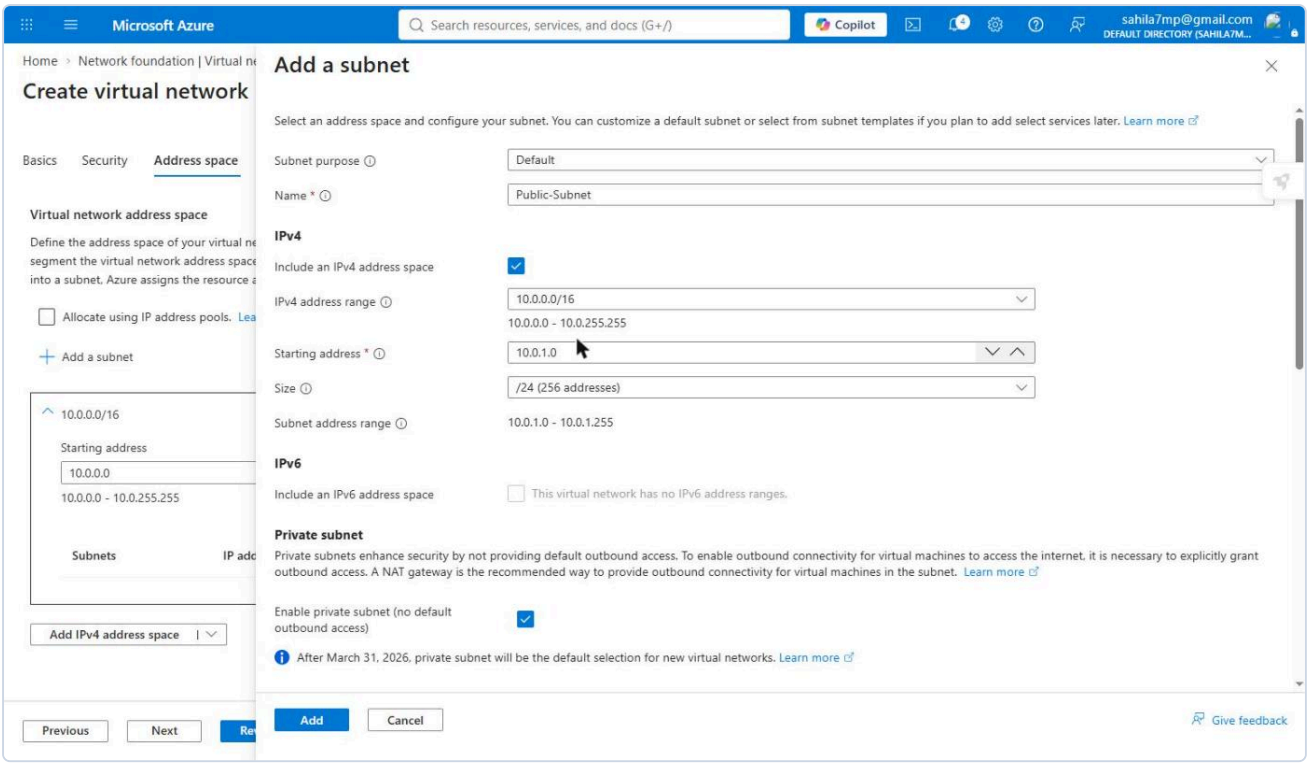
Azure confirms the range `10.0.0.0 – 10.0.255.255` and **65,536 addresses**. The Subnets table below is still empty.

> **KEY CONCEPT**
>
> CIDR maths: `/16` fixes the first 16 bits, leaving 16 host bits → 2^{16} = 65,536 addresses. This is the whole private pool for the VNet; subnets are carved out of it. RFC 1918 reserves 10.0.0.0/8, 172.16.0.0/12 and 192.168.0.0/16 for private use — none of these are routable on the public internet.

4 Add Public-Subnet — 10.0.1.0/24

WHAT WAS DONE

Clicked **+ Add a subnet**. Name `Public-Subnet`, starting address `10.0.1.0`, size `/24`. Critically, **unchecked** “Enable private subnet (no default outbound access)” — this subnet is meant to reach the internet.



WHAT THE RESULT SHOWS

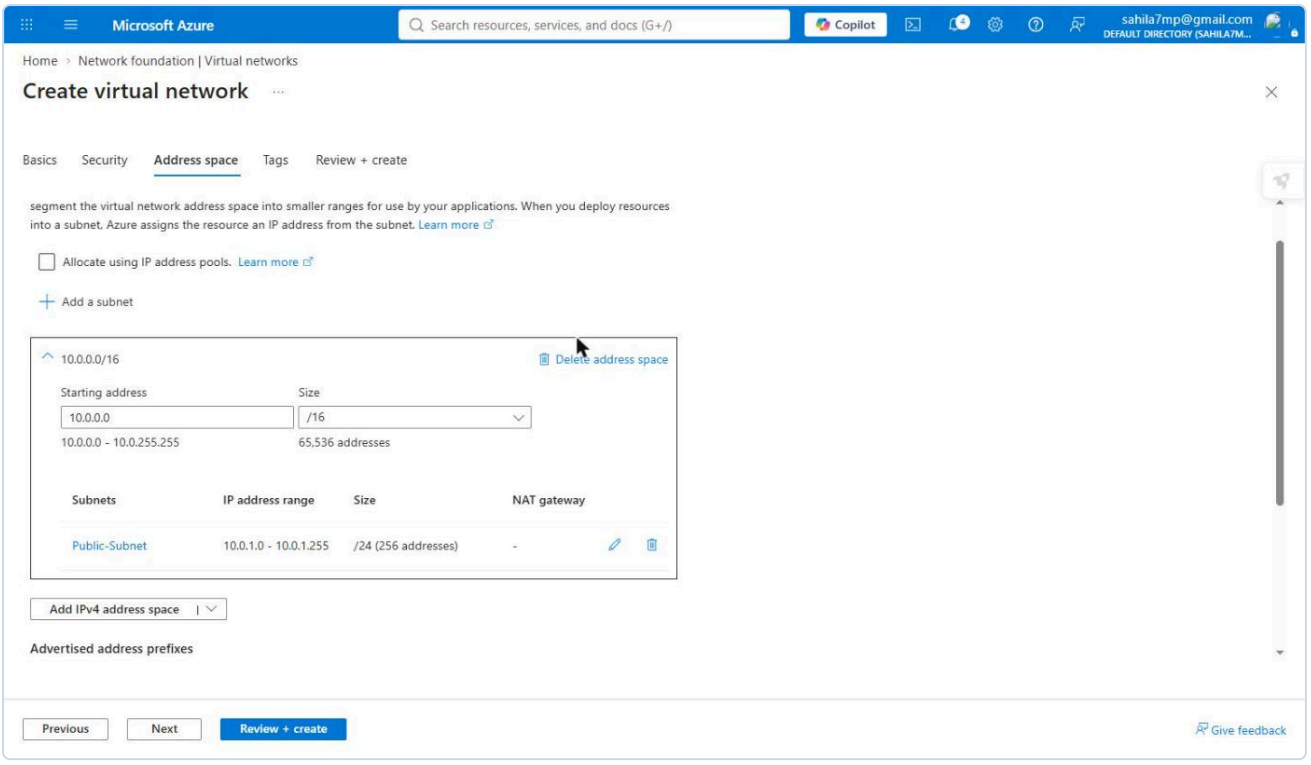
Azure calculates the subnet range as `10.0.1.0 – 10.0.1.255`, 256 addresses. The panel warns that private subnets have no default outbound access and that a NAT Gateway is the recommended way to add it — a hint of what Part 5 of this lab does.

> **KEY CONCEPT**
>
> The **“Enable private subnet”** checkbox controls *default outbound internet access*, not inbound. Leaving it unchecked keeps Azure's implicit outbound SNAT, so VMs here can reach the internet without any extra resource. Note Azure reserves 5 addresses per subnet (network, 3 Azure-internal, broadcast), so only 251 of the 256 are usable.

5 Public-Subnet registered in the address space

WHAT WAS DONE

Clicked **Add** to commit the subnet.



WHAT THE RESULT SHOWS

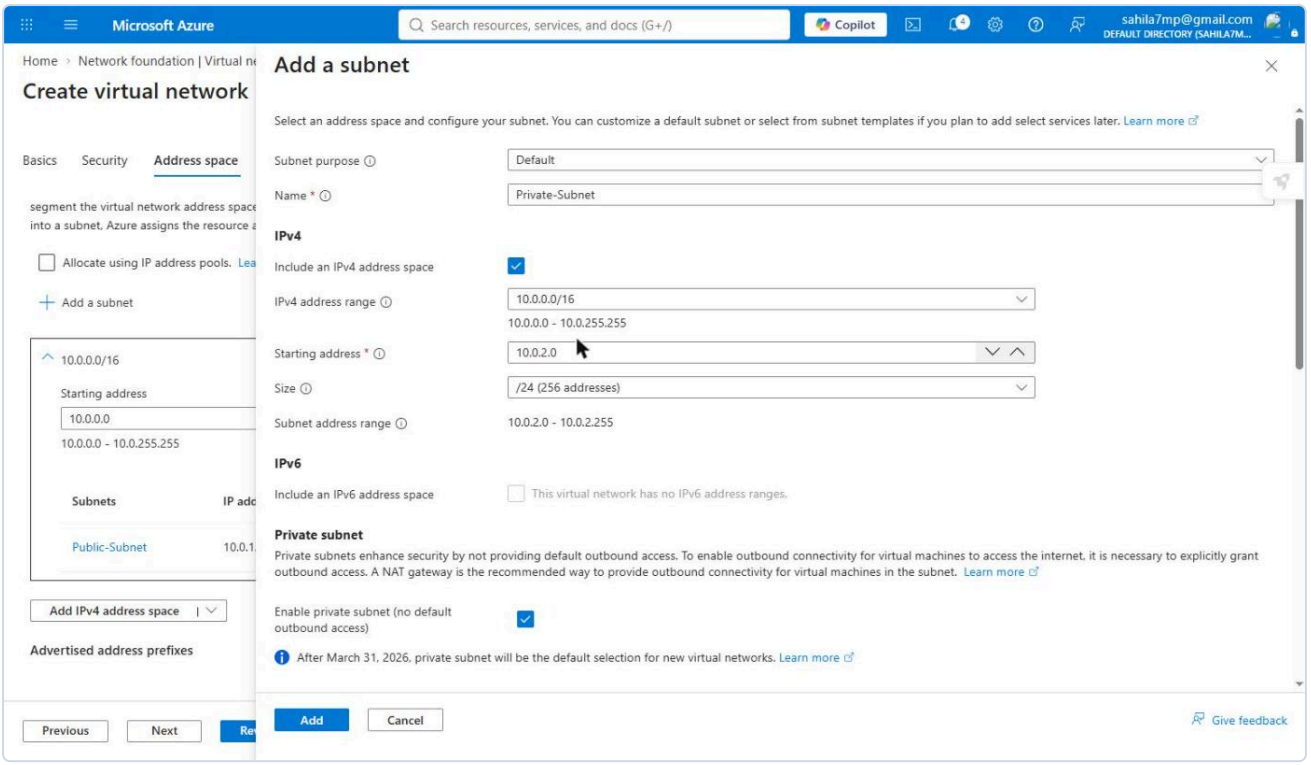
The Subnets table now lists `Public-Subnet` with range `10.0.1.0 – 10.0.1.255`, size /24. The **NAT gateway** column shows "–" — nothing attached yet.

> **KEY CONCEPT**
> That empty NAT gateway column is worth noticing now: Azure surfaces NAT attachment as a per-subnet property, which is exactly how the NAT Gateway will be bound to Private-Subnet later.

6 Add Private-Subnet — 10.0.2.0/24

WHAT WAS DONE

Added a second subnet: name `Private-Subnet`, starting address `10.0.2.0`, size `/24`. This time the **“Enable private subnet”** checkbox was deliberately **left checked**.



WHAT THE RESULT SHOWS

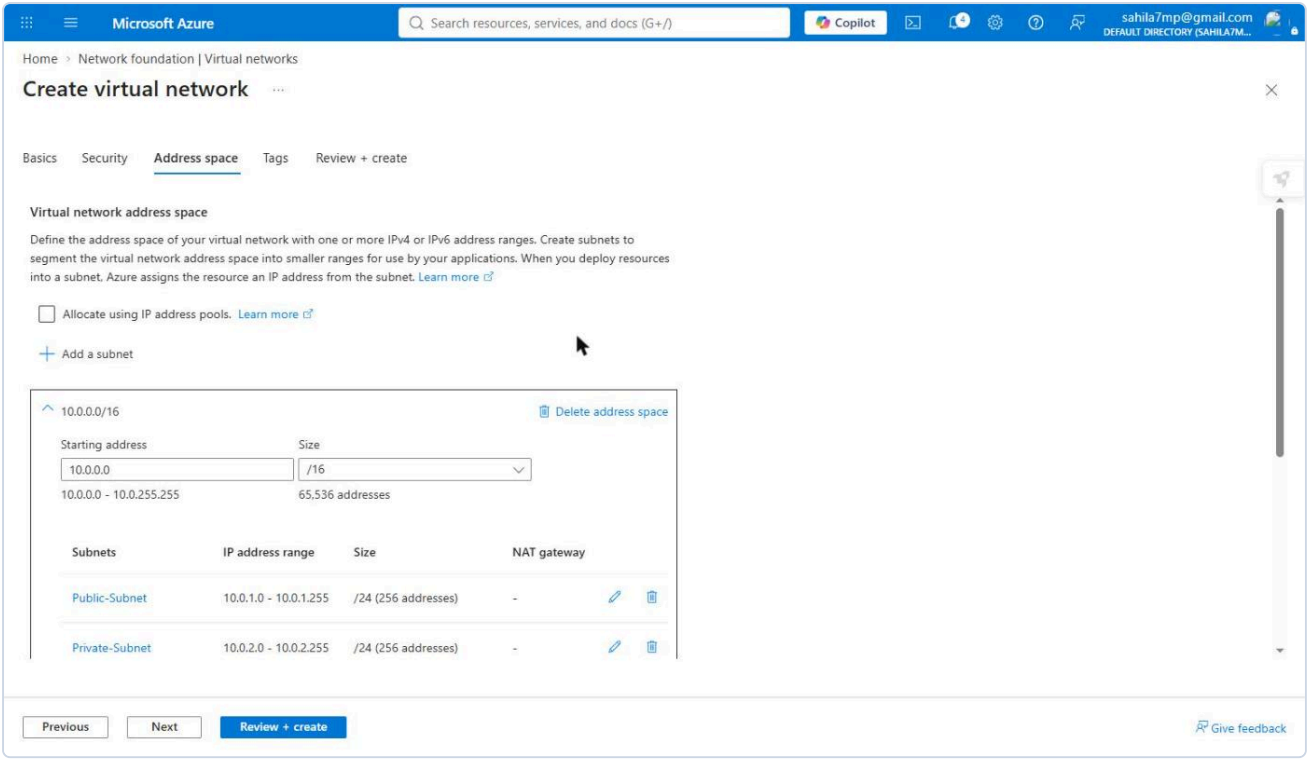
Range calculated as `10.0.2.0 – 10.0.2.255`. Public-Subnet is visible in the background list, so the two ranges are confirmed non-overlapping.

> **KEY CONCEPT**
>
> This one checkbox is the whole difference between the two subnets at the routing layer. Checked = Azure's default outbound SNAT is removed, so a VM here has *no* path to the internet until one is explicitly provided. This is the behaviour proven in Step 27 and fixed in Step 32.

7 Both subnets defined

WHAT WAS DONE

Reviewed the completed address space before creating the VNet.



WHAT THE RESULT SHOWS

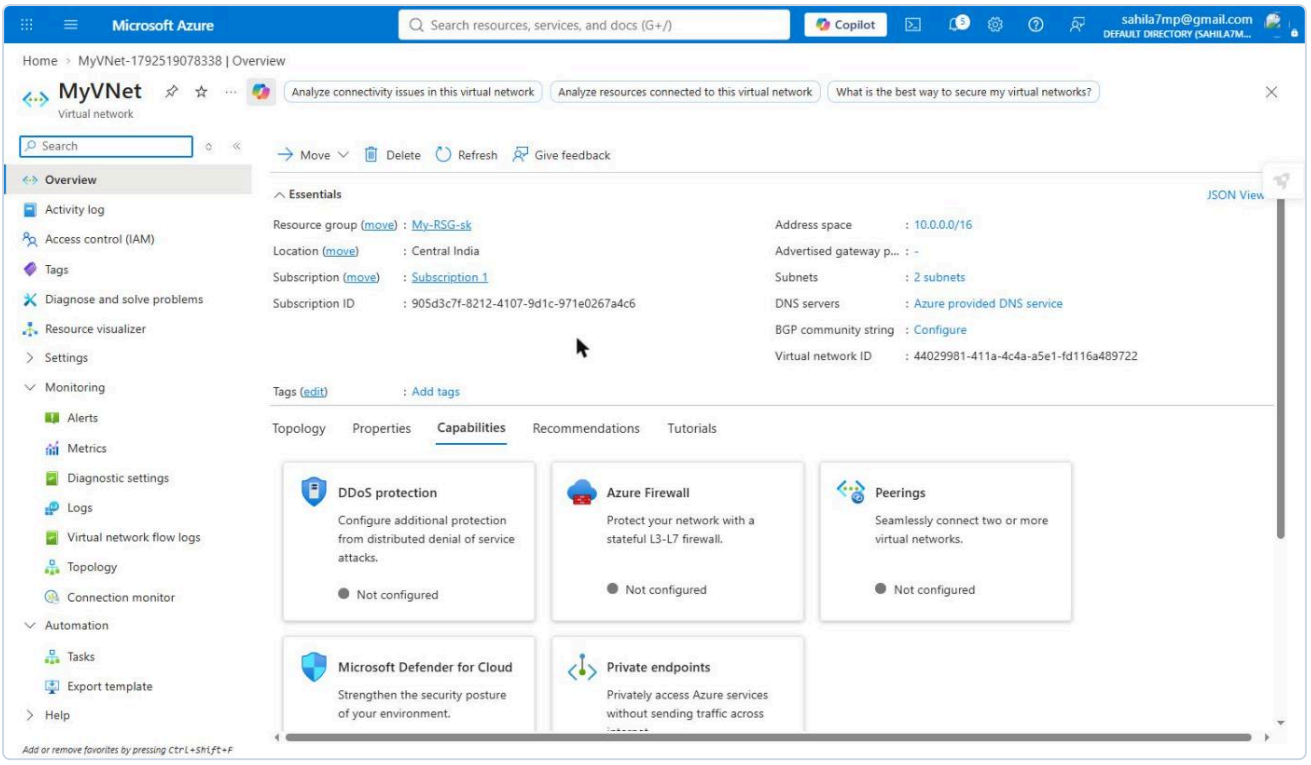
The table shows both subnets under the `10.0.0.0/16` space: `Public-Subnet` 10.0.1.0/24 and `Private-Subnet` 10.0.2.0/24, 256 addresses each.

> **KEY CONCEPT**
>
> Subnets inside a VNet must not overlap. 10.0.1.0/24 and 10.0.2.0/24 are adjacent but distinct. All subnets in a VNet can route to each other by default — the VNet is one flat routing domain, and NSGs are what introduce separation.

8 MyVNet created

WHAT WAS DONE

Clicked **Review + create** and then **Create**, then opened the new VNet's Overview blade.



WHAT THE RESULT SHOWS

MyVNet Overview confirms Resource group `My-RSG-sk`, Location Central India, Address space `10.0.0.0/16` and **2 subnets**. DDoS protection, Azure Firewall and Peerings all show “Not configured” — this is a bare VNet.

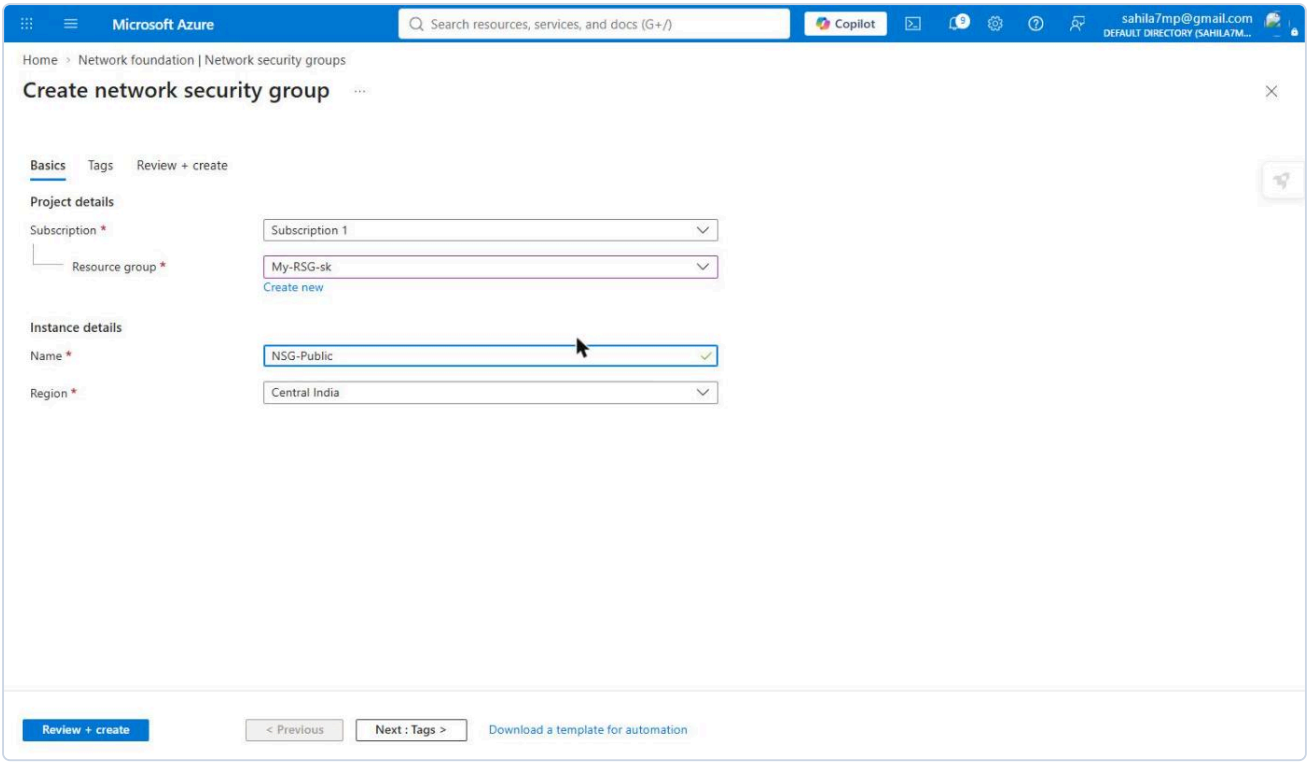
> **KEY CONCEPT**
>
> A freshly created VNet has no firewall of its own. Traffic control comes from NSGs attached to subnets or NICs. Azure Firewall (shown here as not configured) is a separate paid service for L3–L7 inspection; NSGs are free and operate at L3/L4.

Part 2 — Network Security Groups

9 Create NSG-Public

WHAT WAS DONE

Opened **Network security groups → + Create**. Name `NSG-Public`, resource group `My-RSG-sk`, region Central India.



WHAT THE RESULT SHOWS

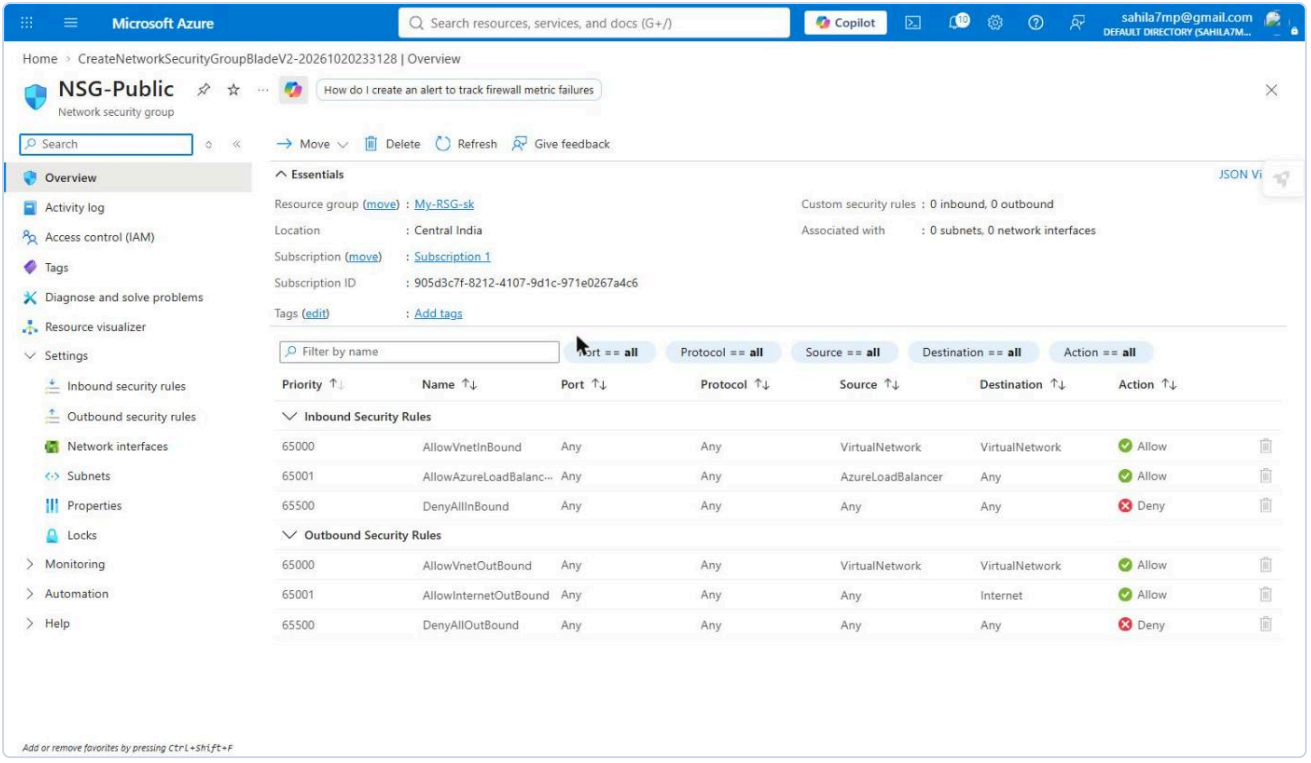
The Basics tab is filled and validated — the green tick next to the name confirms it is unique within the resource group.

> **KEY CONCEPT**
> An NSG is a stateful packet filter made of prioritised allow/deny rules. It is a standalone resource: it is created on its own, then *associated* with one or more subnets or NICs. One NSG can protect many subnets.

10 NSG-Public created — read the default rules

WHAT WAS DONE

Opened the new NSG's Overview to inspect what Azure pre-populates.



WHAT THE RESULT SHOWS

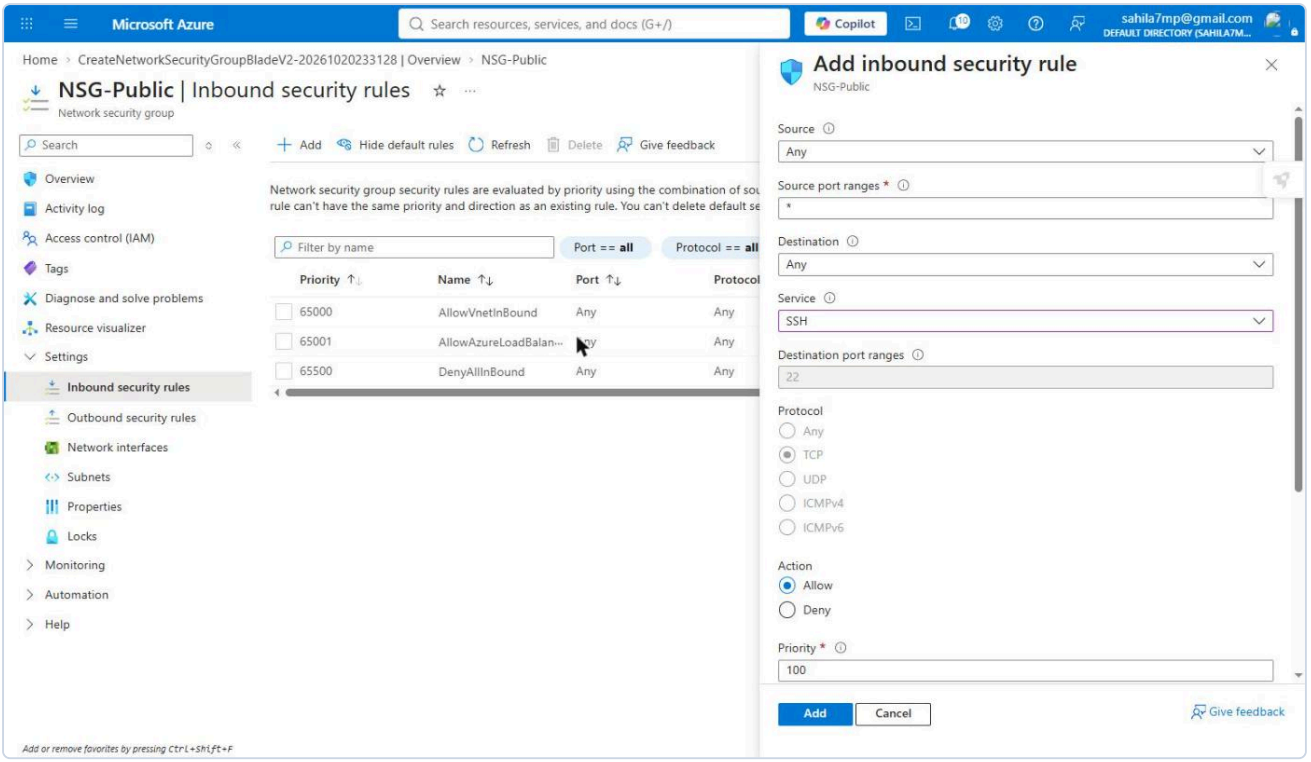
Custom security rules: **0 inbound, 0 outbound**. Associated with **0 subnets, 0 network interfaces**. Six default rules are present:
Inbound — 65000 AllowVnetInBound, 65001 AllowAzureLoadBalancerInBound, **65500 DenyAllInBound**
Outbound — 65000 AllowVnetOutBound, 65001 AllowInternetOutBound, 65500 DenyAllOutBound

> **KEY CONCEPT**
> This screen explains the entire NSG security model. Rules are evaluated **lowest priority number first**, and the first match wins. Because `DenyAllInBound` sits at 65500, anything from the internet is blocked unless a *lower-numbered* rule allows it. Conversely `AllowInternetOutBound` at 65001 beats the outbound deny — so **outbound is open by default, inbound is closed by default**. Custom rules must use priorities 100–4096.

11 Add the SSH inbound rule

WHAT WAS DONE

Went to **Inbound security rules → + Add**. Chose Service = **SSH**, which auto-filled destination port `22` and protocol **TCP**. Source = `Any`, Destination = `Any`, Action = **Allow**, Priority = **100**, Name = `Allow-SSH`.



WHAT THE RESULT SHOWS

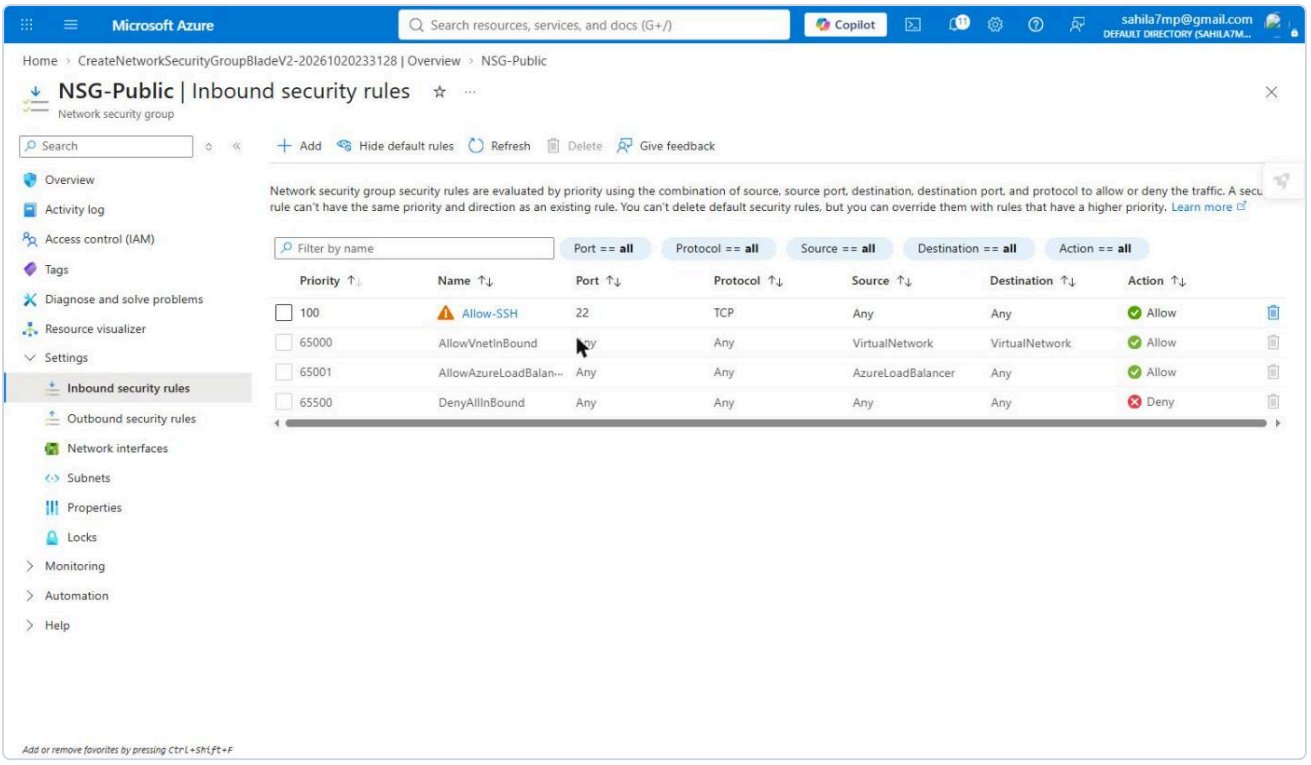
The rule form is complete with port 22 / TCP / Allow / priority 100.

> **KEY CONCEPT**
>
> Picking a **Service** from the dropdown is a shortcut — Azure fills in the well-known port and protocol for you. Priority 100 was chosen deliberately: it is far below 65500, so this allow is evaluated long before the implicit DenyAllInBound.

12 Allow-SSH is live

WHAT WAS DONE

Clicked **Add** and reviewed the rule list.



WHAT THE RESULT SHOWS

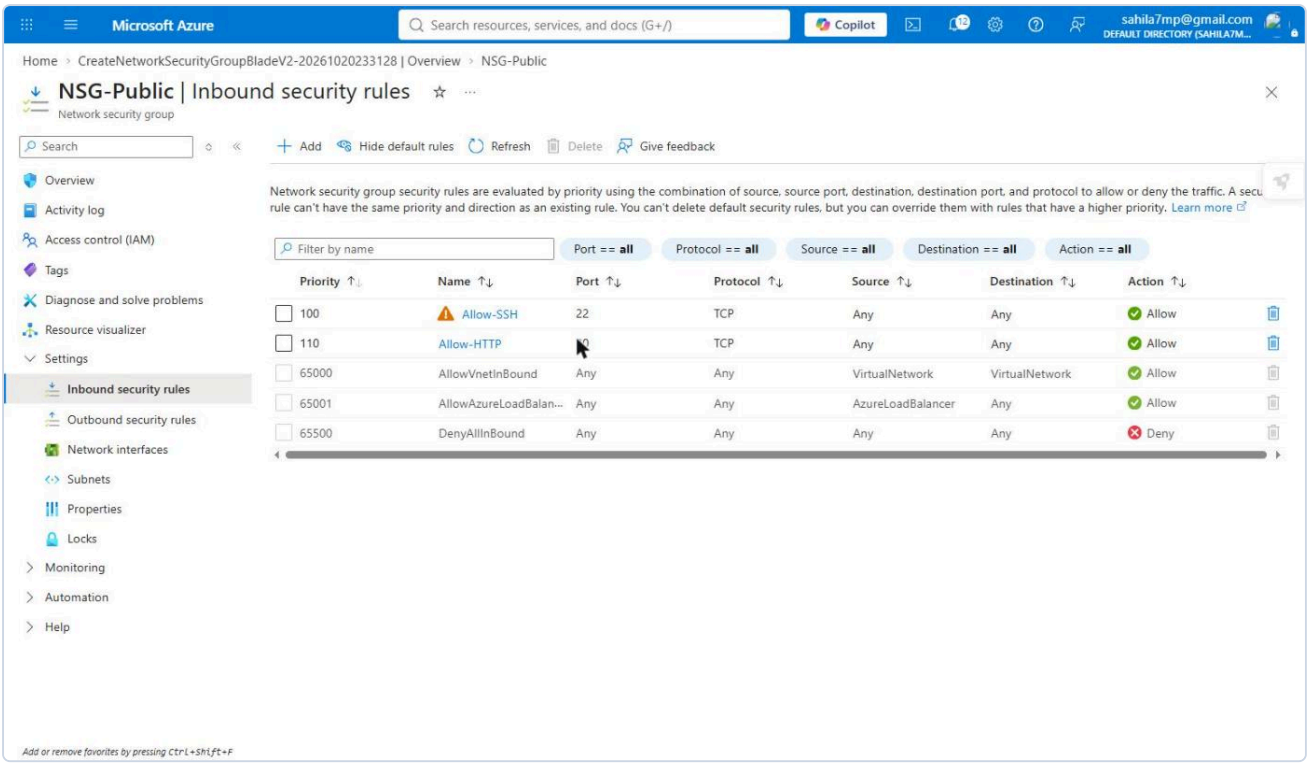
`Allow-SSH` now sits at the top of the list: priority **100**, port **22**, protocol TCP, source Any, destination Any, action **Allow** — above all four default rules.

> **KEY CONCEPT**
>
> The warning triangle beside the rule is Azure flagging that SSH is exposed to the entire internet (source = Any). That is acceptable for a lab, but in production the source should be narrowed to a specific IP or CIDR, or SSH should be replaced by Azure Bastion.

13 Add the HTTP rule — NSG-Public complete

WHAT WAS DONE

Repeated the process for web traffic: Service = **HTTP** (port 80, TCP), Source Any, Action Allow, Priority **110**, Name `Allow-HTTP`.



WHAT THE RESULT SHOWS

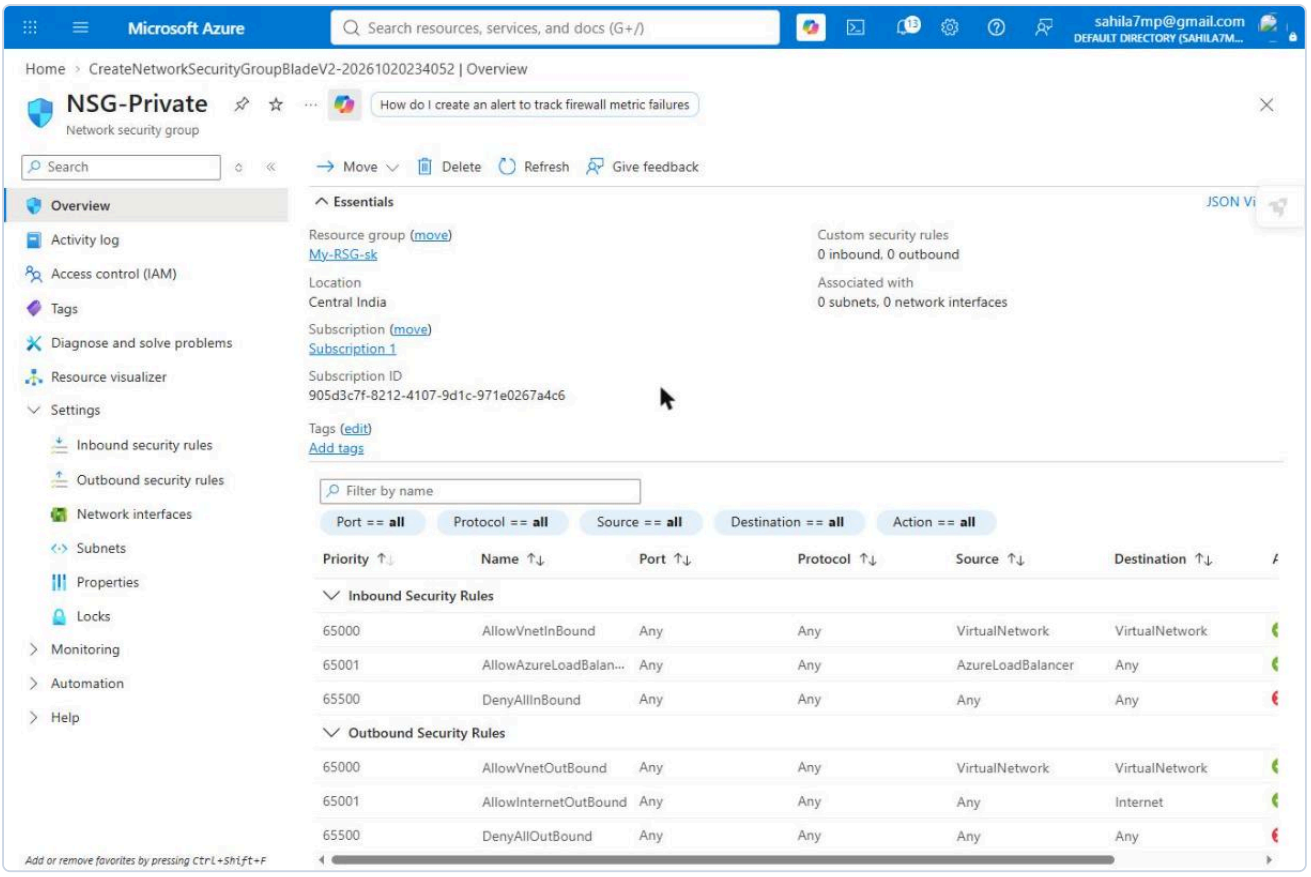
Two custom rules are now active — `Allow-SSH` at 100 and `Allow-HTTP` at 110 — sitting above the three inbound defaults.

> **KEY CONCEPT**
>
> **NSGs are stateful.** Allowing inbound TCP/22 automatically permits the return traffic of that established session outbound — no matching outbound rule is needed. This is why a working SSH or web server only requires inbound rules, and it is the single most commonly misunderstood point about NSGs.

14 Create NSG-Private

WHAT WAS DONE

Created a second NSG named `NSG-Private` in the same resource group and region.



WHAT THE RESULT SHOWS

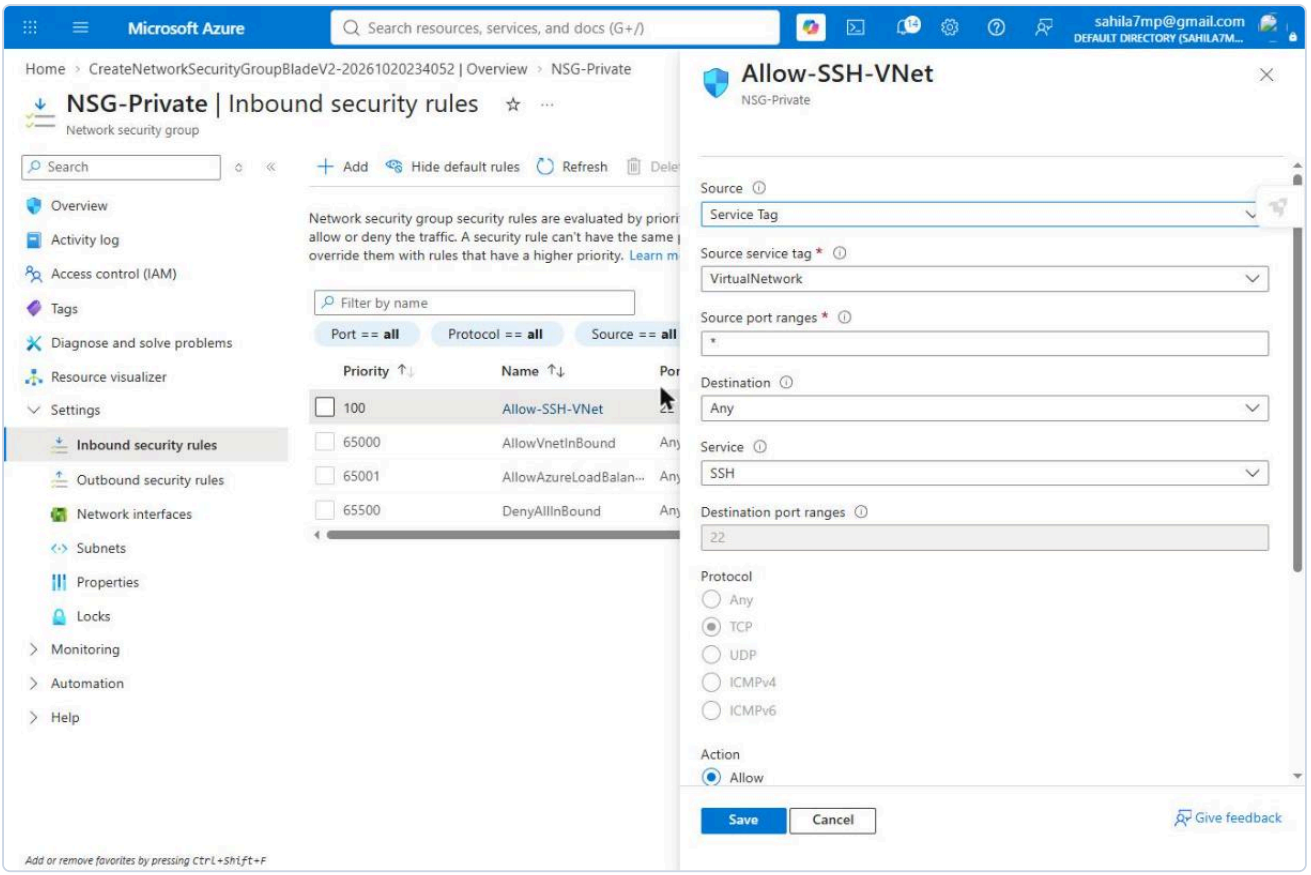
Overview shows 0 custom rules and 0 associations, with the same six Azure defaults as NSG-Public.

> **KEY CONCEPT**
>
> Two separate NSGs are used rather than one shared NSG because the two subnets need genuinely different inbound policies. An NSG is the unit of policy — share one only where the policy is identical.

15 Allow SSH only from inside the VNet

WHAT WAS DONE

Added one inbound rule to NSG-Private. Instead of leaving Source = Any, set Source = **Service Tag** and Source service tag = **VirtualNetwork**. Service = SSH (port 22, TCP), Destination = Any, Action = Allow, Priority 100, Name `Allow-SSH-VNet`.



WHAT THE RESULT SHOWS

The rule panel shows Source: Service Tag → VirtualNetwork, Destination: Any, Port 22, TCP, Allow.

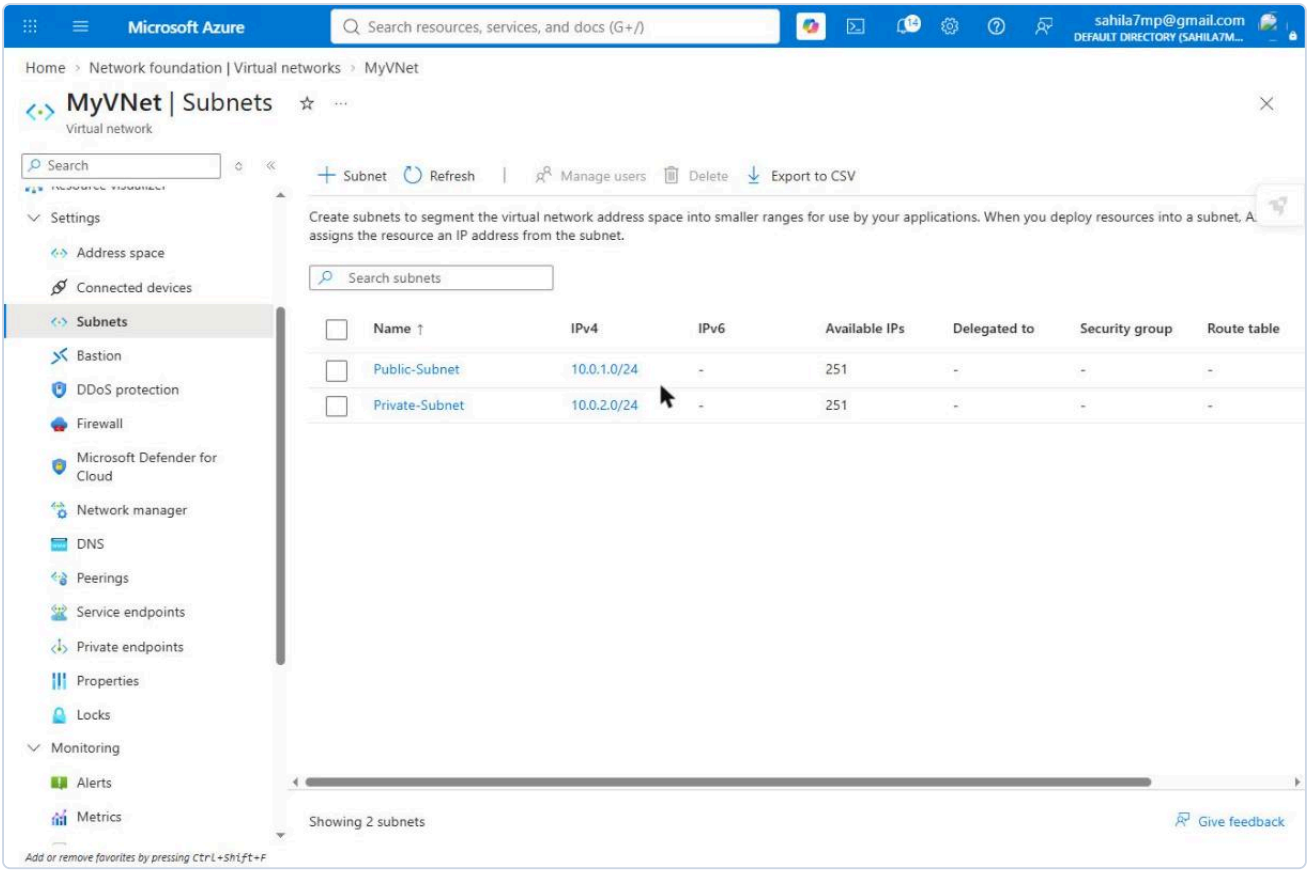
> **KEY CONCEPT**
>
> **Destination = Any looks permissive but is not the control here — Source is.** The `VirtualNetwork` service tag resolves to the VNet's own address space (10.0.0.0/16) plus peered networks. A packet arriving from the public internet has a source address outside that range, so it cannot match this rule and falls through to DenyAllInBound. Service tags are Microsoft-maintained named IP sets — they update automatically, so you never hard-code ranges.

Part 3 — Attaching NSGs to Subnets

16 Troubleshooting: the wrong path

WHAT WAS DONE

First attempt went via **MyVNet → Subnets**, expecting to attach the NSGs from there.



WHAT THE RESULT SHOWS

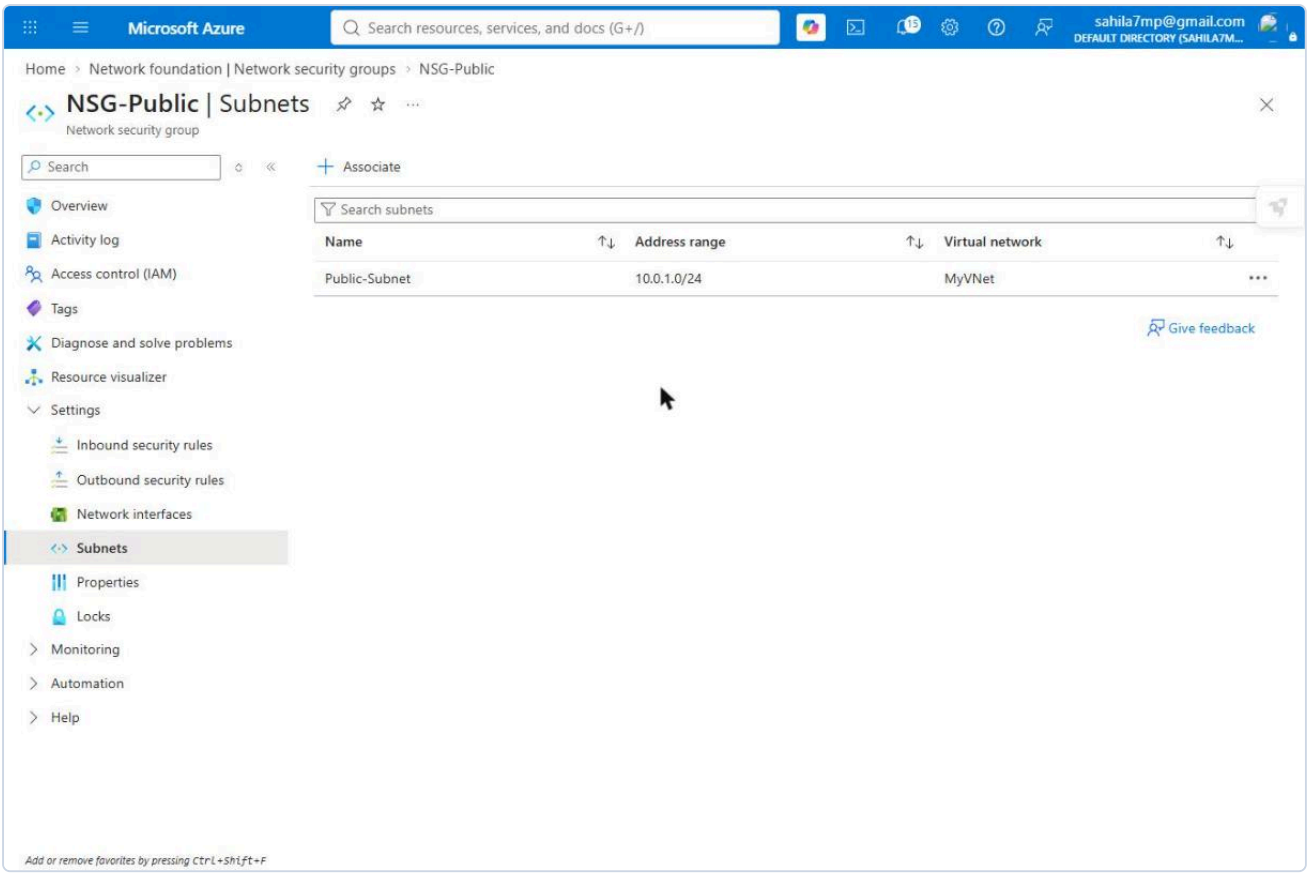
The subnet list shows both subnets with 251 available IPs each — but the **Security group** column reads "–" for both. Nothing was attached.

> **KEY CONCEPT**
>
> This is a real mistake worth keeping in the guide. The VNet's Subnets blade is read-mostly for NSG association. The reliable path is **NSG → Subnets → + Associate**, which is where Azure exposes the write operation. Always verify an association by checking the Security group column afterwards rather than assuming a click worked.

17 Associate NSG-Public with Public-Subnet

WHAT WAS DONE

Navigated to **NSG-Public → Subnets → + Associate**, selected virtual network `MyVNet` and subnet `Public-Subnet`, then saved.



WHAT THE RESULT SHOWS

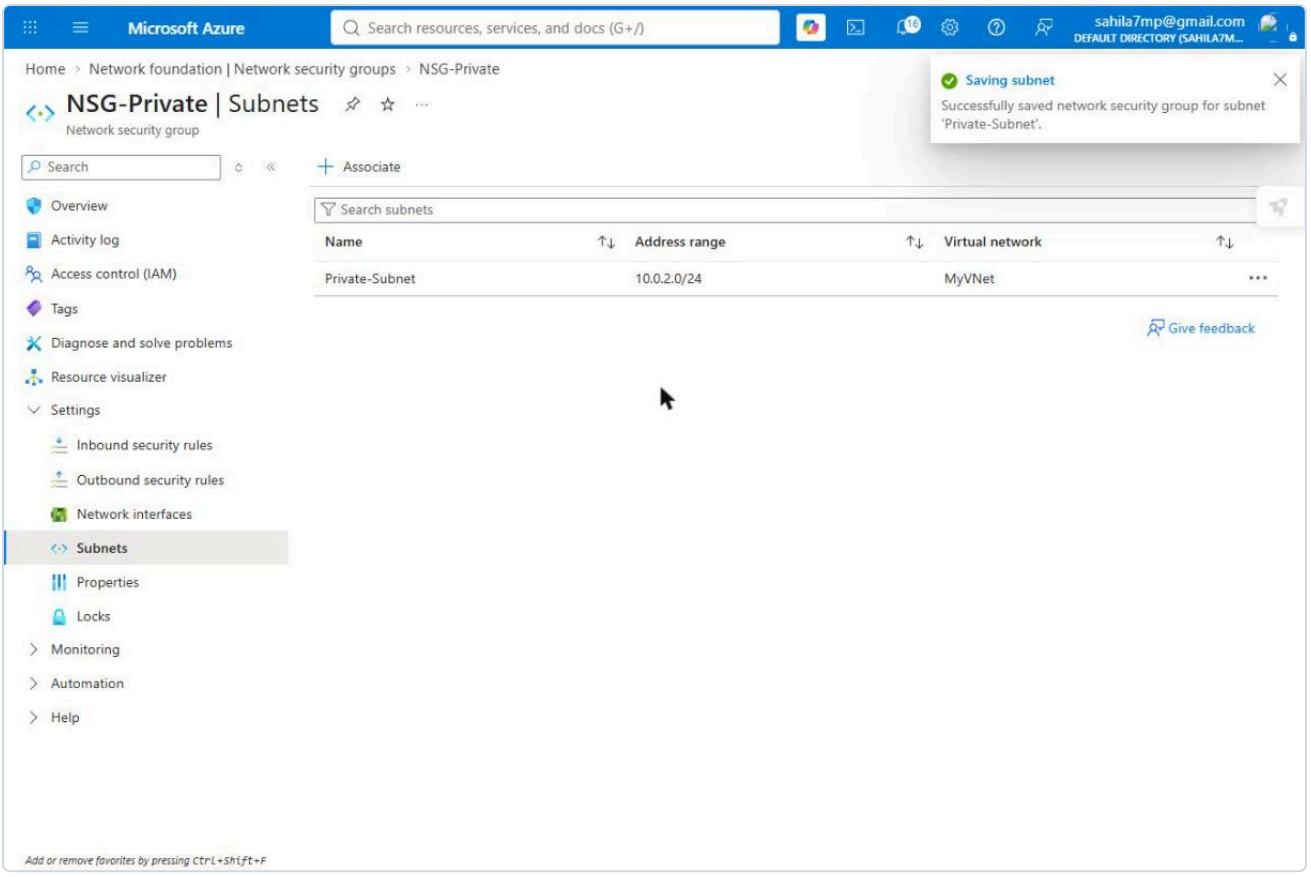
NSG-Public's Subnets blade now lists `Public-Subnet`, address range `10.0.1.0/24`, virtual network `MyVNet`.

> **KEY CONCEPT**
>
> A subnet can have **at most one** NSG. An NSG can serve many subnets. Attaching at subnet level means every NIC placed in that subnet inherits the rules automatically — new VMs are protected the moment they are deployed, with no per-VM configuration.

18 Associate NSG-Private with Private-Subnet

WHAT WAS DONE

Same path on the other NSG: **NSG-Private → Subnets → + Associate →** `Private-Subnet`.



WHAT THE RESULT SHOWS

Green toast: *"Successfully saved network security group for subnet 'Private-Subnet'."* The subnet is listed with range `10.0.2.0/24`.

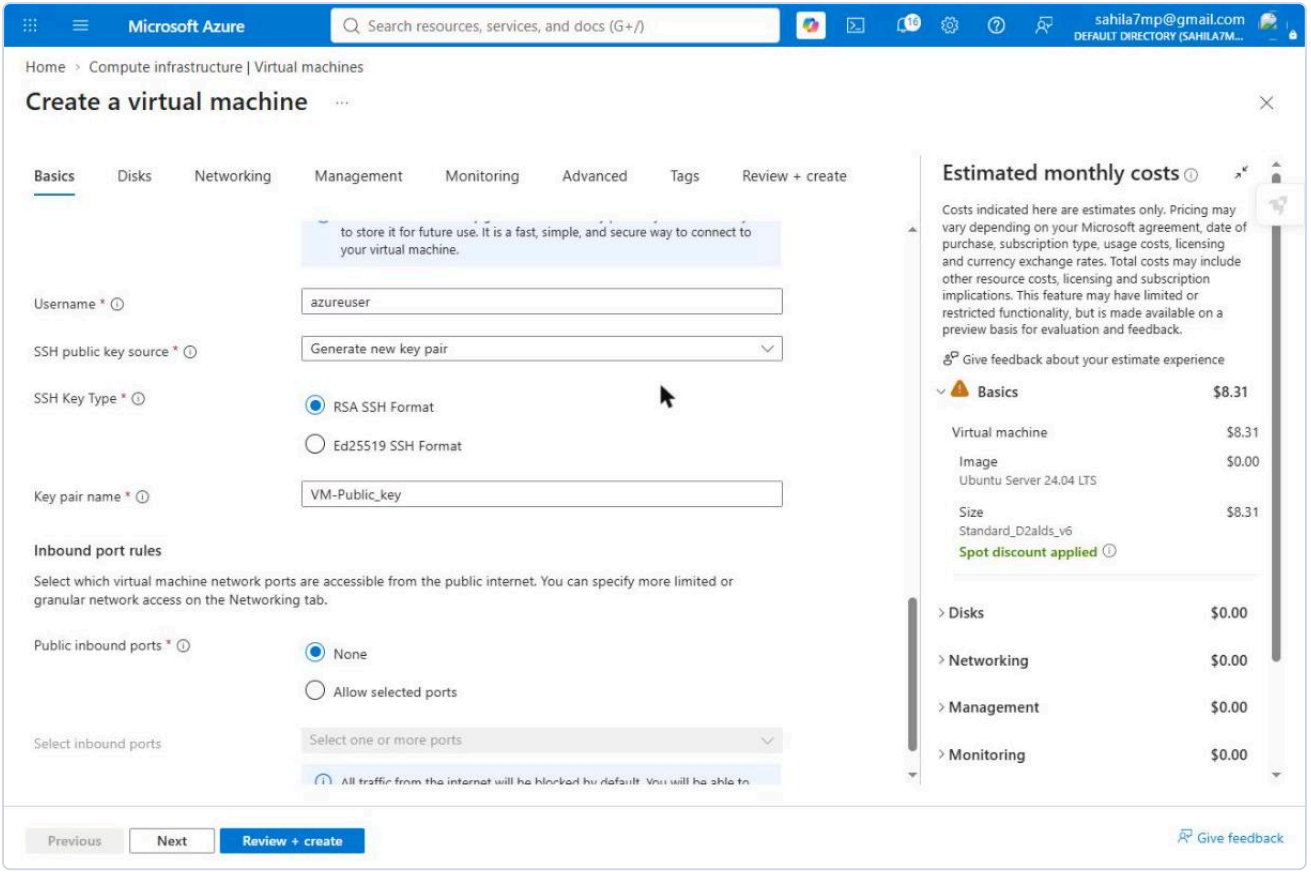
> **KEY CONCEPT**
>
> Both halves of the design are now enforced before a single VM exists. Building the network and its policy first, then dropping compute into it, is the order that scales — the alternative (securing each VM after deployment) leaves a window where the VM is unprotected.

Part 4 — Deploying the VMs

19 VM-Public — set inbound port rules to None

WHAT WAS DONE

Started **Virtual machines → + Create**. Name `VM-Public`, resource group `My-RSG-sk`, Ubuntu Server image, authentication by SSH public key with username `azureuser` and a generated key pair `VM-Public_key`. Under **Inbound port rules**, selected **None**.



WHAT THE RESULT SHOWS

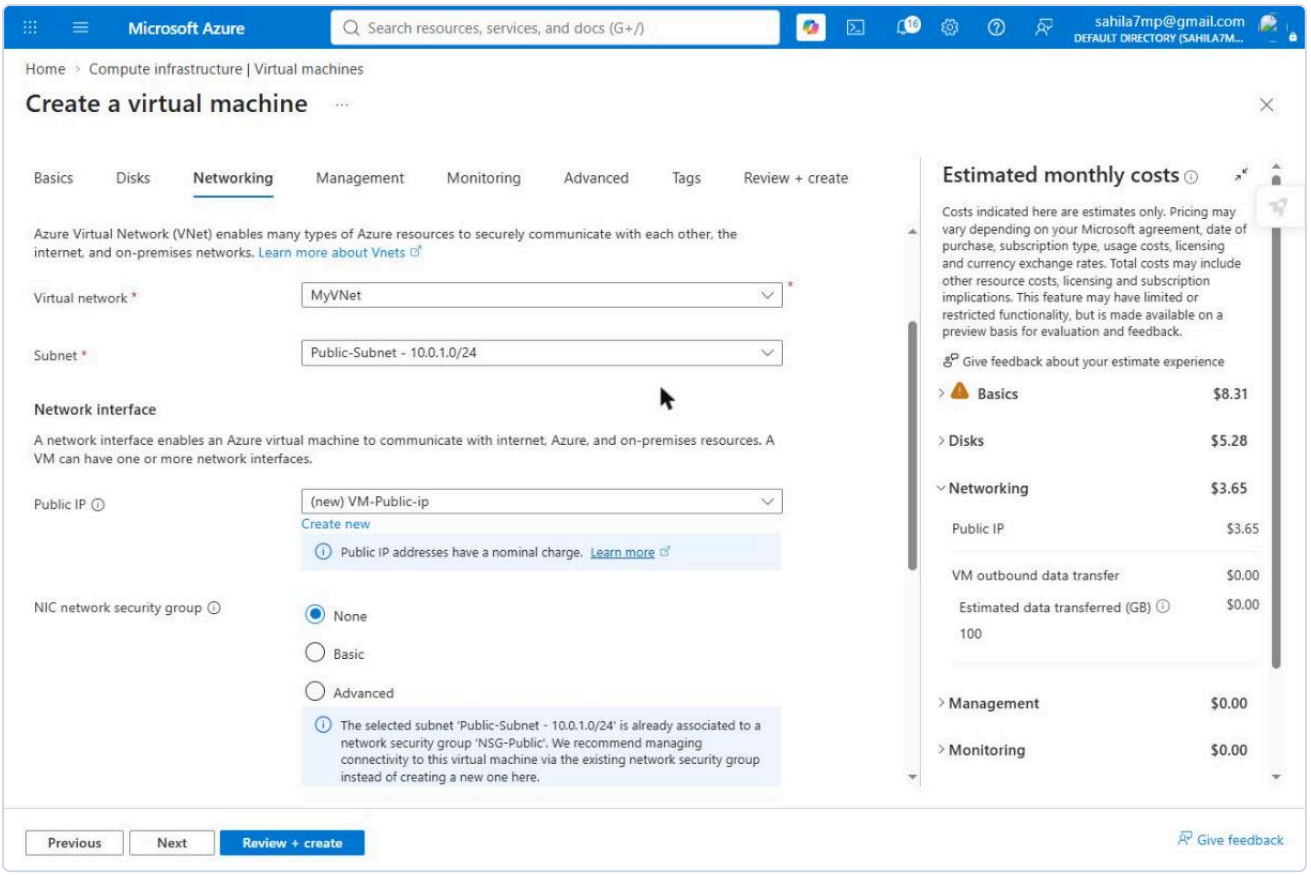
Public inbound ports = None. The earlier “This will allow all IP addresses to access your virtual machine” warning disappears.

> **KEY CONCEPT**
>
> This section of the VM wizard is a shortcut that creates a **brand-new NSG attached to the VM's NIC** — a different layer from the subnet NSG already built. Leaving it on would stack two NSGs on the same traffic path, where *both* must allow a packet for it to pass. Since NSG-Public already permits 22 and 80 at subnet level, None is the clean choice.

20 VM-Public networking — Azure confirms the design

WHAT WAS DONE

Moved to the **Networking** tab and verified the placement: Virtual network `MyVNet`, Subnet `Public-Subnet - 10.0.1.0/24`, Public IP `(new) VM-Public-ip`, NIC network security group **None**.



WHAT THE RESULT SHOWS

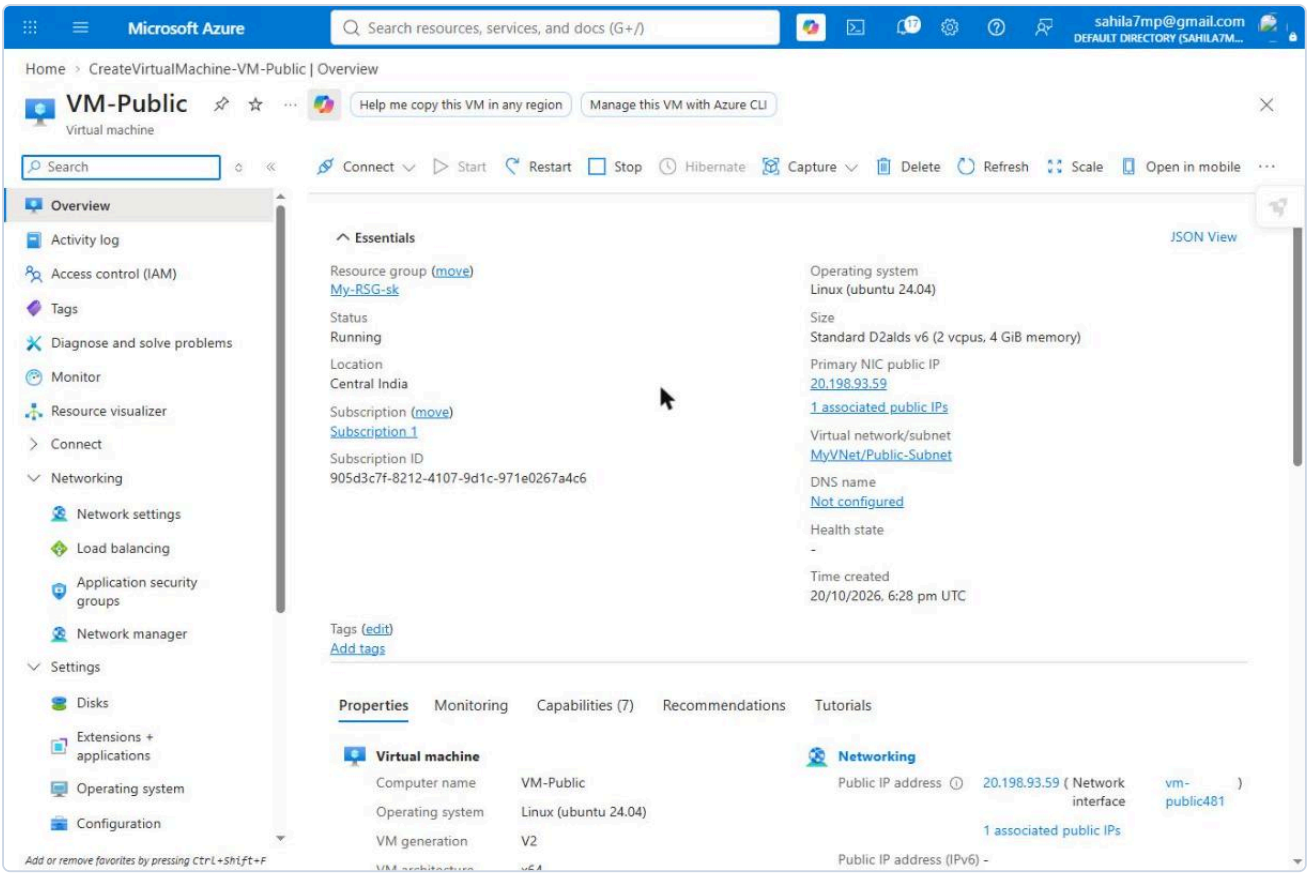
Azure displays an advisory box: *“The selected subnet 'Public-Subnet - 10.0.1.0/24' is already associated to a network security group 'NSG-Public'. We recommend managing connectivity to this virtual machine via the existing network security group instead of creating a new one here.”*

> **KEY CONCEPT**
>
> Azure itself endorses the subnet-level approach. Worth reading as a design principle: put policy at the broadest layer that expresses it correctly, and use NIC-level NSGs only for genuine per-VM exceptions.

21 VM-Public is running

WHAT WAS DONE

Completed **Review + create → Create**, downloaded the private key when prompted, and opened the VM's Overview.



WHAT THE RESULT SHOWS

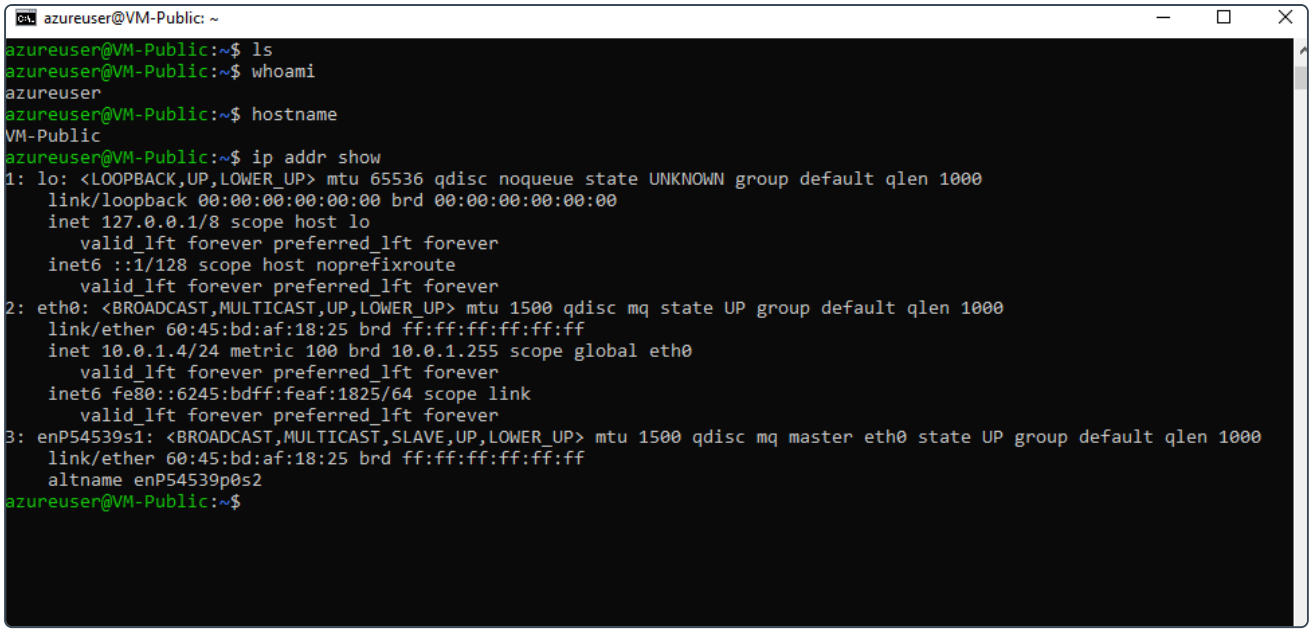
Status **Running**. Virtual network/subnet `MyVNet/Public-Subnet`. Primary NIC public IP **20.198.93.59**. Location Central India.

> **KEY CONCEPT**
>
> Two addresses now exist for this VM: a private address from 10.0.1.0/24 used inside the VNet, and a public IP that Azure maps to it via 1:1 NAT. The VM's own OS only ever sees the private address — `ip addr` inside the guest will never show 20.198.93.59.

22 TEST 1 — SSH from the internet into VM-Public PASS

WHAT WAS DONE

From a local terminal, ran `ssh -i VM-Public_key.pem azureuser@20.198.93.59`, then `whoami`, `hostname` and `ip addr show` to confirm identity.



WHAT THE RESULT SHOWS

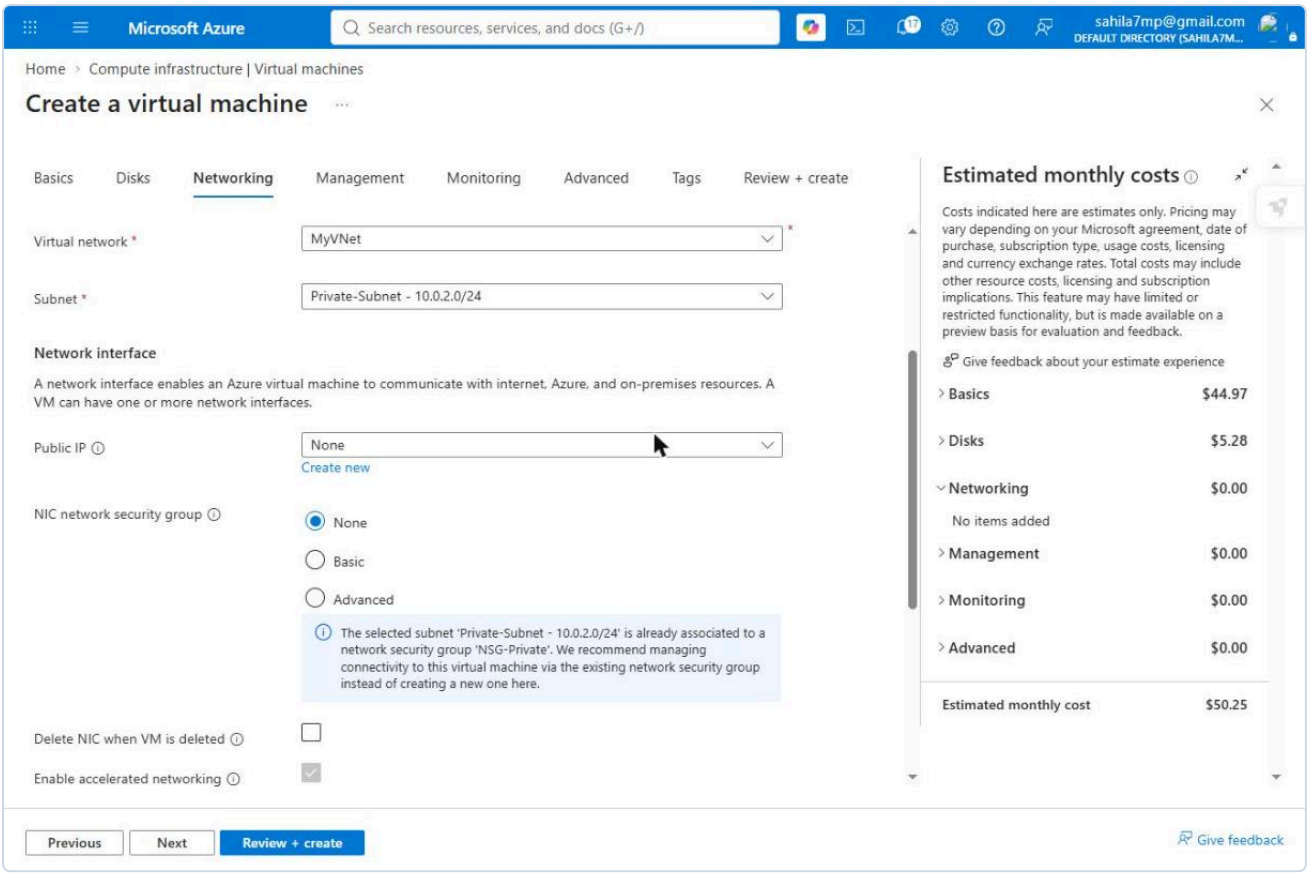
Login succeeded. `whoami` → **azureuser**, `hostname` → **VM-Public**, and `eth0` carries `inet 10.0.1.4/24`.

> **KEY CONCEPT**
>
> First proof point: `Allow-SSH` (priority 100) in NSG-Public beat `DenyAllInBound` (65500), so inbound TCP/22 from the internet was permitted. Note the interface address is **10.0.1.4** — confirming the theory above that the public IP lives in Azure's fabric, not in the guest OS. 10.0.1.0–.3 are Azure-reserved, so .4 is the first address a VM can receive.

23 VM-Private networking — no public IP

WHAT WAS DONE

Created a second VM, `VM-Private`. On the Networking tab selected subnet `Private-Subnet - 10.0.2.0/24` and set **Public IP = None**, NIC NSG = None.



WHAT THE RESULT SHOWS

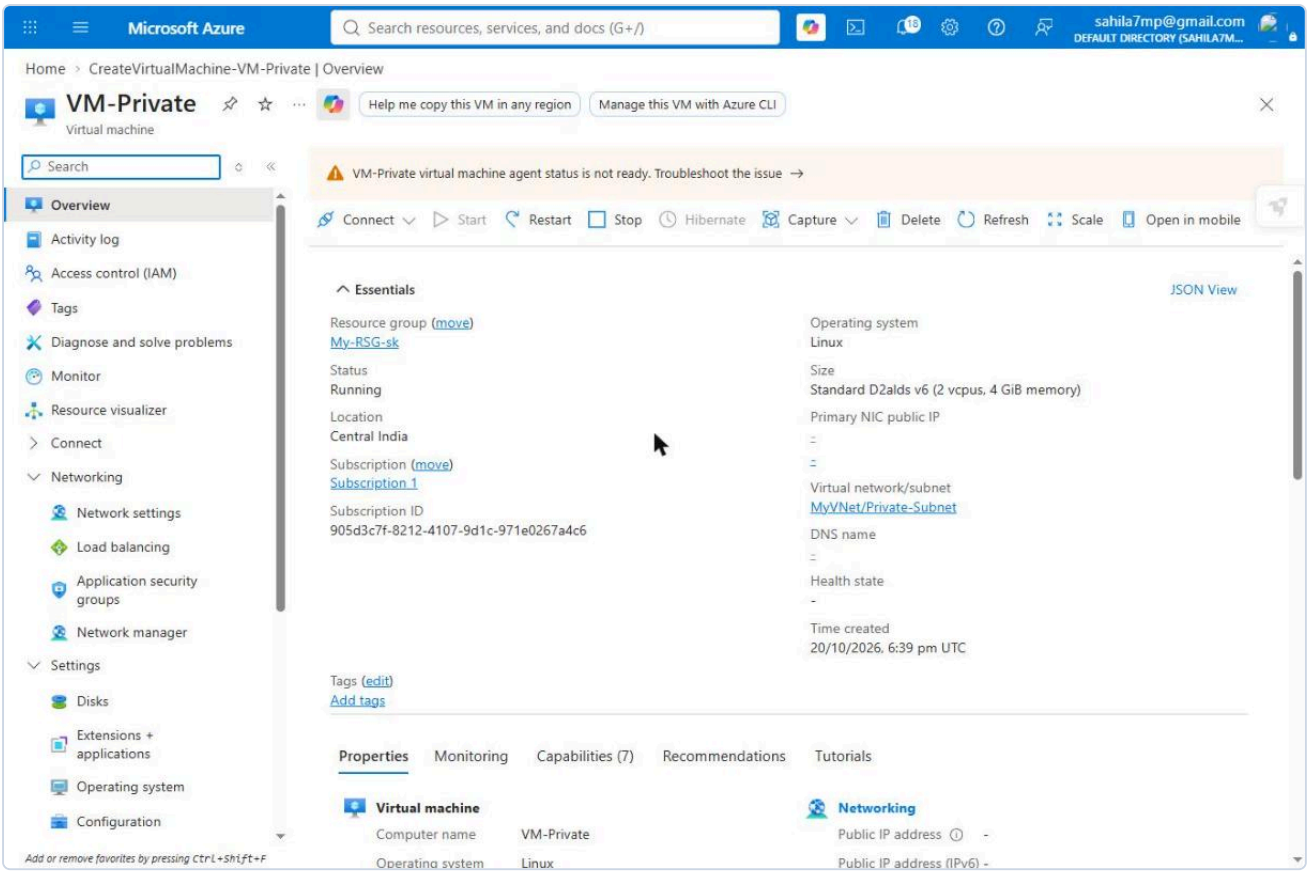
The form shows MyVNet / Private-Subnet / Public IP None, and Azure repeats its advisory that the subnet is already protected by `NSG-Private`.

> **KEY CONCEPT**
>
> Omitting the public IP removes the inbound path entirely — there is no address for the internet to target. Combined with the `VirtualNetwork`-scoped NSG rule, that is **defense in depth**: two independent controls, either of which alone would block external SSH.

24 VM-Private is running

WHAT WAS DONE

Completed the create and opened the Overview.



WHAT THE RESULT SHOWS

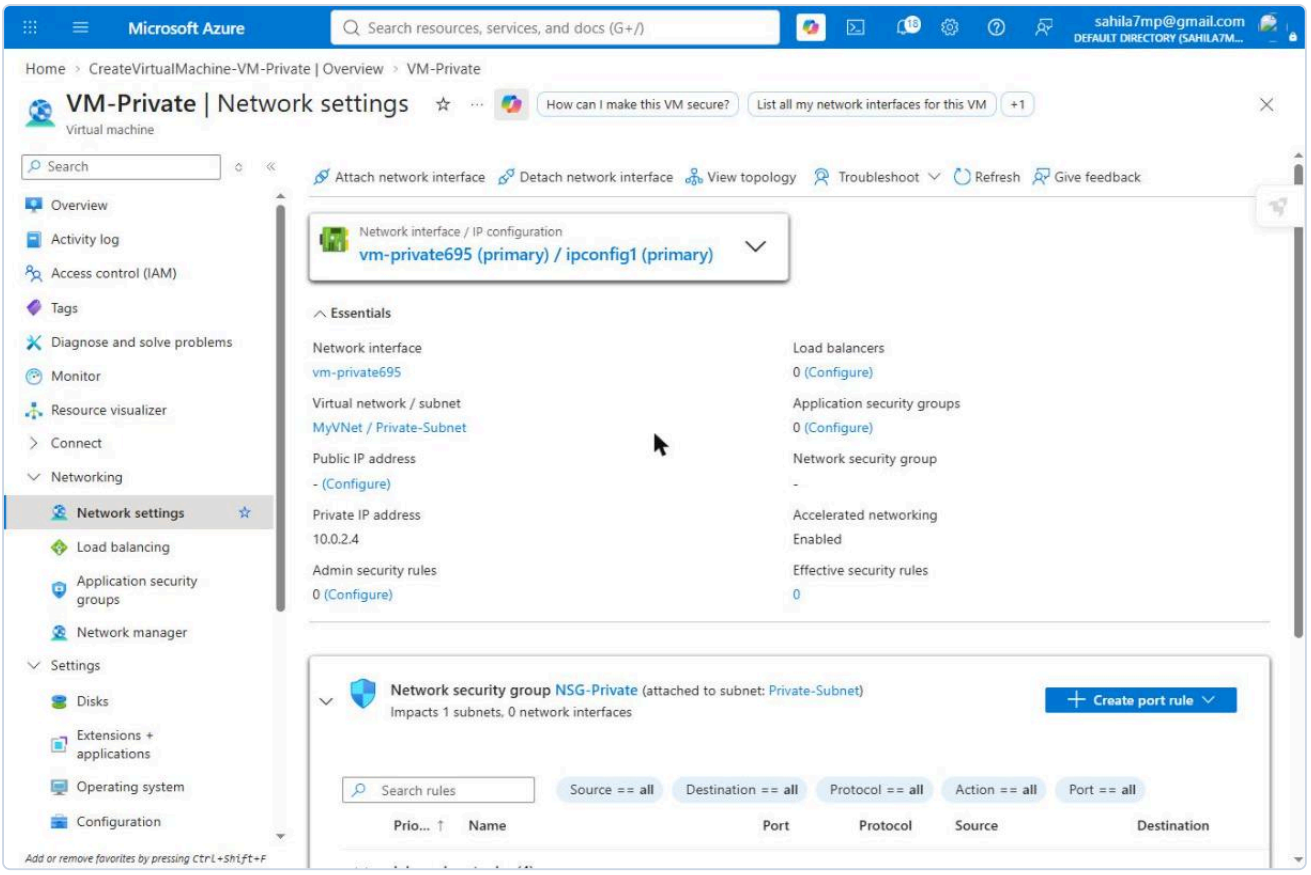
Status **Running**, Virtual network/subnet `MyVNet/Private-Subnet`, and **Primary NIC public IP: –**.

> **KEY CONCEPT**
>
> That dash is the security property. A VM with no public IP cannot be reached from the internet regardless of its NSG rules — and, with “Enable private subnet” checked on its subnet, it also has no default route *out*. Both directions are closed.

25 Find the private IP

WHAT WAS DONE

Opened **VM-Private → Network settings** to read the NIC details needed for the next test.



WHAT THE RESULT SHOWS

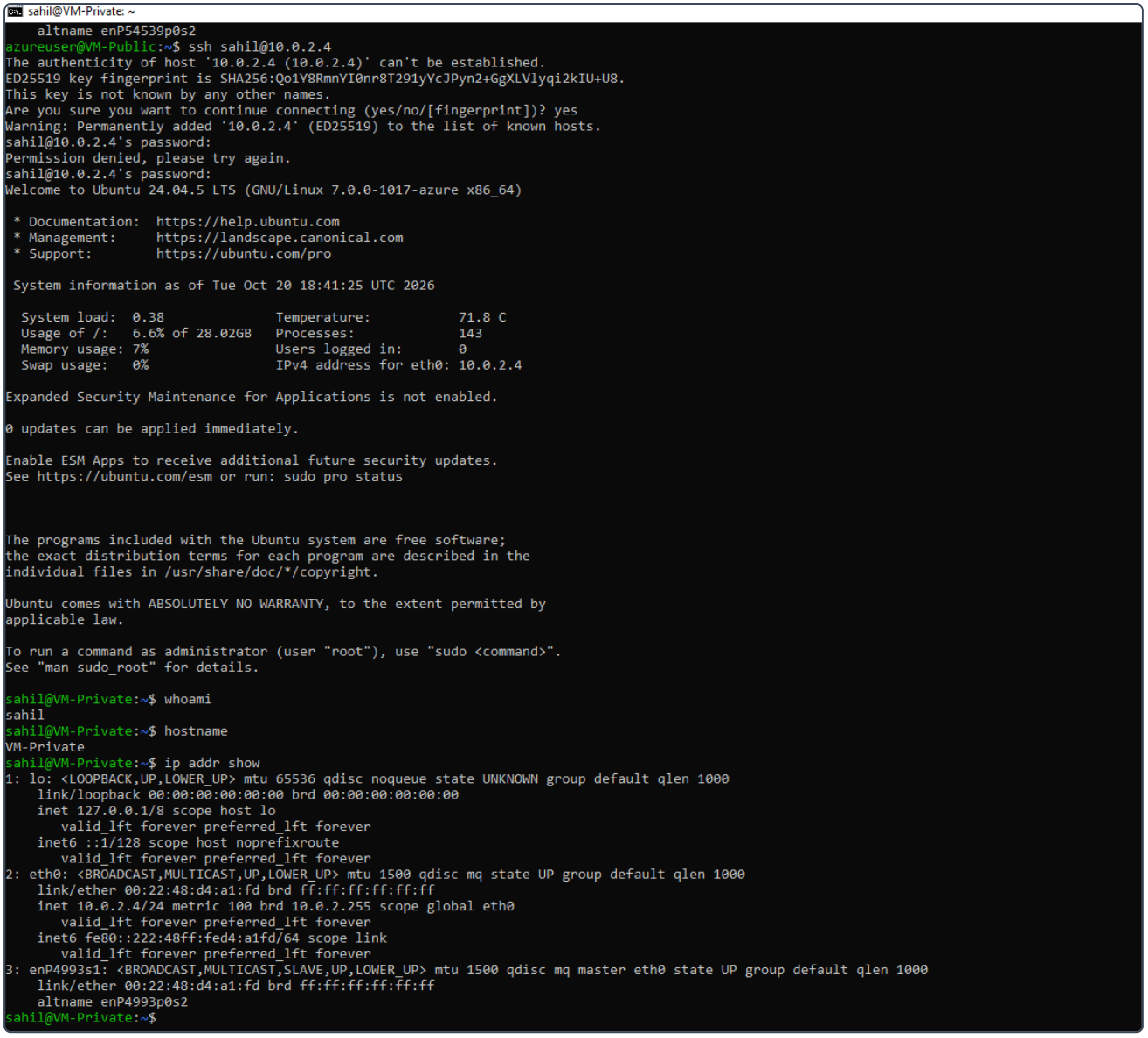
One screen confirms the full picture: Virtual network/subnet `MyVNet / Private-Subnet`, **Public IP address: –**, **Private IP address: 10.0.2.4**, and below it *Network security group **NSG-Private** (attached to subnet: Private-Subnet)*.

> **KEY CONCEPT**
>
> The Network settings blade is the single best place to verify a VM's effective network posture — subnet, both addresses, and which NSG applies and at what level, all in one view. Use it to confirm an association rather than trusting that a save worked.

26 TEST 2 — Jump host: VM-Public → VM-Private PASS

WHAT WAS DONE

From inside the existing VM-Public session, ran `ssh sahil@10.0.2.4` — SSH-ing laterally across the VNet rather than from the internet. Confirmed with `whoami`, `hostname` and `ip addr show`.



WHAT THE RESULT SHOWS

Connected. Prompt changes to `sahil@VM-Private`, `hostname` → **VM-Private**, and `eth0` shows `inet 10.0.2.4/24` — an address in Private-Subnet.

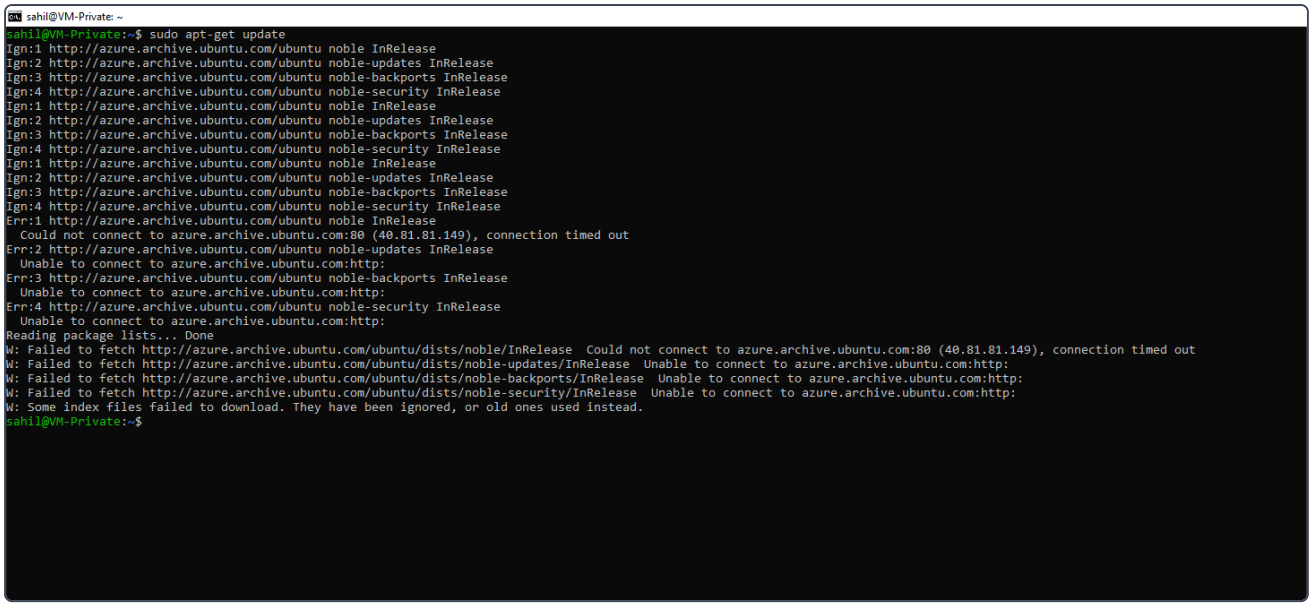
> **KEY CONCEPT**
>
> Second proof point, and the payoff for Step 15. The packet's source was **10.0.1.4**, which falls inside the `VirtualNetwork` service tag, so `Allow-SSH-VNet` matched and permitted it. The same connection attempt from the internet could never match that rule. This is the **bastion / jump host pattern**: one hardened, monitored entry point, with everything else reachable only through it.

Part 5 — Outbound Access and the NAT Gateway

27 TEST 3 — VM-Private has no internet (expected failure) BLOCKED

WHAT WAS DONE

On VM-Private, ran `sudo apt-get update`.



WHAT THE RESULT SHOWS

DNS resolved — the logs show `azure.archive.ubuntu.com` mapping to `40.81.81.149` — but every fetch failed:

`Err:1 ... Could not connect to azure.archive.ubuntu.com:80 (40.81.81.149), connection timed out`

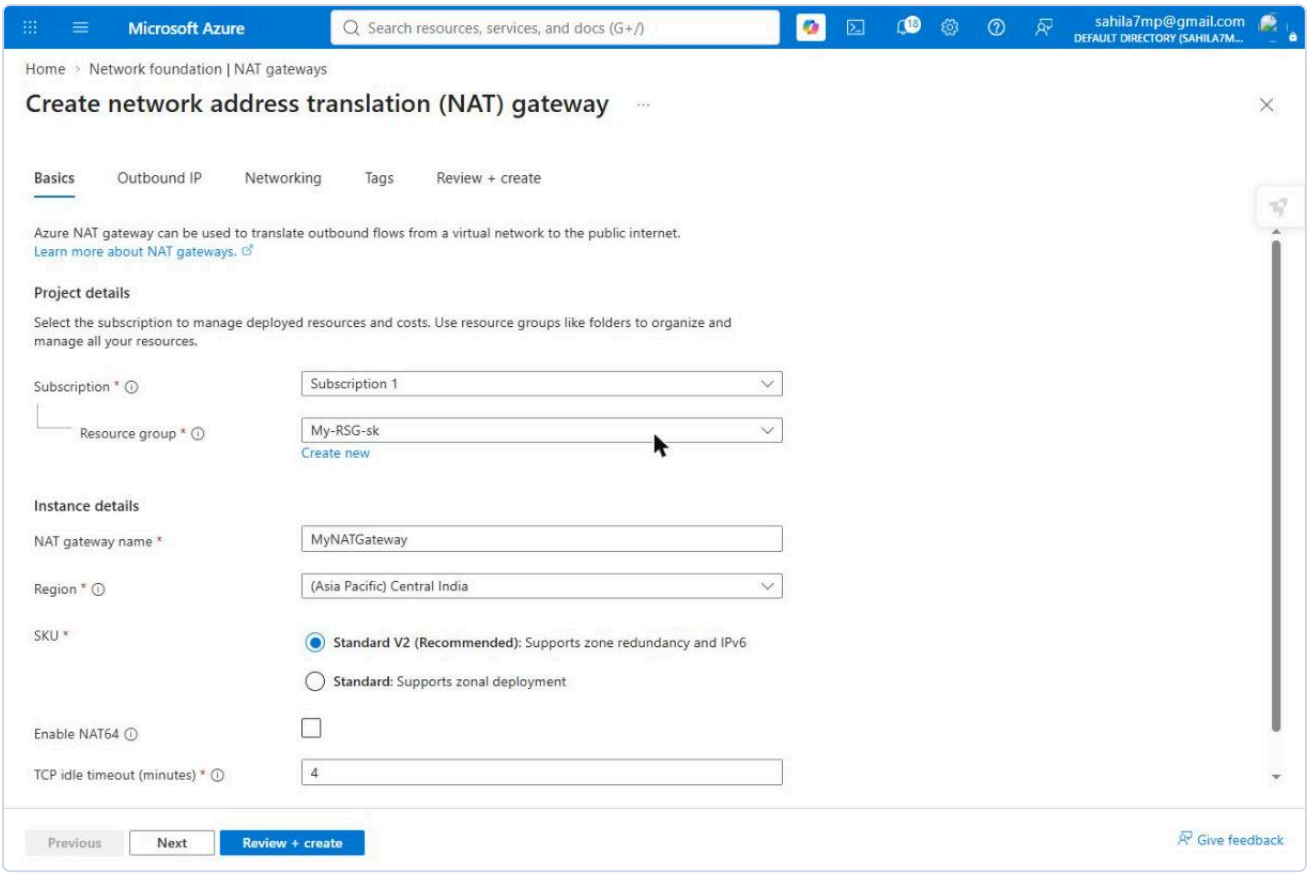
Four `Err` lines, four `W: Failed to fetch` warnings, no packages retrieved.

> **KEY CONCEPT**
>
> Read the failure carefully: `Ign` then `Err` means **name resolution worked but the TCP handshake did not**. Azure-provided DNS is reachable inside the VNet, so lookups succeed; what is missing is an outbound path to the public internet, removed by the “Enable private subnet” setting in Step 6. A *timeout* rather than an instant refusal is the signature of a packet with nowhere to go — distinguishing it from an NSG deny.

28 Create the NAT Gateway

WHAT WAS DONE

Opened **NAT gateways → + Create**. Name `MyNATGateway`, resource group `My-RSG-sk`, region Central India, SKU **Standard V2**, TCP idle timeout **4** minutes.



WHAT THE RESULT SHOWS

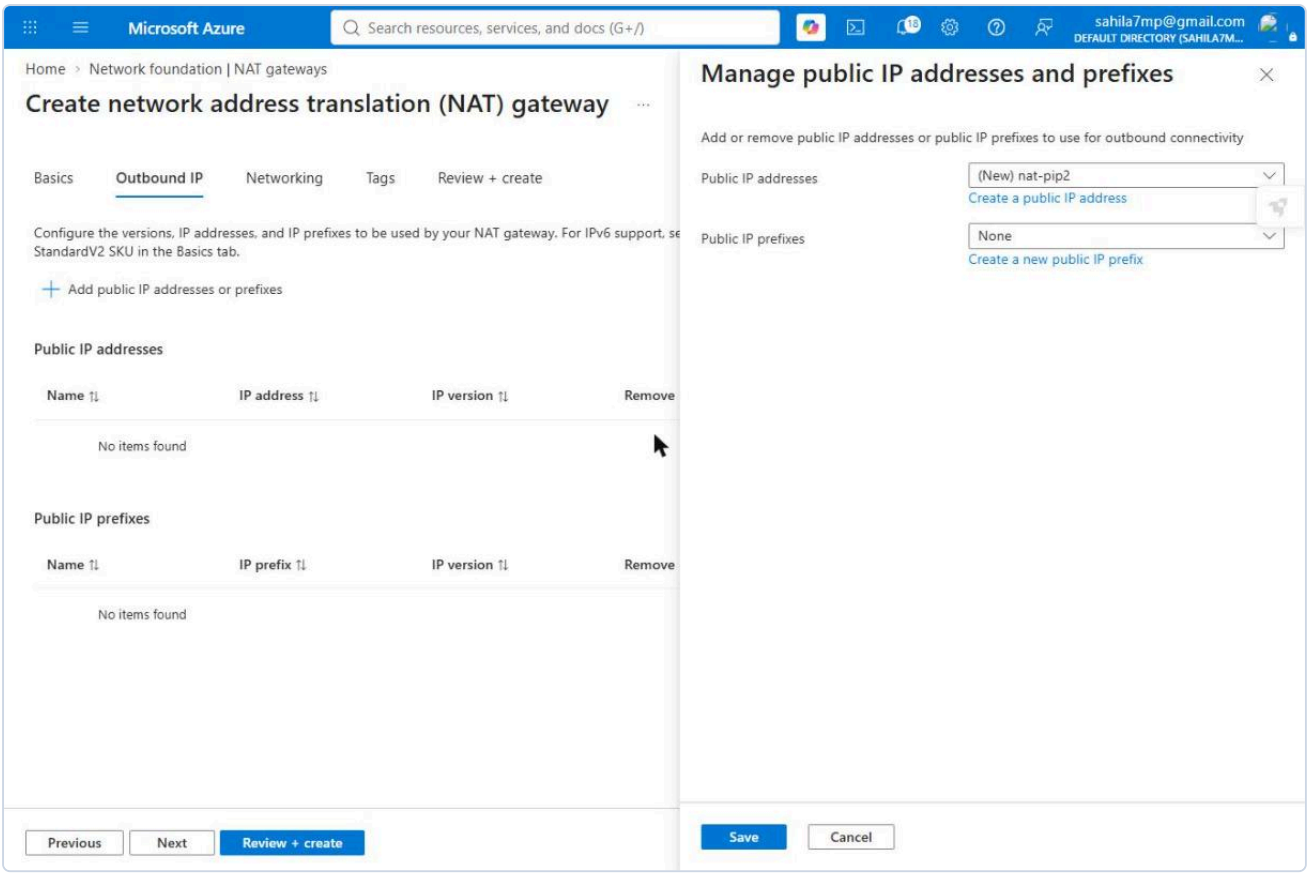
The Basics tab is complete and validated.

> **KEY CONCEPT**
>
> A NAT Gateway provides **outbound-only** connectivity. It is fully managed, gives a stable and predictable egress IP (useful for third-party allow-lists), and scales SNAT ports far better than default outbound access. The idle timeout governs how long an unused TCP flow holds its SNAT port.

29 Attach a public IP to the NAT Gateway

WHAT WAS DONE

On the **Outbound IP** tab, clicked **Add public IP addresses or prefixes** and used **Create a public IP address**, which generated `nat-pip2`.



WHAT THE RESULT SHOWS

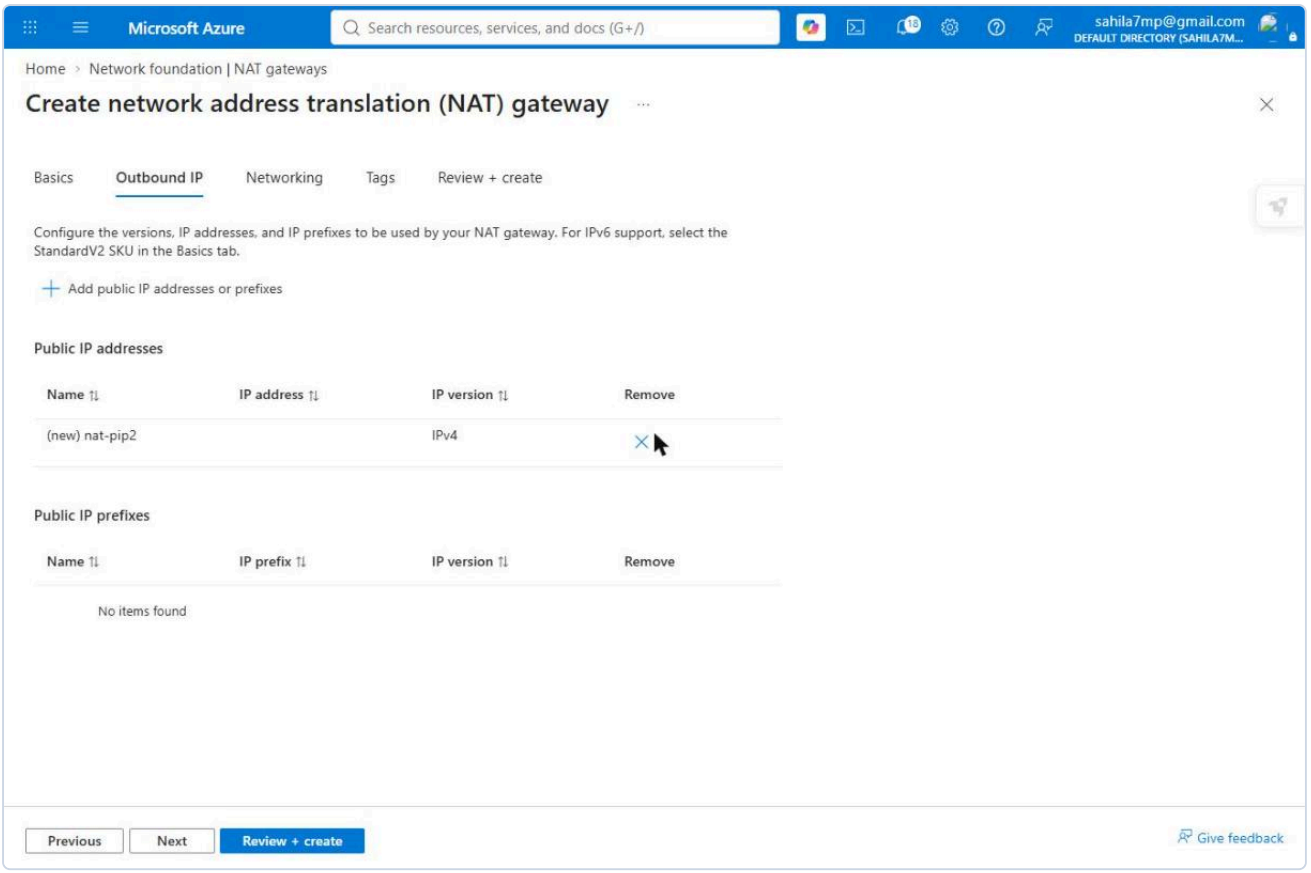
The panel shows Public IP addresses = `(New) nat-pip2`, ready to save.

> **KEY CONCEPT**
>
> This is the address the whole Private-Subnet will share when it talks to the internet. Outbound traffic from any VM in the subnet is source-NAT'd to this one IP — which is precisely what makes it stable enough to hand to a partner for firewall allow-listing.

30 Outbound IP confirmed

WHAT WAS DONE

Saved the selection and reviewed the Outbound IP tab.



WHAT THE RESULT SHOWS

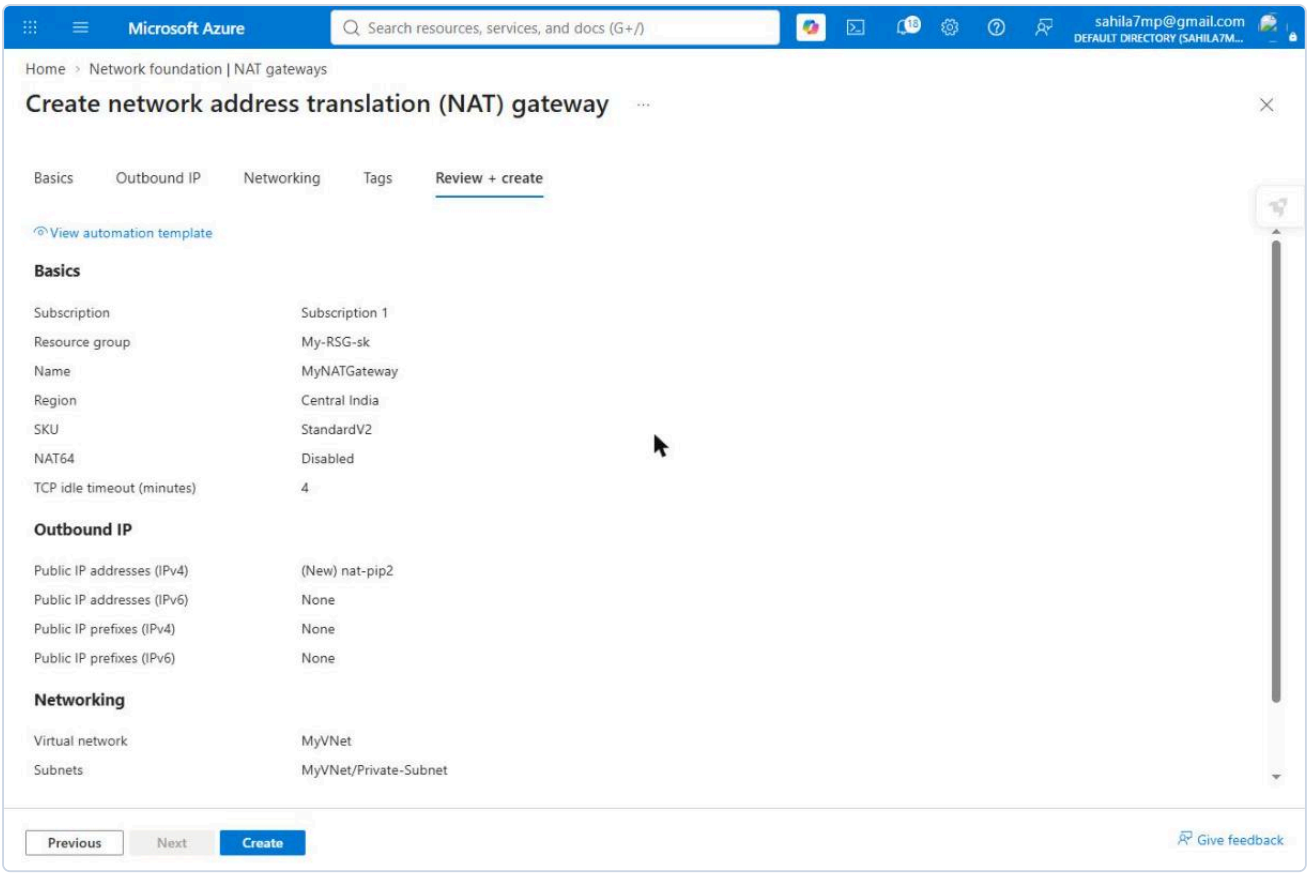
The Public IP addresses table now lists `(new) nat-pip2`, IP version **IPv4**.

> **KEY CONCEPT**
>
> A NAT Gateway can hold up to 16 public IPs or a public IP prefix. Each IP contributes roughly 64,000 SNAT ports, so adding IPs is the lever for scaling a subnet with many concurrent outbound connections.

31 Bind the NAT Gateway to Private-Subnet

WHAT WAS DONE

On the **Networking** tab selected virtual network `MyVNet` and subnet `Private-Subnet`, then opened **Review + create** to verify everything.



WHAT THE RESULT SHOWS

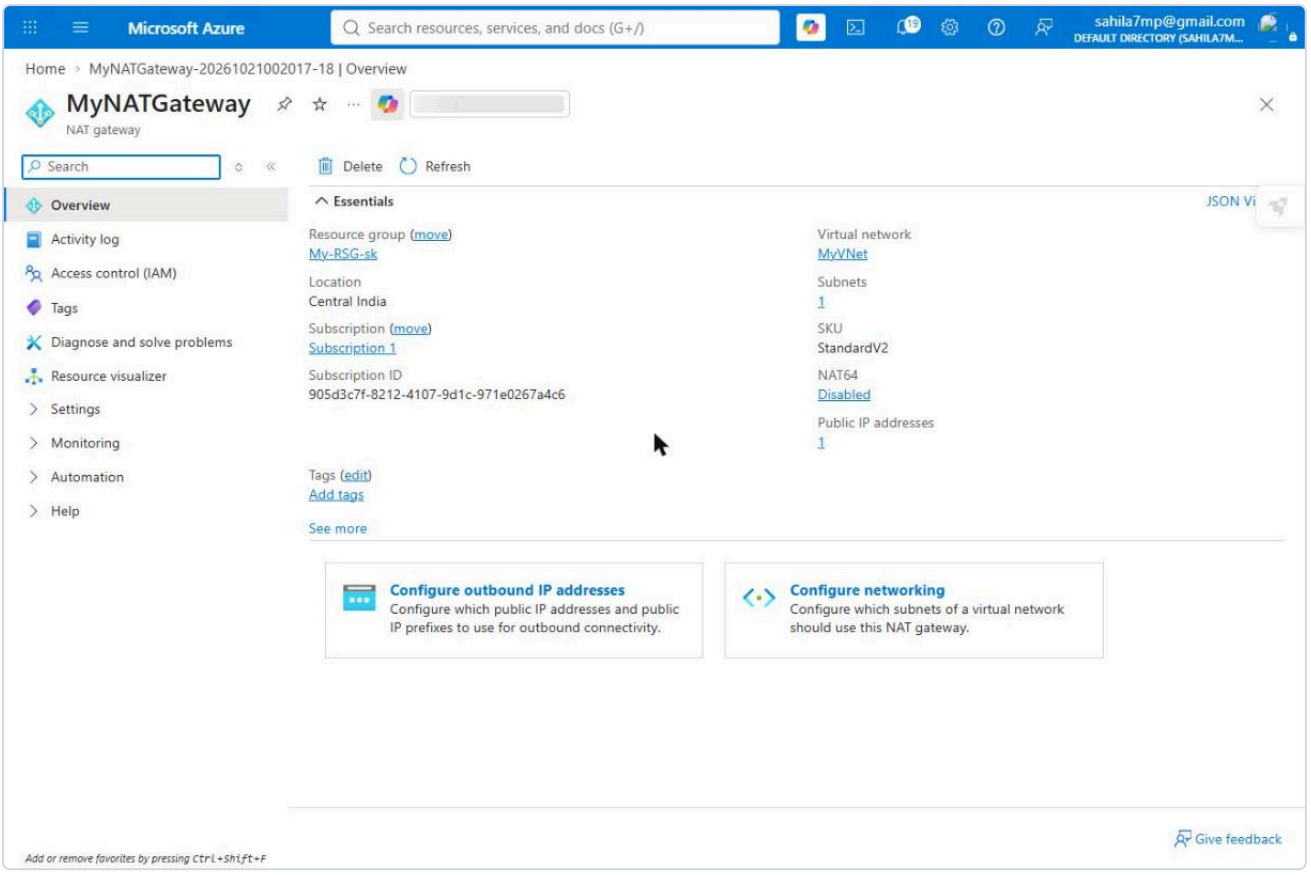
The review page confirms Name `MyNATGateway`, SKU StandardV2, Public IP addresses (IPv4) `(New) nat-pip2`, Virtual network `MyVNet`, Subnets **`MyVNet/Private-Subnet`**.

> **KEY CONCEPT**
>
> NAT Gateway attaches at **subnet** level, exactly like an NSG — and deliberately only to Private-Subnet. Public-Subnet keeps its own default outbound path and its VM's public IP, so it needs no NAT. A subnet can use only one NAT Gateway.

32 MyNATGateway deployed

WHAT WAS DONE

Clicked **Create** and opened the resource.



WHAT THE RESULT SHOWS

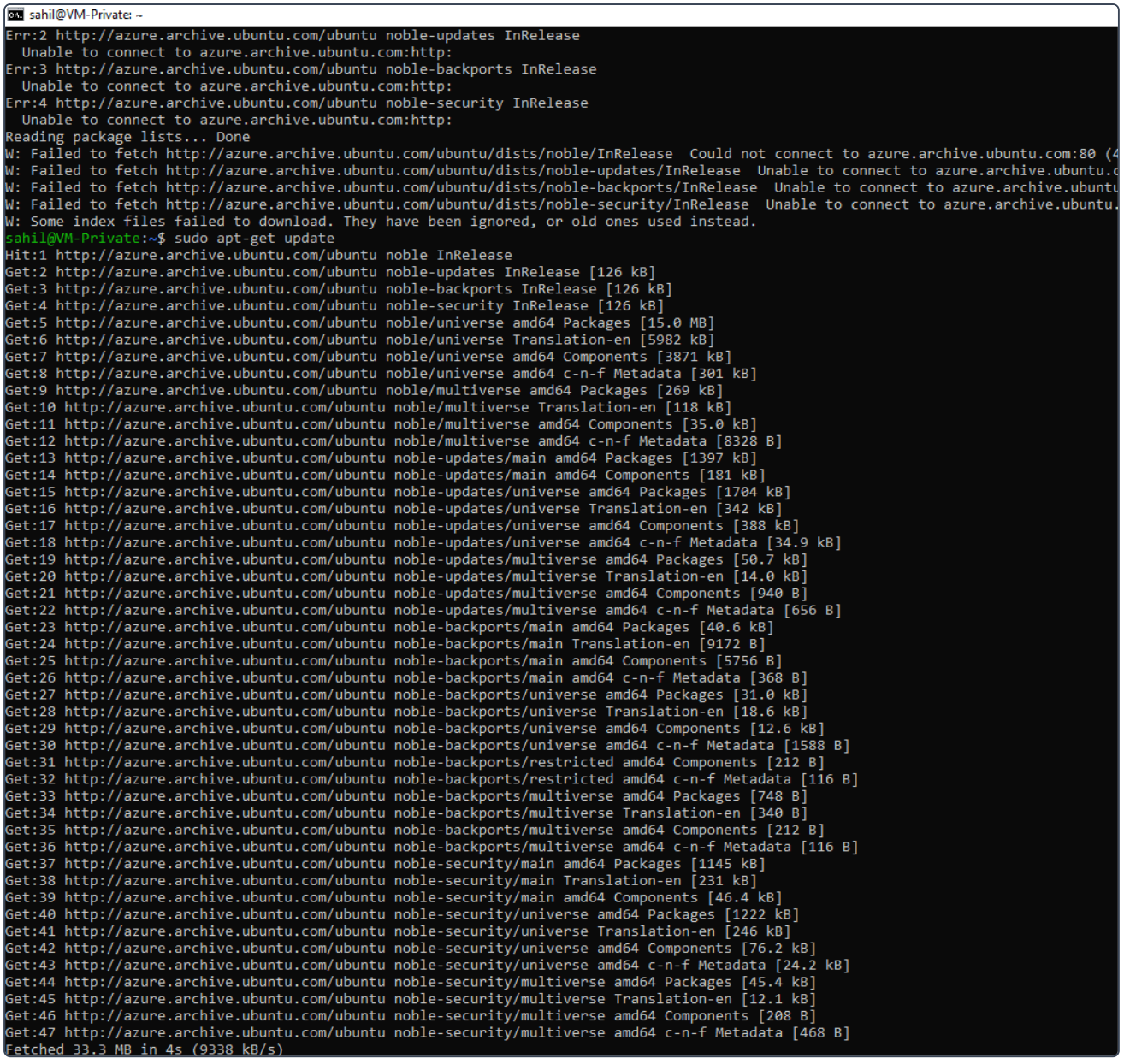
Overview confirms Virtual network `MyVNet`, **Subnets: 1**, **Public IP addresses: 1**, SKU StandardV2.

> **KEY CONCEPT**
>
> Subnets 1 and Public IP addresses 1 are the two values to check — they confirm both ends of the attachment. Once bound, Azure installs the outbound route for the subnet automatically; no User-Defined Route is required.

33 TEST 4 — VM-Private now reaches the internet PASS

WHAT WAS DONE

Back on VM-Private, re-ran the identical command: `sudo apt-get update`.



WHAT THE RESULT SHOWS

Complete reversal. The failed run is still visible at the top of the terminal for comparison; the new run shows `Hit:1` and 47 successful `Get:` lines, finishing with **`Fetched 33.3 MB in 4s (9338 kB/s)`**.

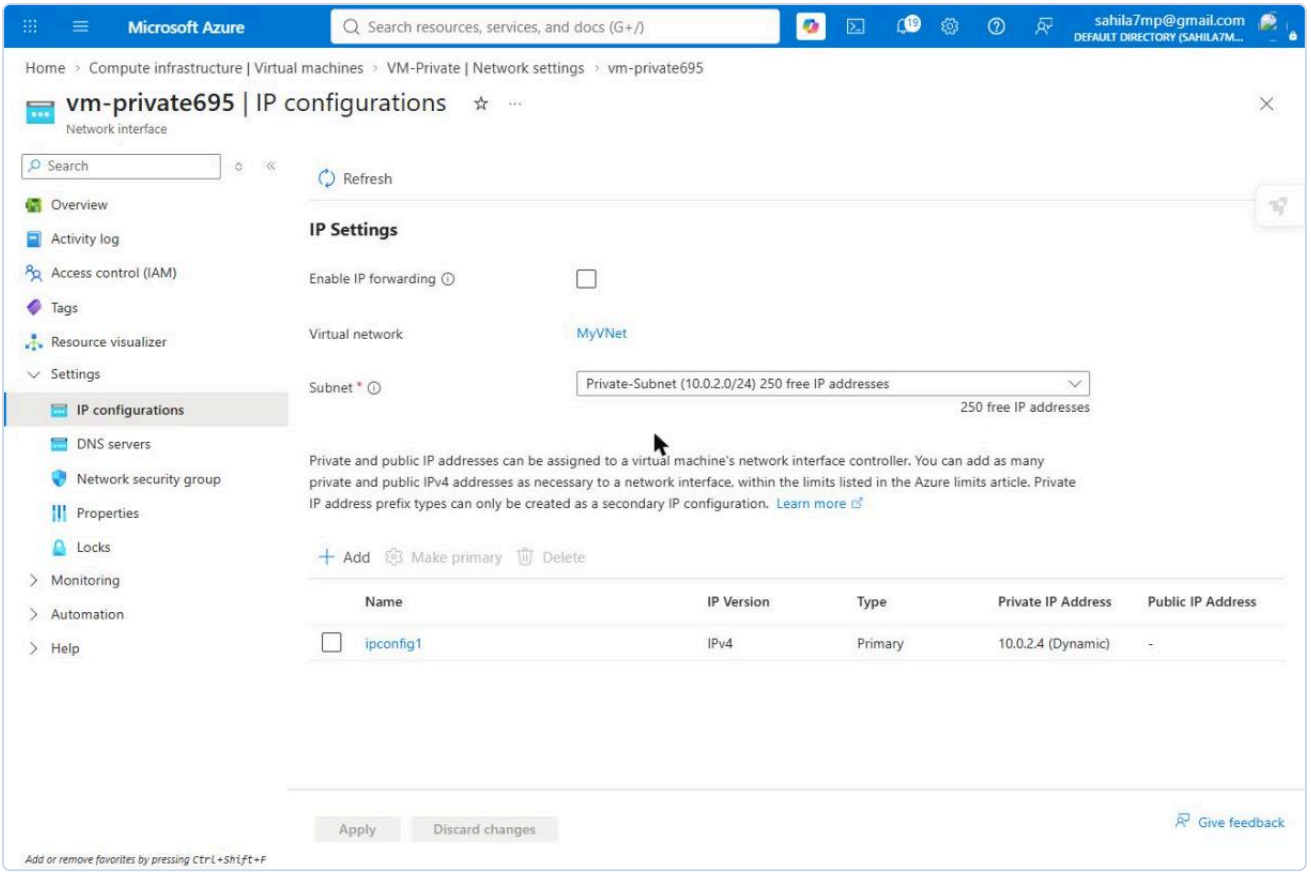
> **KEY CONCEPT**
>
> Final proof point, and the cleanest possible before/after: nothing changed on the VM, in its NSG, or in its addressing — **only the subnet's outbound path**. VM-Private still has no public IP and is still unreachable from the internet; it can now *initiate* outbound connections through the NAT Gateway's shared IP. Outbound access and inbound exposure are independent concerns, and this is how you get the first without the second.

Part 6 — Static Private IP Addressing

34 Inspect the current IP configuration

WHAT WAS DONE

Opened **VM-Private → Network settings**, clicked the network interface `vm-private695`, then **Settings → IP configurations**.



WHAT THE RESULT SHOWS

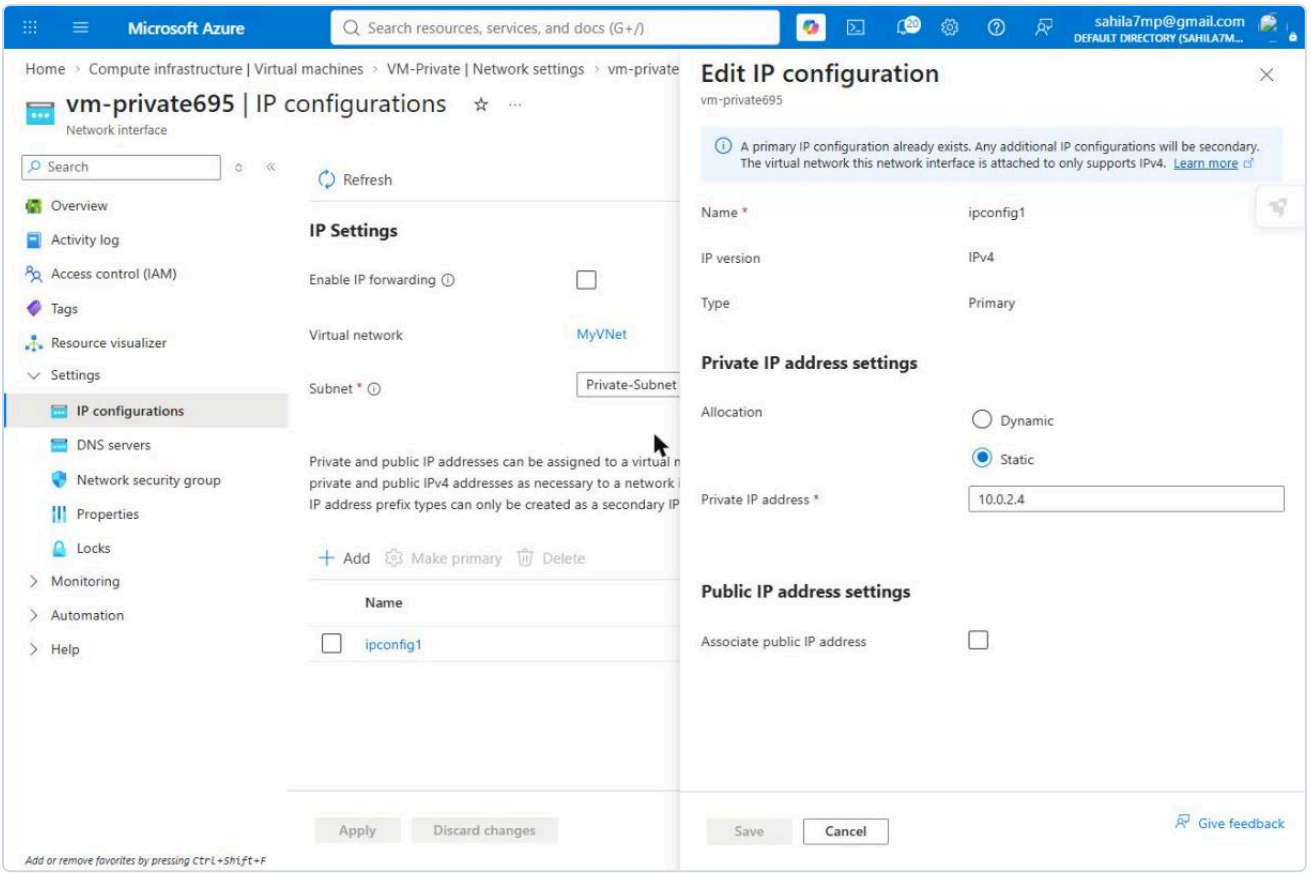
The NIC is attached to `Private-Subnet (10.0.2.0/24)` with 250 free addresses, and `ipconfig1` holds **10.0.2.4 (Dynamic)**.

> **KEY CONCEPT**
> Every Azure NIC gets a **dynamic** private IP by default. Dynamic here does not mean the address changes while the VM runs — it means Azure only guarantees the lease while the VM is allocated. Deallocate the VM and the address returns to the pool, so a restart can produce a different one.

35 Switch the allocation to Static

WHAT WAS DONE

Clicked `ipconfig1` to open **Edit IP configuration** and changed **Allocation** from Dynamic to **Static**.



WHAT THE RESULT SHOWS

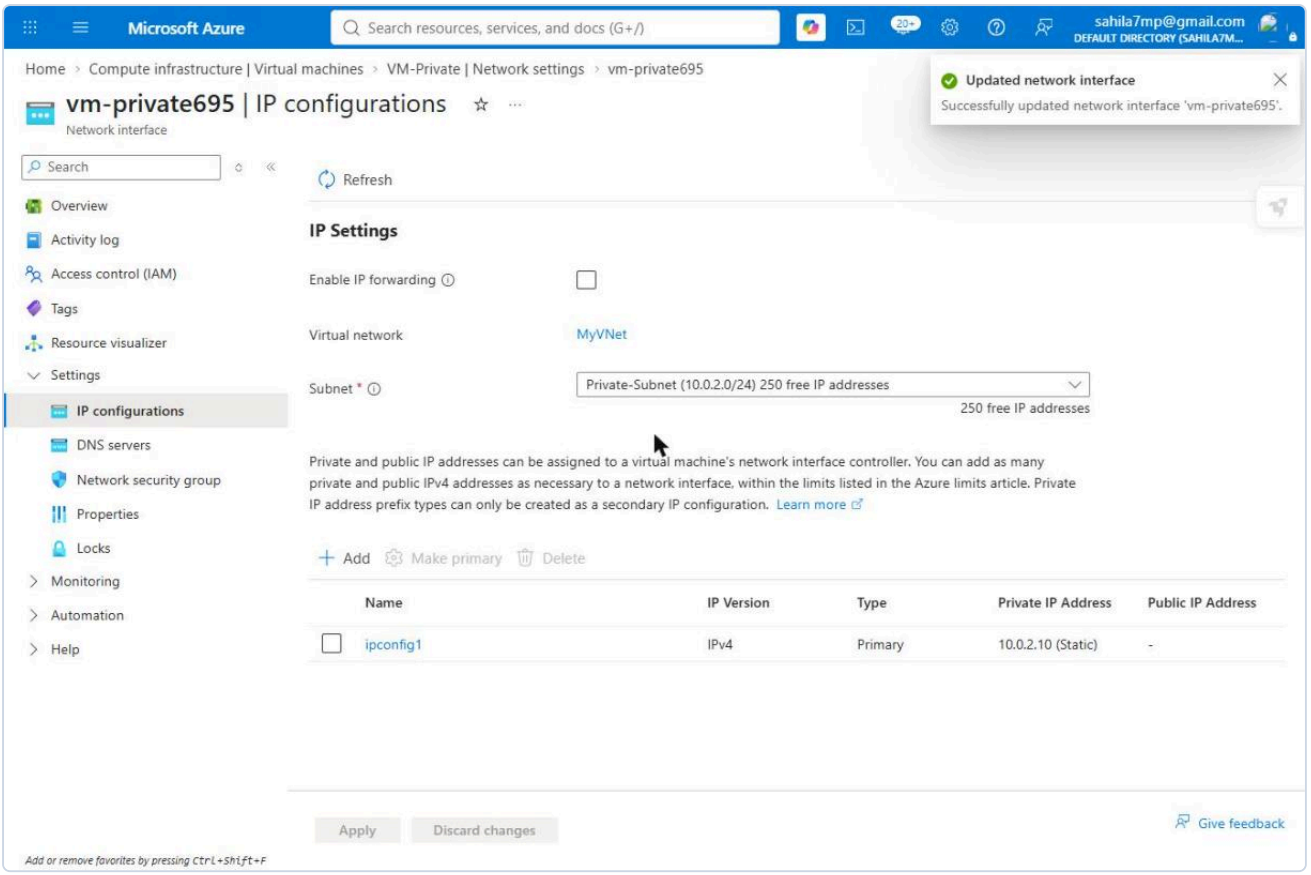
Selecting Static makes the **Private IP address** field editable. It still carries the old value `10.0.2.4`, and **Save stays greyed out** until the value actually changes.

> **KEY CONCEPT**
>
> Switching to Static is only half the change — Azure keeps the current address as the proposed static value. A greyed-out Save button is the portal telling you nothing has been modified yet; it is a useful signal to read rather than click past.

36 Assign 10.0.2.10 and save

WHAT WAS DONE

Replaced the address with `10.0.2.10` and clicked **Save**.



WHAT THE RESULT SHOWS

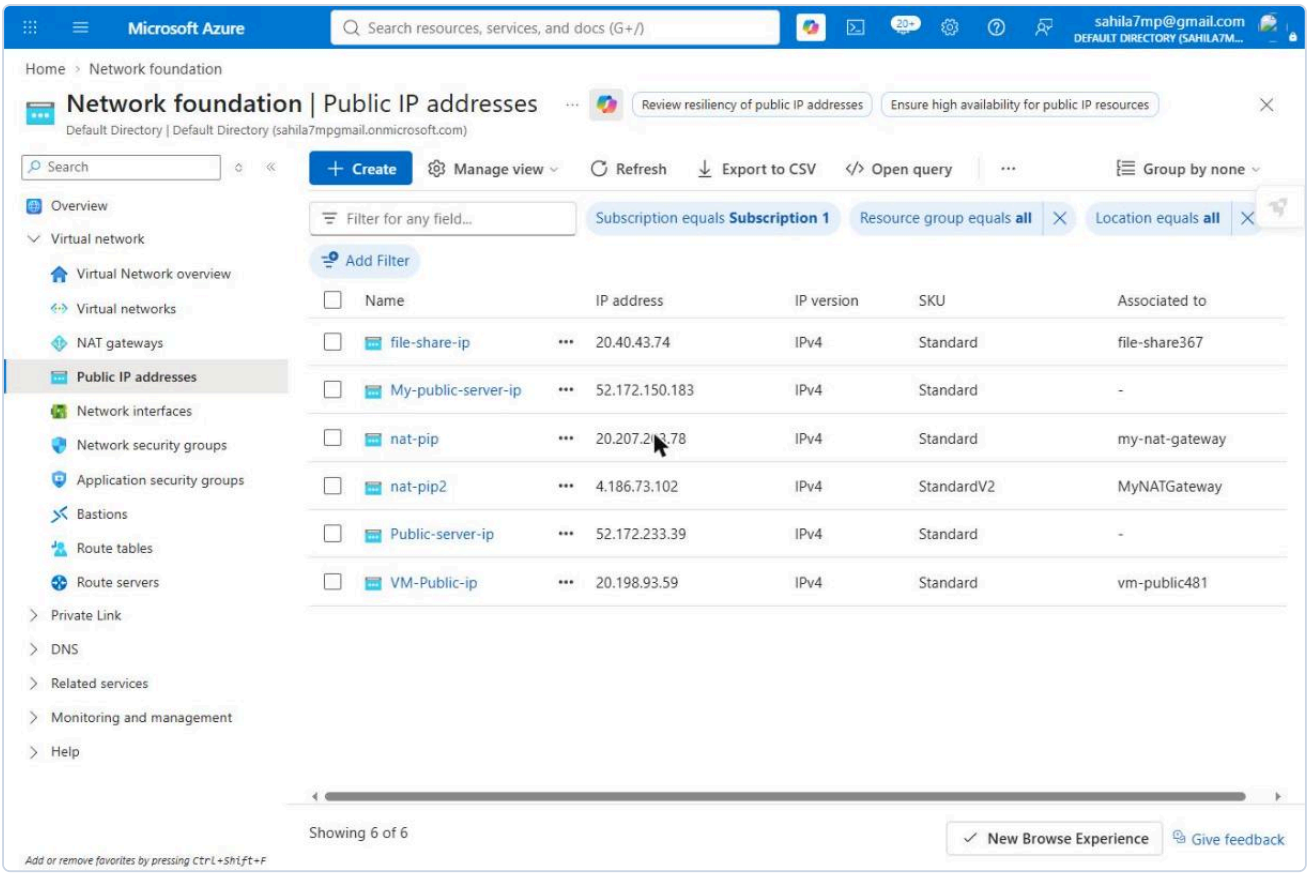
Green toast: *"Successfully updated network interface 'vm-private695'."* The table now reads `ipconfig1 → 10.0.2.10 (Static)`.

> **KEY CONCEPT**
>
> A static private IP is reserved on the NIC and survives deallocation. Pick addresses well above the bottom of the range — Azure reserves `.0` – `.3` and the broadcast address in every subnet, and dynamic leases are handed out from the low end, so `.10` avoids a future collision. Use static addressing for domain controllers, DNS servers, databases and anything referenced by IP in a config file.

37 Public IP allocation — why nothing needed changing

WHAT WAS DONE

Opened **Public IP addresses** to check whether `VM-Public-ip` also needed pinning.



WHAT THE RESULT SHOWS

Every public IP in the subscription is listed with its SKU. `VM-Public-ip` (20.198.93.59) is **Standard**, and `nat-pip2` (4.186.73.102) is **StandardV2**.

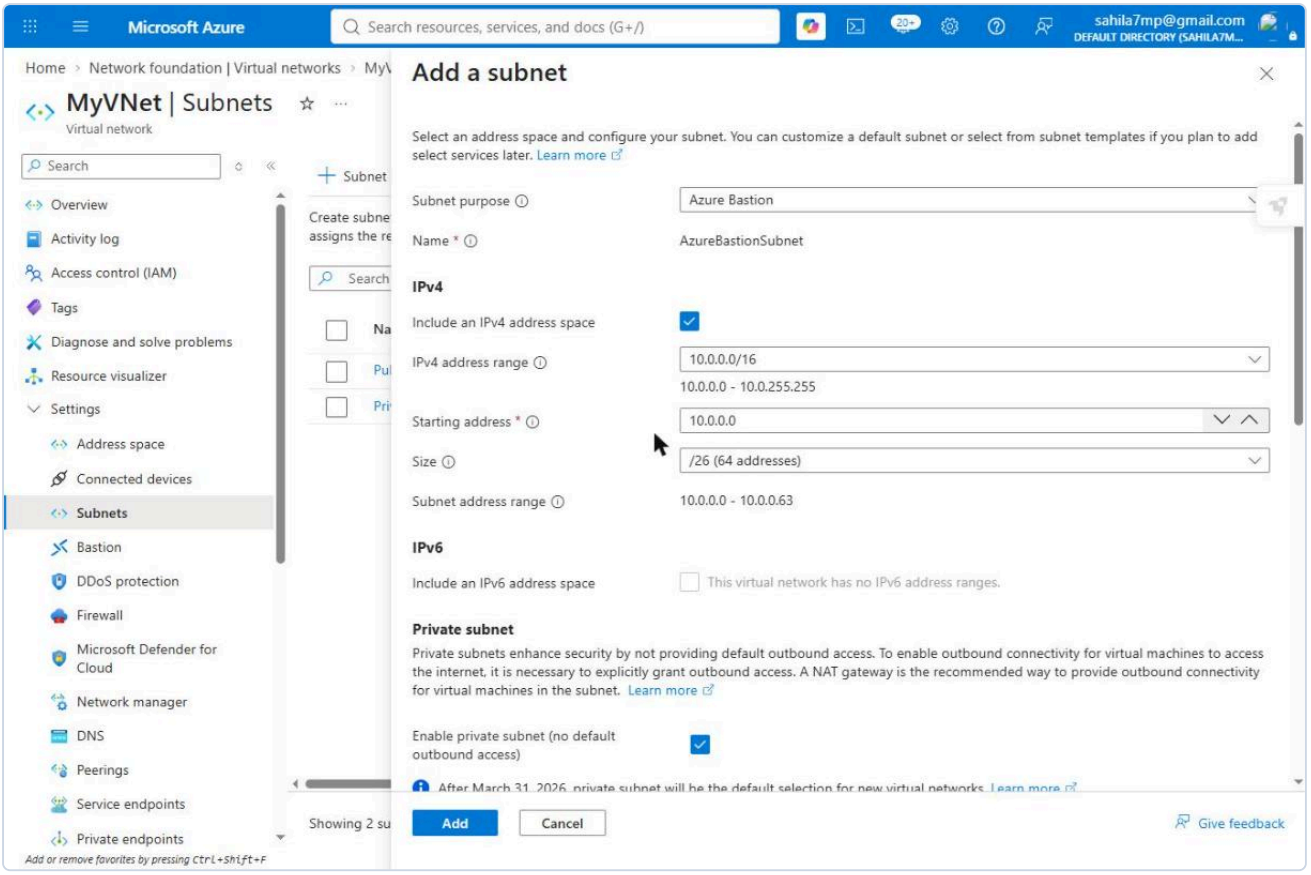
> **KEY CONCEPT**
>
> **Standard SKU public IPs are always static** — the Dynamic option does not exist for them, so there was nothing to change. Only the retired Basic SKU offered dynamic allocation, where the address was released on deallocation. This is also why the NAT Gateway's egress IP is dependable enough to hand to a partner for firewall allow-listing, as noted in Step 29.

Part 7 — Azure Bastion

38 Add the Bastion subnet — first attempt

WHAT WAS DONE

Opened **MyVNet → Subnets → + Subnet** and set **Subnet purpose** to **Azure Bastion**, which auto-filled and locked the name `AzureBastionSubnet`. Size `/26`.



WHAT THE RESULT SHOWS

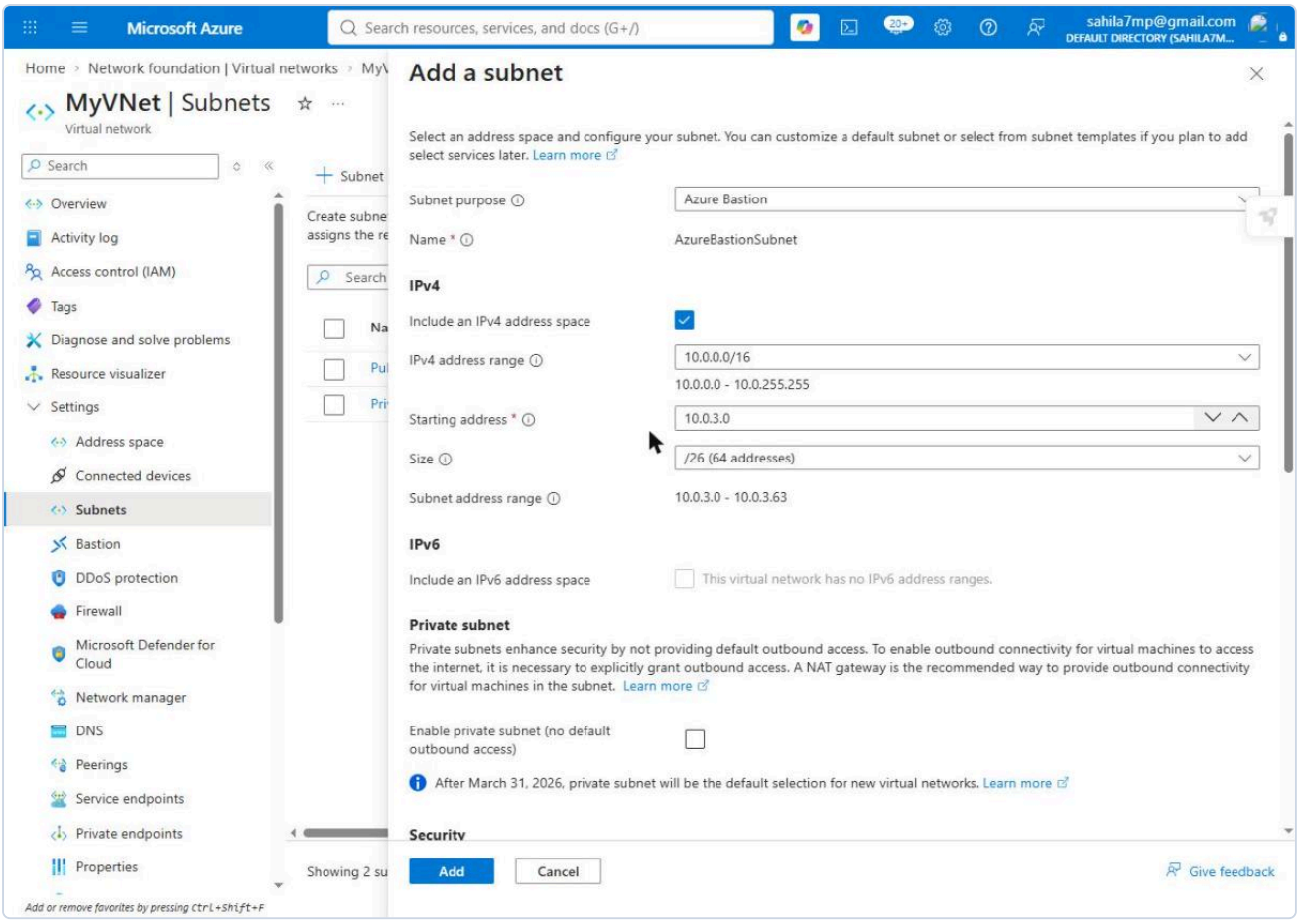
Two problems on the form: the starting address defaulted to `10.0.0.0` (range 10.0.0.0–10.0.0.63, not the planned 10.0.3.0), and **“Enable private subnet”** was ticked by default.

> **KEY CONCEPT**
>
> The name is non-negotiable — Azure locates the Bastion host by the literal string `AzureBastionSubnet`, and `/26` is the minimum size. The private-subnet tick is the trap: Bastion needs outbound reachability to the Azure control plane, so leaving it on risks a failed deployment. Always read a pre-filled form rather than trusting the defaults.

39 Corrected: 10.0.3.0/26, private subnet off

WHAT WAS DONE

Changed the starting address to `10.0.3.0` and unticked **Enable private subnet**.



WHAT THE RESULT SHOWS

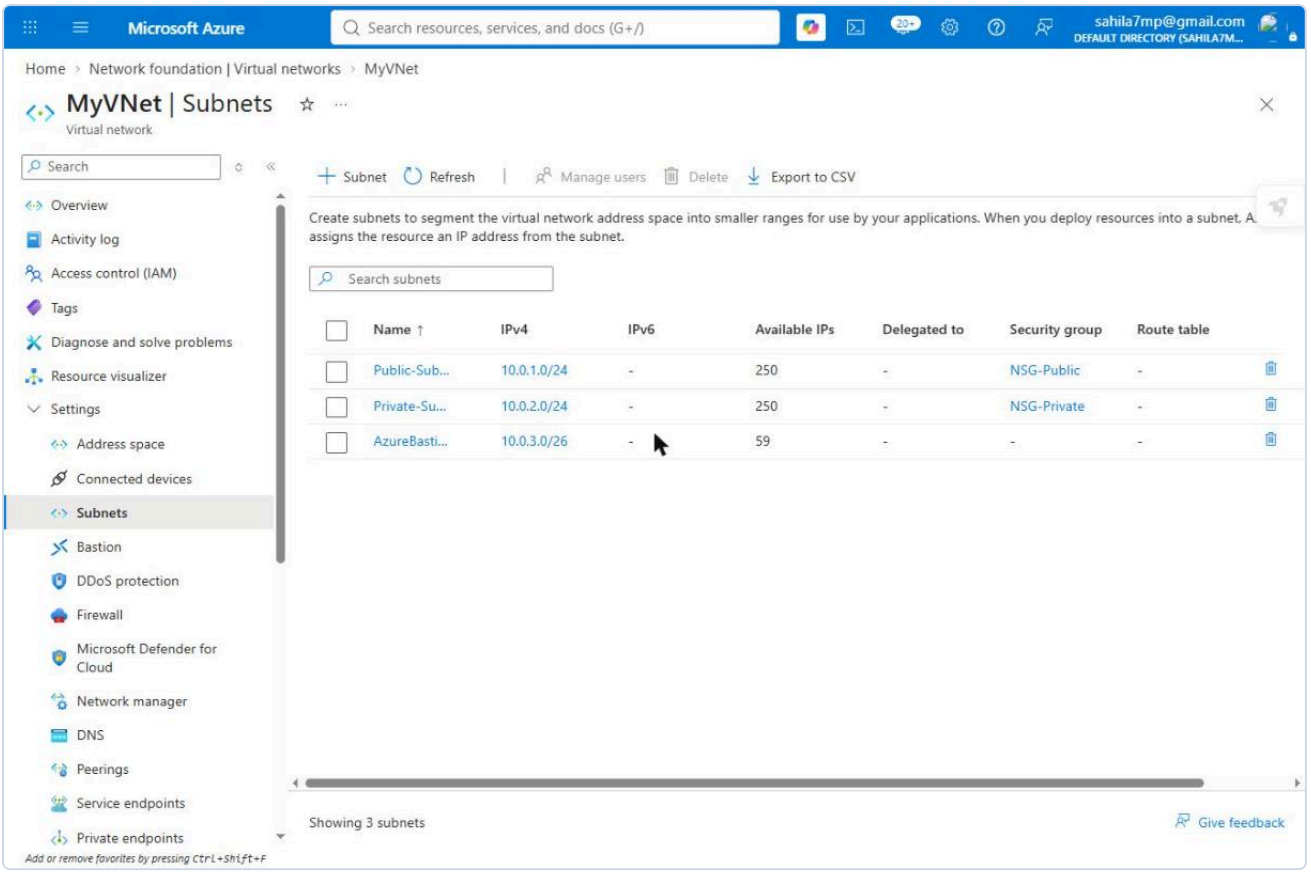
Subnet address range recalculates to **10.0.3.0 – 10.0.3.63**, and the private-subnet box is clear. The form is now correct.

> **KEY CONCEPT**
>
> A `/26` gives 64 addresses, which is the floor Bastion requires for its managed host pool and for future scaling. Going smaller is rejected outright; going larger is allowed but wasteful.

40 Three subnets in MyVNet

WHAT WAS DONE

Clicked **Add** and reviewed the subnet list.



WHAT THE RESULT SHOWS

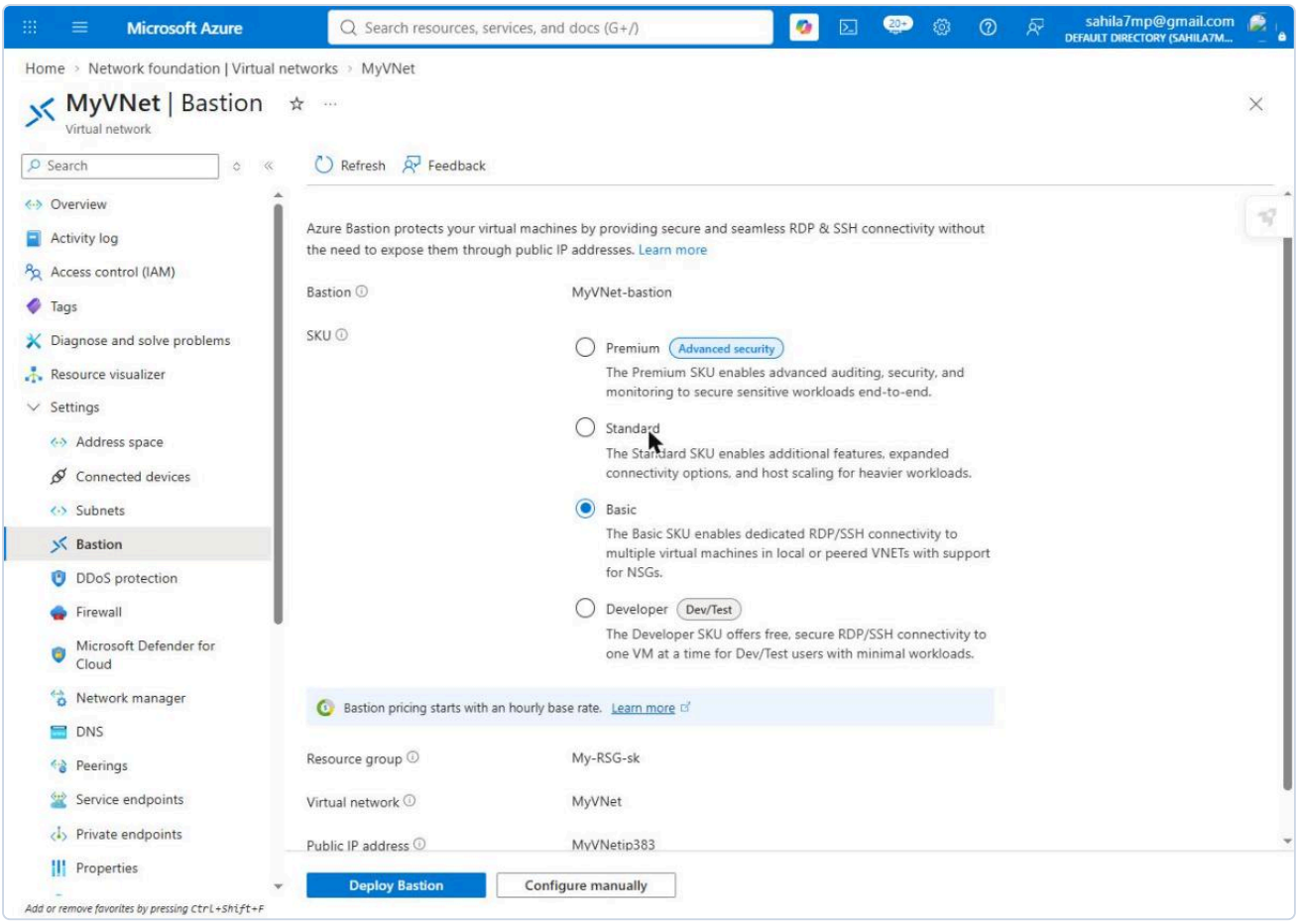
MyVNet now holds three subnets: `Public-Subnet` 10.0.1.0/24 (NSG-Public, 250 free), `Private-Subnet` 10.0.2.0/24 (NSG-Private, 250 free), and `AzureBastionSubnet` 10.0.3.0/26 (**no NSG**, **59 free**).

> **KEY CONCEPT**
>
> Two details worth reading. **59 free of 64** confirms Azure's five-address reservation applies here too. **No security group** is expected — an NSG on the Bastion subnet is optional and, if added, must follow a strict Microsoft-documented rule set, so leaving it unset is the safe default for a lab.

41 Choose the Bastion SKU

WHAT WAS DONE

Opened **MyVNet → Bastion**, which offers a quick-deploy form with everything pre-filled: name `MyVNet-bastion`, resource group `My-RSG-sk`, virtual network `MyVNet` and an auto-created public IP. Left the SKU on **Basic**.



WHAT THE RESULT SHOWS

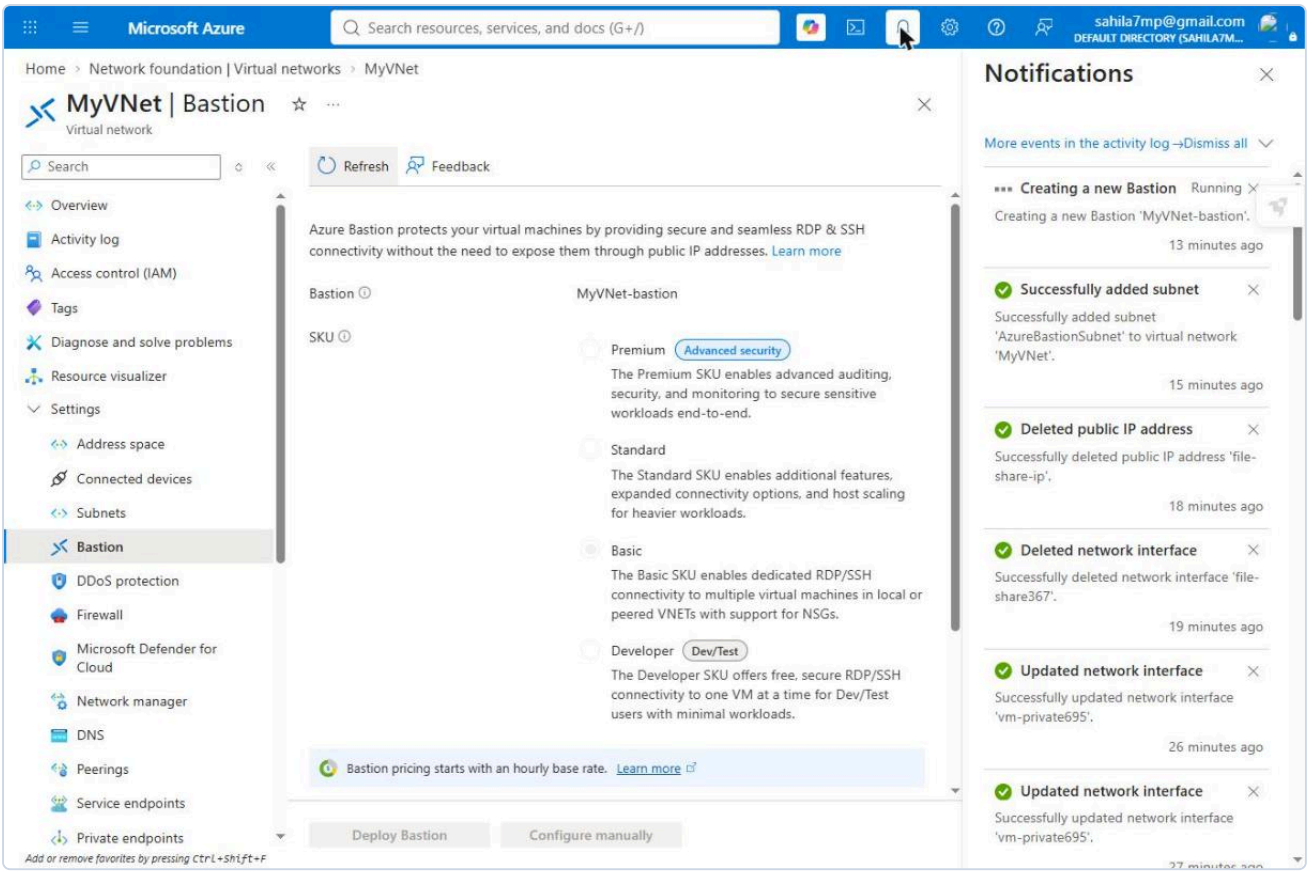
Four SKUs are offered — **Premium** (session recording, advanced auditing), **Standard** (host scaling, native client, IP-based connect), **Basic** (dedicated RDP/SSH to multiple VMs, NSG support) and **Developer** (free, one VM at a time, Dev/Test only).

> **KEY CONCEPT**
>
> The SKU ladder maps to how the host is run. **Developer** is free because it uses shared infrastructure and needs no `AzureBastionSubnet` at all — fine for a quick connect, wrong for demonstrating the architecture. **Basic** is the entry point for a dedicated, VNet-resident host, which is what this lab is proving. Note that Bastion bills an hourly base rate whether or not anyone is connected.

42 Deployment in progress

WHAT WAS DONE

Clicked **Deploy Bastion** and opened the notifications panel to follow it.



WHAT THE RESULT SHOWS

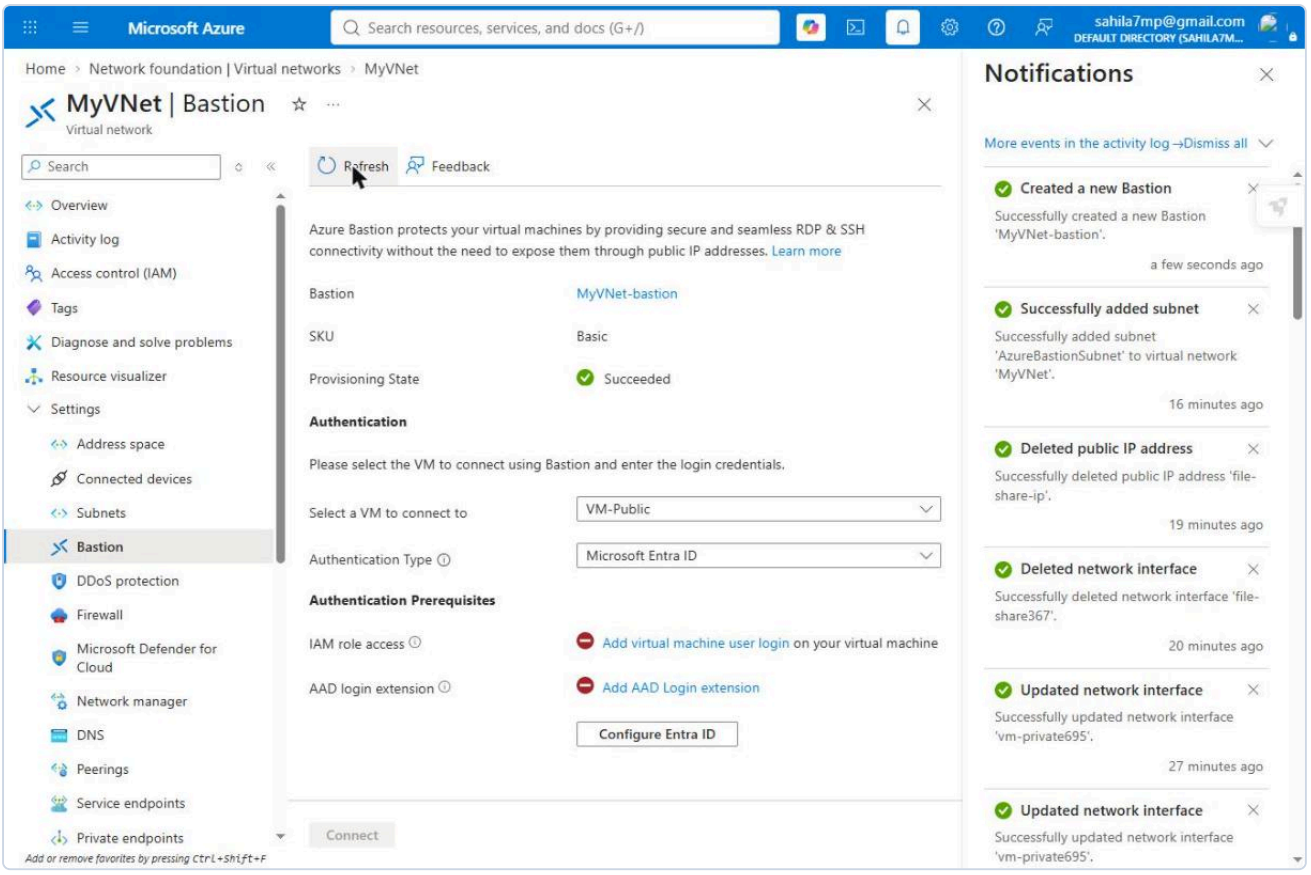
Creating a new Bastion — **Running**. The form's controls grey out for the duration. The panel also confirms the earlier step: *Successfully added subnet* '*AzureBastionSubnet*' *to virtual network* '*MyVNet*'.

> **KEY CONCEPT**
>
> Bastion takes roughly 5–10 minutes because Azure is provisioning a managed host pool inside your subnet, not just writing a configuration record. The notifications panel is the reliable place to watch long deployments — the blade itself often shows no progress indicator.

43 MyVNet-bastion is ready

WHAT WAS DONE

Refreshed once the notification turned green.



WHAT THE RESULT SHOWS

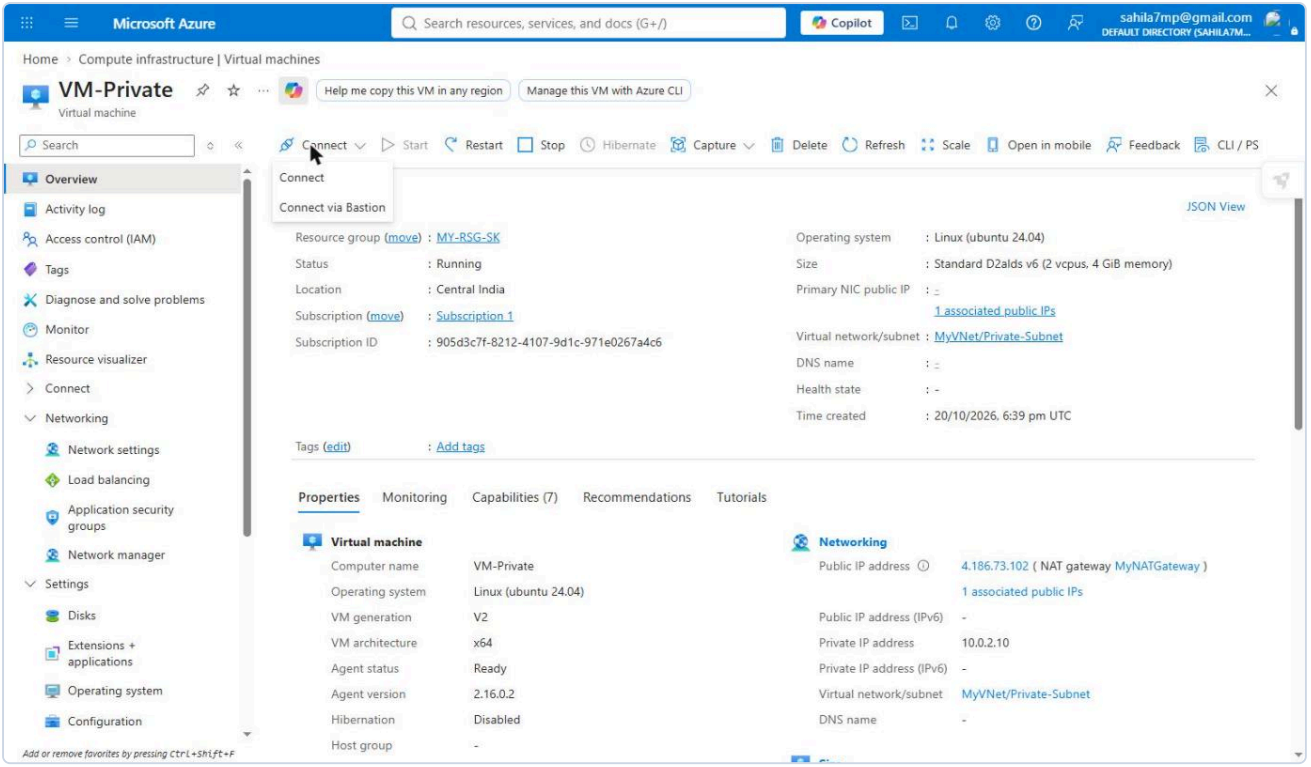
Provisioning State: Succeeded, SKU Basic, and the blade switches from a deploy form to a **connect** form asking which VM to reach and how to authenticate.

> **KEY CONCEPT**
>
> The blade changing shape is the signal that Bastion is live. Its default authentication offer is **Microsoft Entra ID**, which shows red prerequisites — it needs an IAM role assignment and the AAD login extension on the VM. For a lab, SSH key or VM password is the direct path.

44 Connect via Bastion — and a second confirmation

WHAT WAS DONE

Opened **VM-Private → Connect → Connect via Bastion**.



WHAT THE RESULT SHOWS

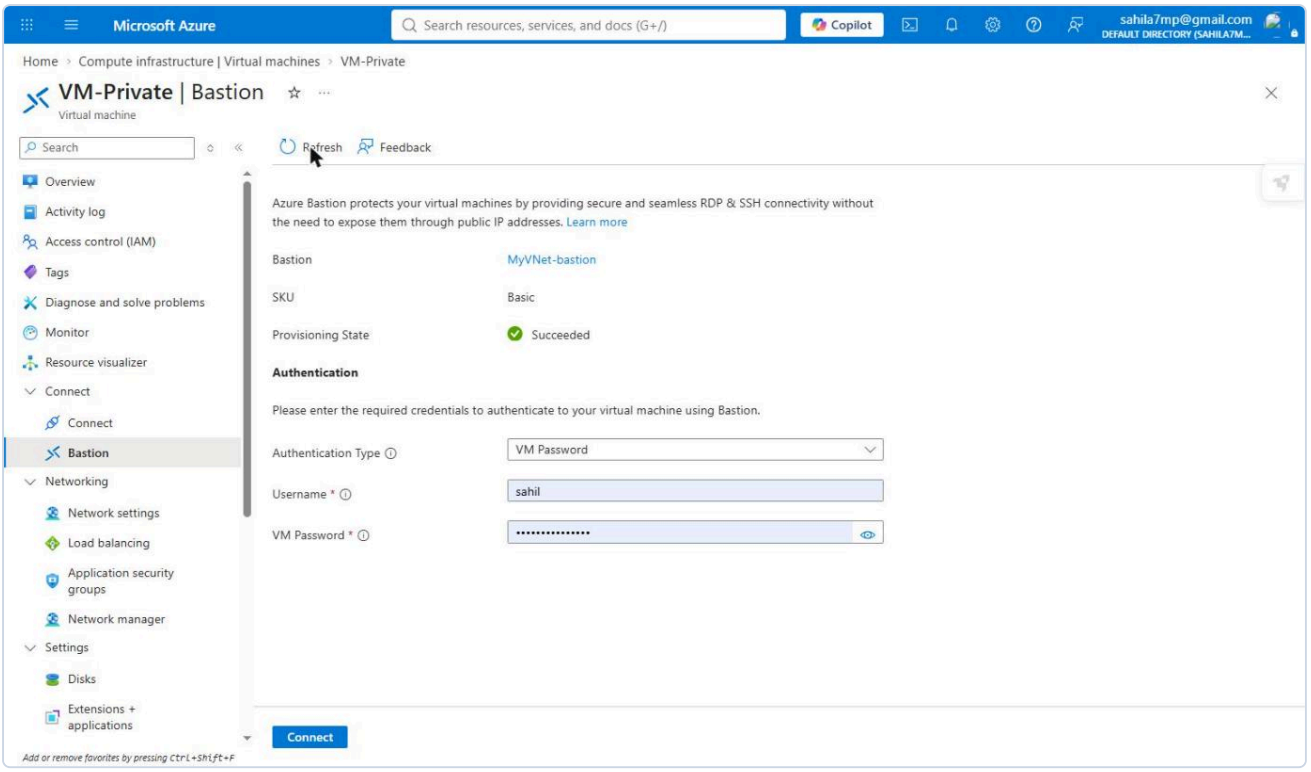
The VM Overview confirms two things at once: **Private IP address 10.0.2.10** — the static address from Part 6 — and **Public IP address 4.186.73.102 (NAT gateway MyNATGateway)**, while **Primary NIC public IP** remains **–**.

> **KEY CONCEPT**
>
> Read those two public-IP fields carefully, because they are easy to confuse. The NAT Gateway address is the subnet's shared *outbound* identity; **Primary NIC public IP** is the *inbound* one, and its dash is what means “unreachable from the internet”. Outbound presence and inbound exposure are independent.

45 Authenticate

WHAT WAS DONE

Set **Authentication Type** to **VM Password** and entered the `sahil` account credentials, then clicked **Connect**.



WHAT THE RESULT SHOWS

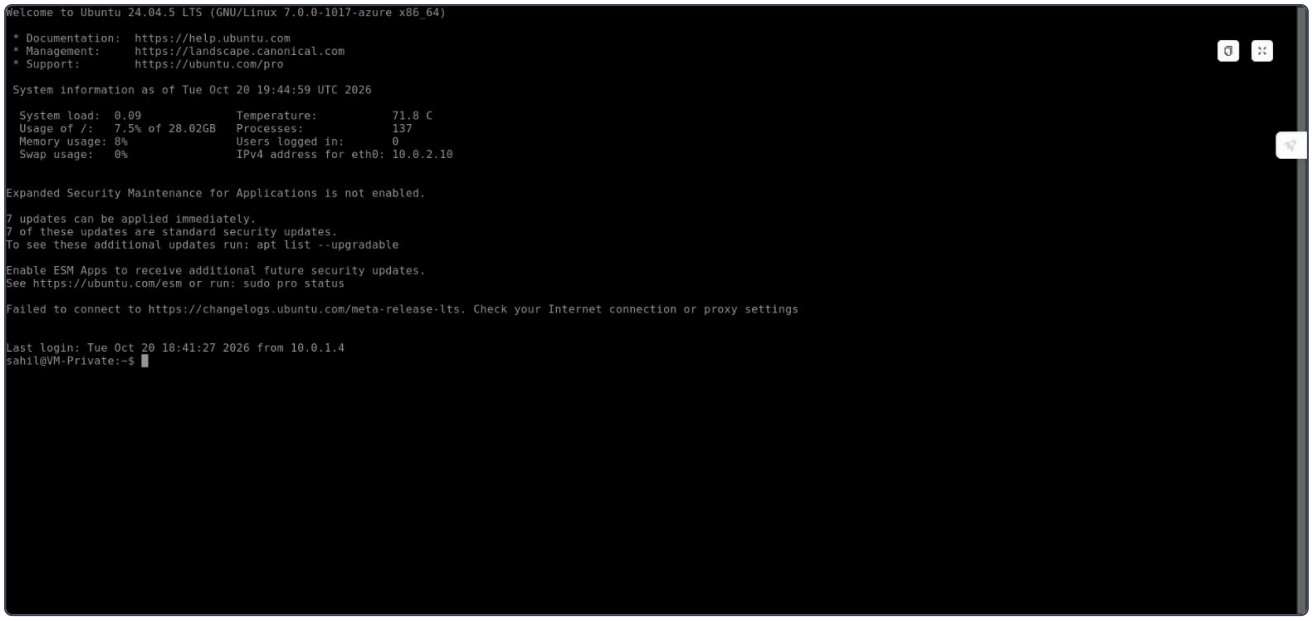
The blade confirms Bastion `MyVNet-bastion`, SKU Basic, Provisioning State Succeeded, with the credential form filled and the breadcrumb reading **VM-Private | Bastion**.

> **KEY CONCEPT**
>
> Bastion accepts a VM password, an SSH private key pasted inline, a key file from local disk, or a key stored in Azure Key Vault. The credentials travel to the Bastion host over TLS and are used for the SSH session it opens inside the VNet — the VM's SSH port is never published.

46 TEST 5 — a shell in the browser, with no public IP PASS

WHAT WAS DONE

Clicked Connect. A new browser tab opened on `bst-….bastion.azure.com` carrying a full terminal.



WHAT THE RESULT SHOWS

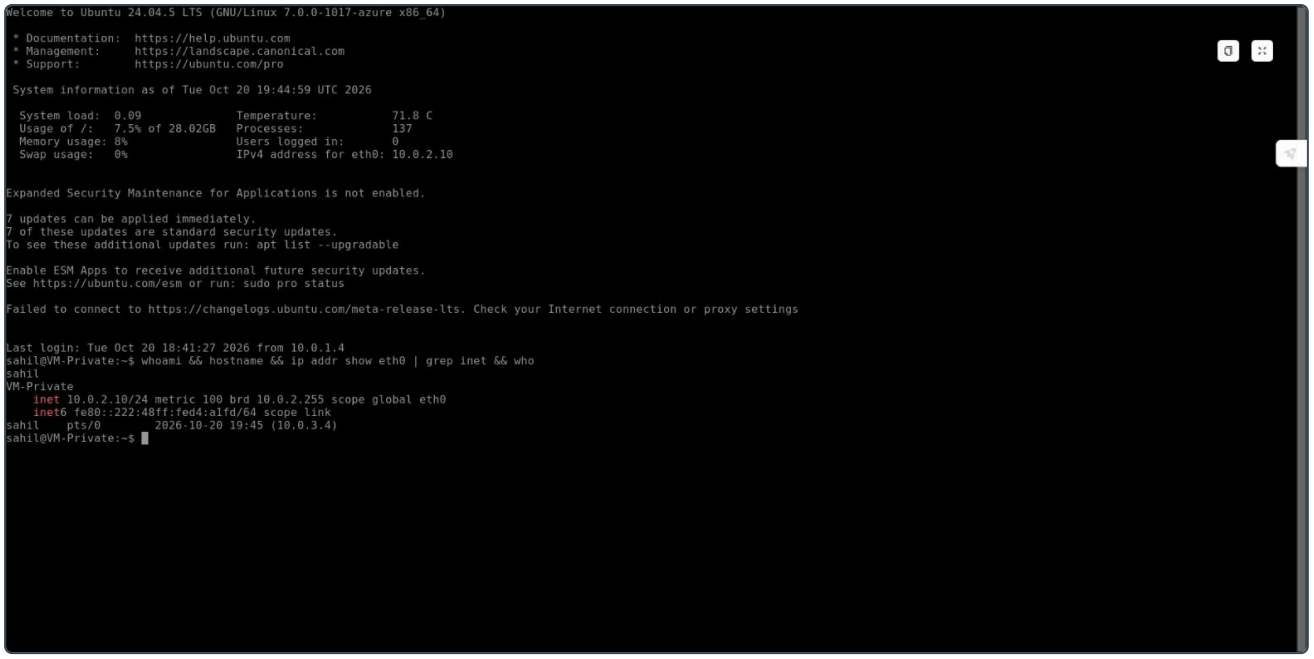
Logged straight in as `sahil@VM-Private`. The Ubuntu banner reports **IPv4 address for eth0: 10.0.2.10**, and the previous login is recorded as coming *from 10.0.1.4* — the jump-host session of Step 26.

> **KEY CONCEPT**
>
> This is the Bastion payoff. The browser speaks **HTTPS on 443** to a Microsoft-managed endpoint; Bastion then opens SSH *from inside the VNet* to the VM's private address. The VM never needs a public IP and port 22 is never exposed to the internet. The banner's failure to reach `changelogs.ubuntu.com` is only Ubuntu's MOTD check over HTTPS, unrelated to the session.

47 Where the connection actually came from PASS

WHAT WAS DONE

Ran `whoami && hostname && ip addr show eth0 | grep inet && who`.



WHAT THE RESULT SHOWS

`sahil` / `VM-Private` / `inet 10.0.2.10/24`, and the decisive line from `who`:

`sahil pts/0 2026-10-20 19:45 (10.0.3.4)`

KEY CONCEPT

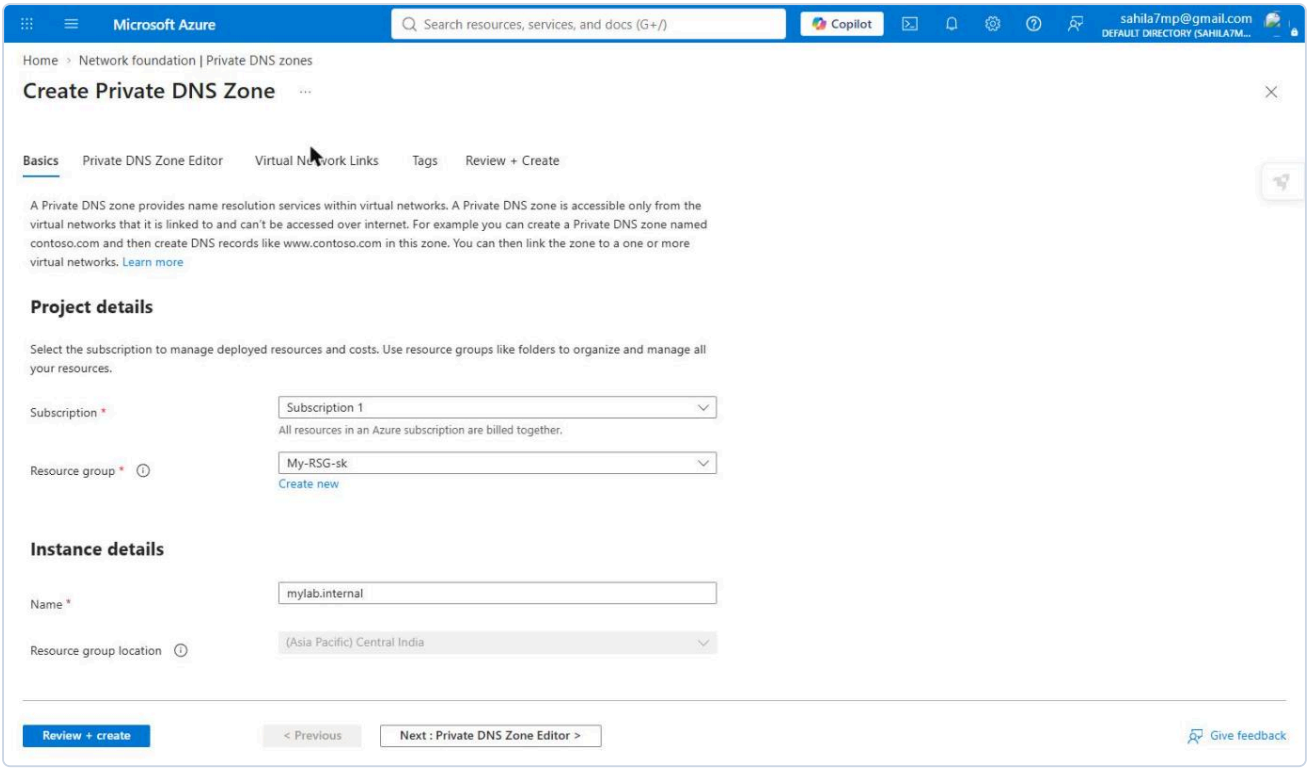
10.0.3.4 is an address in AzureBastionSubnet — proof that the SSH packets originated inside the VNet, not from the internet. And because 10.0.3.0/26 falls within the VNet's address space, the `VirtualNetwork` service tag already covered it: `Allow-SSH-VNet` from Step 15 permitted Bastion with **no new NSG rule written**. That is the dividend of scoping a rule by service tag rather than by hard-coded IP ranges.

Part 8 — Azure Private DNS Zone

48 Create the private zone mylab.internal

WHAT WAS DONE

Opened **Private DNS zones → + Create**. Resource group `My-RSG-sk`, name `mylab.internal`.



WHAT THE RESULT SHOWS

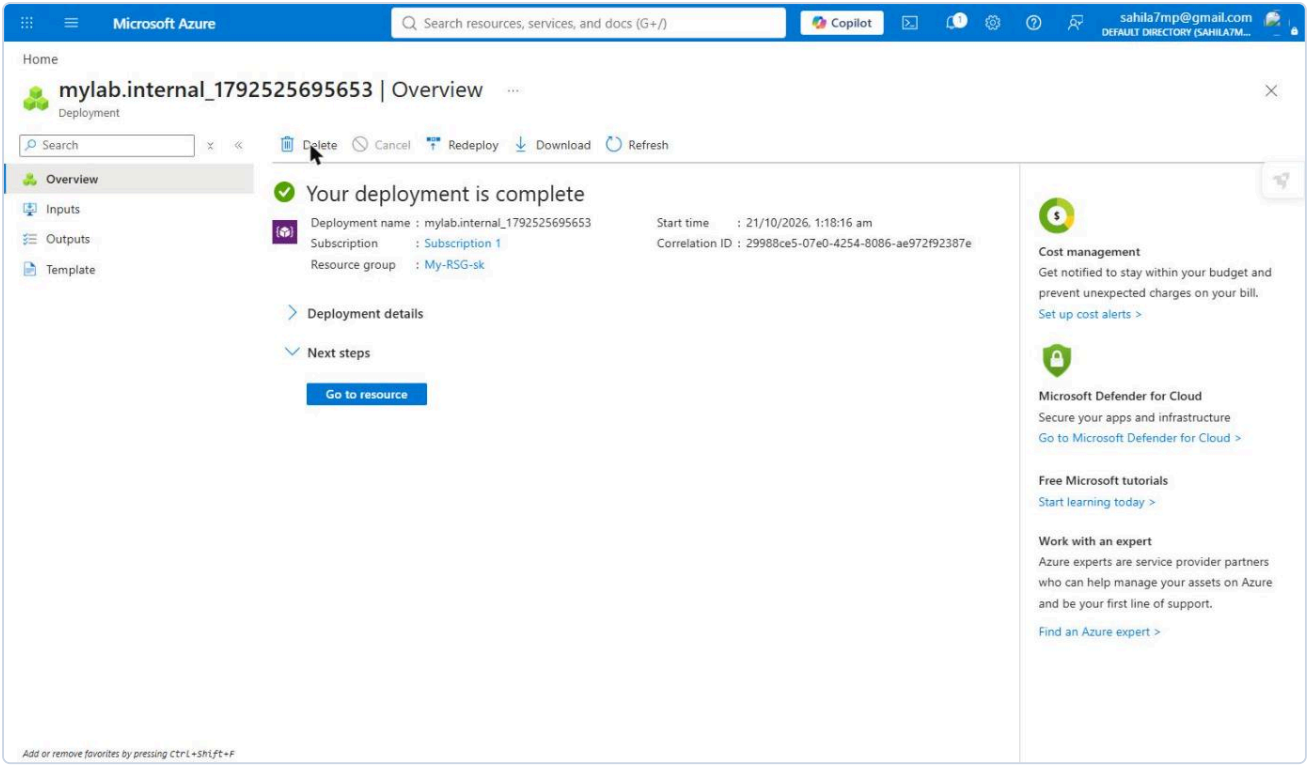
The Basics tab is complete. Note that **Resource group location** is greyed out — a private DNS zone is a **global** resource and has no region of its own.

> **KEY CONCEPT**
> A private DNS zone resolves names only for the virtual networks it is linked to and is unreachable from the internet, which is why any name can be used — `.internal`, `.local`, or a real domain you own. Avoid `.local` in practice, as it collides with mDNS on many Linux distributions.

49 Zone deployed

WHAT WAS DONE

Clicked **Review + Create → Create**.



WHAT THE RESULT SHOWS

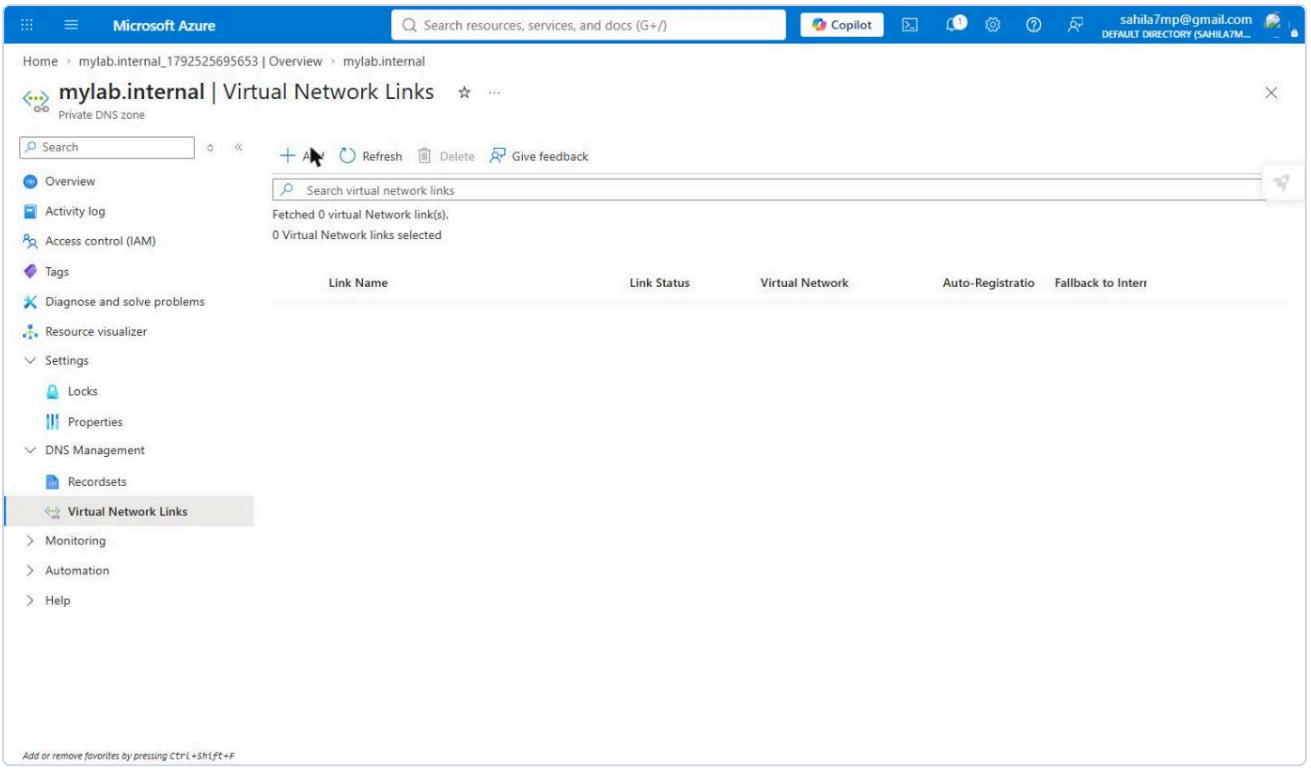
Your deployment is complete, deployed into resource group `My-RSG-sk`. The zone itself provisions in seconds.

> **KEY CONCEPT**
>
> The zone is now an empty namespace. On its own it resolves nothing — it needs a **virtual network link** before any VM can query it. The zone and the link are deliberately separate resources, which is what lets one zone serve many VNets.

50 Troubleshooting: the link was never created

WHAT WAS DONE

Opened **DNS Management → Virtual Network Links** to confirm the link configured during the create wizard.



WHAT THE RESULT SHOWS

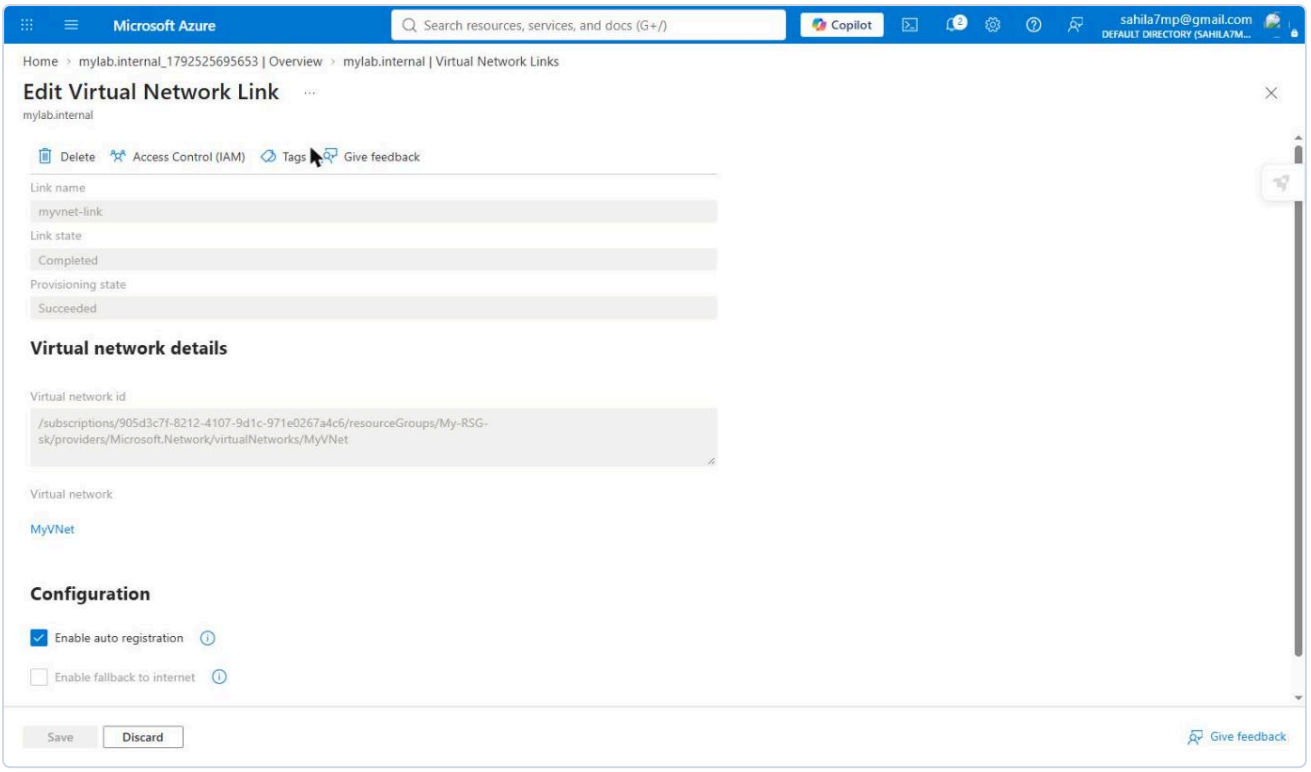
Fetched 0 virtual Network link(s). The list is empty — the wizard's Virtual Network Links tab had not been committed.

> **KEY CONCEPT**
>
> A second verification habit worth keeping, alongside the NSG association check in Step 16. A multi-tab create wizard does not always persist every tab, and a zone with no link fails silently: queries simply fall through to Azure's default DNS and return nothing for your names. Always open the resource afterwards and confirm what was actually written.

51 Link MyVNet with auto-registration

WHAT WAS DONE

Clicked **+ Add** and created the link manually: name `myvnet-link`, virtual network `MyVNet`, and ticked **Enable auto registration**.



WHAT THE RESULT SHOWS

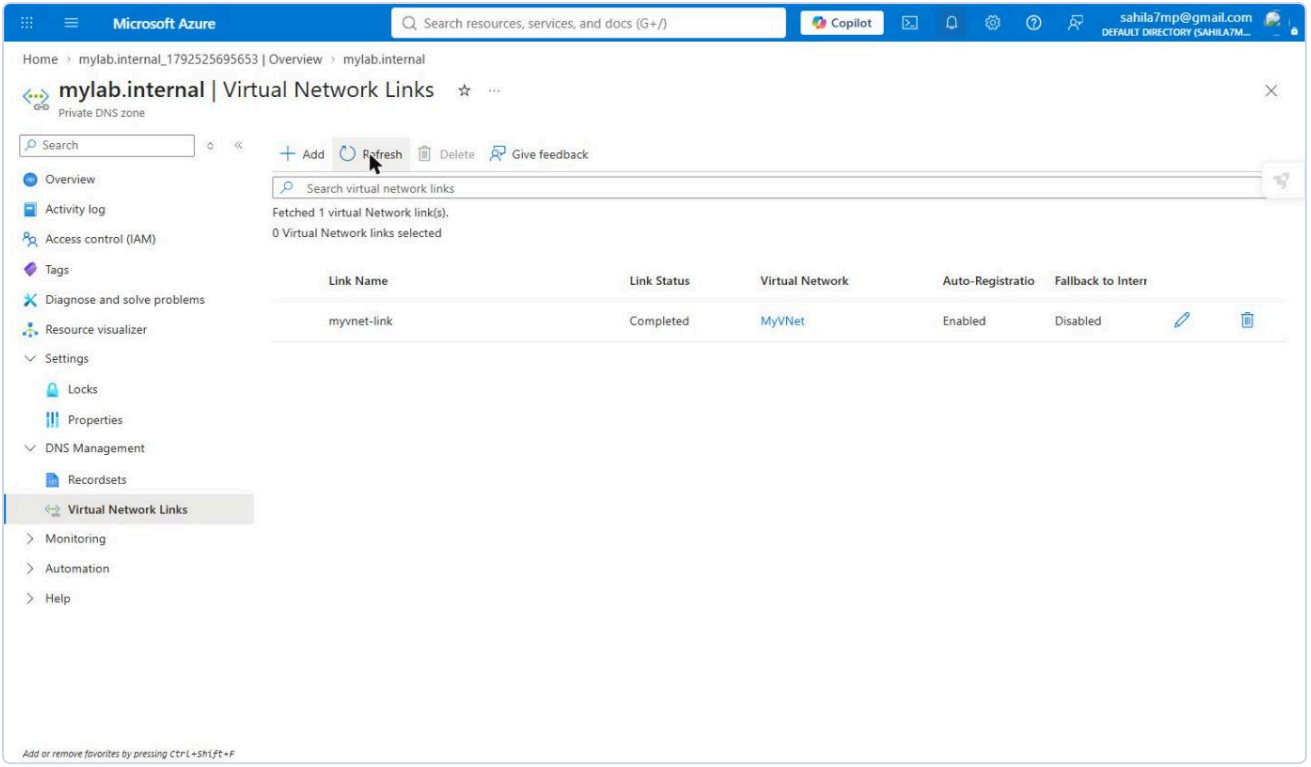
The link detail shows **Link state: Completed**, **Provisioning state: Succeeded**, virtual network `MyVNet`, and **Enable auto registration** checked. *Enable fallback to internet* is left off.

> **KEY CONCEPT**
>
> **Auto-registration is the feature that makes this zone self-maintaining.** With it on, Azure writes and updates an A record for every VM in the linked VNet automatically — no manual records, and the record follows the VM if its address ever changes. A VNet can be linked to many zones, but to only **one** zone with auto-registration enabled.

52 Link reaches Completed

WHAT WAS DONE

Returned to the list and refreshed while Azure finished provisioning.



WHAT THE RESULT SHOWS

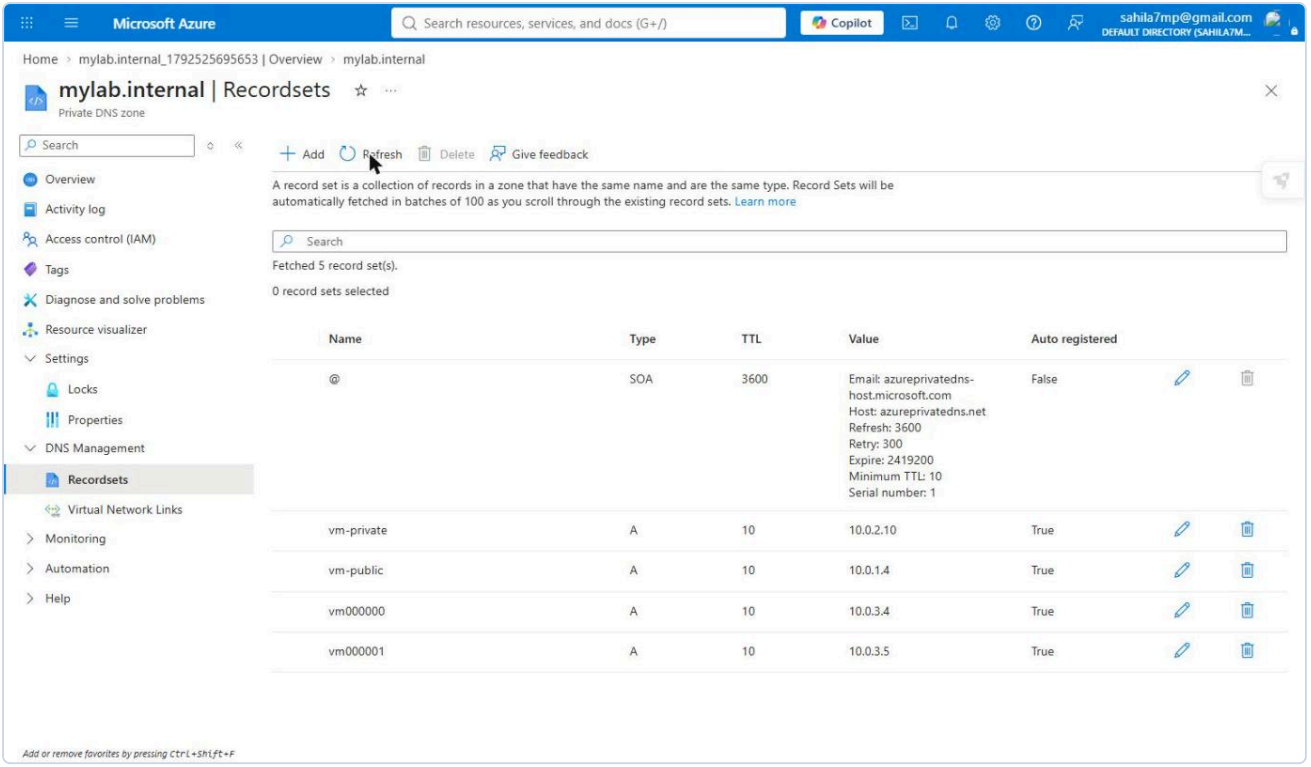
`myvnet-link` — Link Status **Completed**, Virtual Network **MyVNet**, Auto-Registration **Enabled**, Fallback to internet **Disabled**.

> **KEY CONCEPT**
>
> The status passes through `InProgress` first; records do not appear until it reads `Completed`. *Fallback to internet*, left disabled, would allow a name that misses in this zone to be retried against public DNS — useful in split-horizon setups, unnecessary here.

53 Records appear by themselves

WHAT WAS DONE

Opened **DNS Management → Recordsets**.



WHAT THE RESULT SHOWS

Five record sets, written with no manual input:

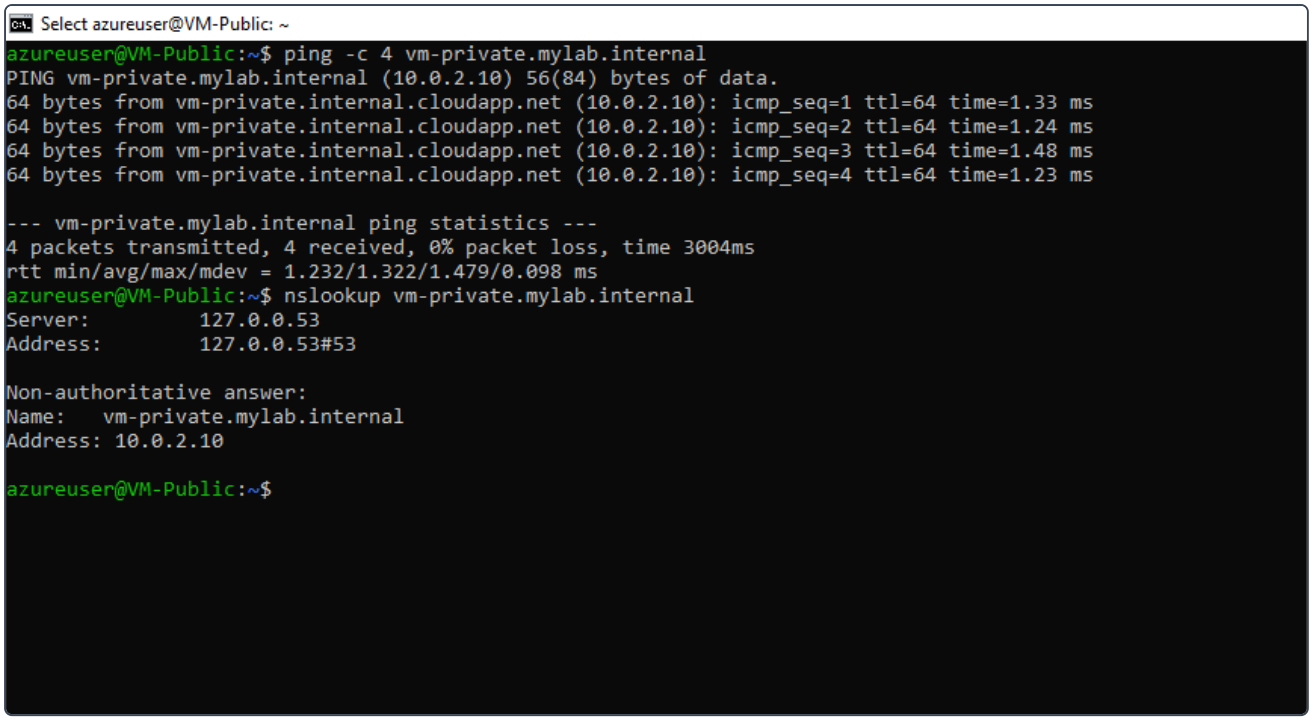
`@` SOA · TTL 3600 · auto-registered **False**
`vm-private` A · TTL 10 · **10.0.2.10** · **True**
`vm-public` A · TTL 10 · **10.0.1.4** · **True**
`vm000000` A · TTL 10 · **10.0.3.4** · **True**
`vm000001` A · TTL 10 · **10.0.3.5** · **True**

> **KEY CONCEPT**
>
> Three things to read here. `vm-private` resolves to **10.0.2.10**, tying Part 6's static address to Part 8's DNS. **`vm000000` and `vm000001` are the Bastion host instances** — Azure runs Bastion as a redundant pair inside your subnet, and 10.0.3.4 is exactly the address `who` reported in Step 47. And the **10-second TTL** on auto-registered records (against 3600 on the SOA) is deliberate: it lets a changed VM address propagate almost immediately.

54 TEST 6 — reach VM-Private by name PASS

WHAT WAS DONE

From VM-Public, ran `ping -c 4 vm-private.mylab.internal` followed by `nslookup vm-private.mylab.internal`.



WHAT THE RESULT SHOWS

`PING vm-private.mylab.internal (10.0.2.10)` — **4 packets transmitted, 4 received, 0% packet loss**, round-trip around **1.3 ms**. `nslookup` returns Server `127.0.0.53` and `Name: vm-private.mylab.internal / Address: 10.0.2.10`.

> **KEY CONCEPT**
>
> The full chain in one command. `127.0.0.53` is Ubuntu's systemd-resolved stub, which forwards to Azure's platform DNS at **168.63.129.16** — a fixed virtual address reachable from every VNet — which consults the linked private zone. The **1.3 ms** round trip confirms the traffic stayed east-west inside the VNet. ICMP succeeded even though NSG-Private only names TCP/22, because the default `AllowVnetInBound` rule at priority 65000 permits all intra-VNet protocols. The reply text shows `vm-private.internal.cloudapp.net` because reverse lookups still use Azure's default suffix — forward and reverse resolution are configured separately.

Results & Concept Summary

Six tests were run against the finished architecture. Each one isolates a single control, so a pass or a failure points at exactly one piece of configuration.

Verification matrix

TEST	RESULT	WHY
Internet → VM-Public, SSH :22	✓ **Allowed**	`Allow-SSH` priority 100 in NSG-Public is evaluated before `DenyAllInBound` 65500.
Internet → VM-Private, SSH :22	✗ **Impossible**	No public IP to target, and an internet source cannot match the `VirtualNetwork` service tag.
VM-Public → VM-Private, SSH :22	✓ **Allowed**	Source 10.0.1.4 falls inside `VirtualNetwork`, so `Allow-SSH-VNet` matches.
VM-Private → internet, before NAT	✗ **Timed out**	“Enable private subnet” removed the default outbound path; DNS resolved but TCP had no route.
VM-Private → internet, after NAT	✓ **33.3 MB in 4s**	MyNATGateway supplies outbound-only egress via `nat-pip2`, shared by the subnet.
Browser → VM-Private via Bastion	✓ **Shell opened**	HTTPS to the Bastion endpoint; Bastion then SSHes from 10.0.3.4, inside the `VirtualNetwork` tag.
VM-Public → `vm-private.mylab.internal`	✓ **4/4, ~1.3 ms**	Auto-registered A record resolves to 10.0.2.10; `AllowVnetInBound` permits intra-VNet ICMP.

Concepts covered

CONCEPT	WHAT IT MEANS IN THIS LAB
Resource Group	A logical container for lifecycle, billing and RBAC. Not a network or security boundary.
VNet & CIDR	Region-bound private network. `/16` = 65,536 addresses, `/24` = 256. Subnets must not overlap; Azure reserves 5 addresses per subnet.
NSG rule evaluation	Prioritised list, lowest number first, first match wins. Custom rules use 100–4096. `DenyAllInBound` at 65500 closes inbound by default; `AllowInternetOutBound` at 65001 leaves outbound open by default.
Stateful filtering	An inbound allow automatically permits that session's return traffic. You do not write a matching outbound rule for SSH or HTTP.
Service tags	Microsoft-maintained named IP sets such as `VirtualNetwork`, `Internet` and `AzureLoadBalancer`. They update automatically — never hard-code the ranges.
Source vs destination	Scoping the **Source** is what restricts access. `Destination = Any` with `Source = VirtualNetwork` is a tight rule.
Attachment level	Subnet-level NSG covers every NIC in the subnet, including VMs deployed later. NIC-level NSGs stack on top — both must allow the packet. Prefer subnet level.
Private subnet flag	“Enable private subnet” removes Azure's *default outbound* access. It does not affect inbound, which is governed by the NSG and by whether a public IP exists.

CONCEPT	WHAT IT MEANS IN THIS LAB
Bastion / jump host	One hardened, monitored, internet-facing host; everything else reachable only laterally across the VNet. Shrinks the attack surface to a single auditable door.
NAT Gateway	Managed, outbound-only egress with a stable shared public IP. Attaches per subnet, scales SNAT ports per IP, and needs no User-Defined Route.
Defense in depth	VM-Private is protected twice over — no public IP *and* a VNet-scoped NSG rule. Either control alone would block external access.
Reading failures	`Ign` then `Err` with a **timeout** means DNS worked but no route existed. An NSG deny usually fails faster and differently.
Static vs dynamic IP	Dynamic leases are held only while the VM is allocated; deallocation returns the address to the pool. Static pins it to the NIC. Standard-SKU public IPs are always static.
Address reservations	Azure reserves five addresses per subnet and hands dynamic leases from the low end, so a static address like `.10` avoids future collisions.
Azure Bastion	Managed jump host inside the VNet. Browser speaks HTTPS/443 to Microsoft; Bastion opens SSH from a private address. Requires a subnet named exactly `AzureBastionSubnet`, minimum `/26`.
Service tags compound	Because AzureBastionSubnet sits inside 10.0.0.0/16, `Allow-SSH-VNet` covered Bastion with no new rule — the benefit of scoping by tag rather than by hard-coded ranges.
Private DNS zone	A global, internet-unreachable namespace. Resolves only for the VNets linked to it, so any domain name may be used.
Auto-registration	Azure writes and updates an A record per VM in the linked VNet. TTL 10s so changes propagate fast. One VNet may auto-register into only **one** zone.
Azure platform DNS	`168.63.129.16` is a fixed virtual address reachable from every VNet. The guest's `127.0.0.53` stub forwards to it, and it consults linked zones.
Verify, don't assume	Twice in this lab a save silently did nothing — the NSG association in Step 16 and the DNS link in Step 50. Open the resource afterwards and read what was written.

Resources created: resource group `My-RSG-sk`, virtual network `MyVNet` (10.0.0.0/16) with `Public-Subnet` (10.0.1.0/24), `Private-Subnet` (10.0.2.0/24) and `AzureBastionSubnet` (10.0.3.0/26); network security groups `NSG-Public` and `NSG-Private`; virtual machines `VM-Public` and `VM-Private` (static 10.0.2.10); `MyNATGateway` with public IP `nat-pip2`; `MyVNet-bastion`; and private DNS zone `mylab.internal` linked to MyVNet with auto-registration. **Azure Bastion and the NAT Gateway both bill an hourly base rate whether or not anyone is connected** — delete the resource group once finished to remove everything and stop further charges.